

DE GRUYTER

GRADUATE

Leonid Berlyand, Pierre-Emmanuel Jabin

MATHEMATICS OF DEEP LEARNING

AN INTRODUCTION TO FOUNDATIONAL MATHEMATICS
OF NEURAL NETS

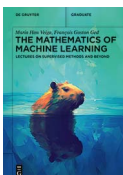
2ND EDITION



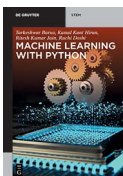
Leonid Berlyand, Pierre-Emmanuel Jabin

Mathematics of Deep Learning

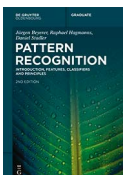
Also of Interest



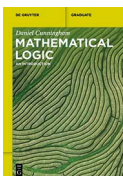
The Mathematics of Machine Learning
Lectures on Supervised Methods and Beyond
Maria Han Veiga, François Gaston Ged, 2024
ISBN 978-3-11-128847-5, e-ISBN (PDF) 978-3-11-128899-4



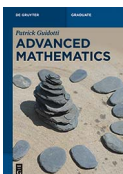
Machine Learning with Python
Tarkeshwar Barua, Kamal Kant Hiran, Ritesh Kumar Jain, Ruchi Doshi, 2024
ISBN 978-3-11-069716-2, e-ISBN (PDF) 978-3-11-069718-6



Pattern Recognition
Introduction, Features, Classifiers and Principles
Jürgen Beyerer, Raphael Hagmanns, Daniel Stadler, 2024
ISBN 978-3-11-133919-1, e-ISBN (PDF) 978-3-11-133920-7



Mathematical Logic
An Introduction
Daniel Cunningham, 2023
ISBN 978-3-11-078201-1, e-ISBN (PDF) 978-3-11-078207-3



Advanced Mathematics
An Invitation in Preparation for Graduate School
Patrick Guidotti, 2022
ISBN 978-3-11-078085-7, e-ISBN (PDF) 978-3-11-078092-5

Leonid Berlyand, Pierre-Emmanuel Jabin

Mathematics of Deep Learning

An Introduction to Foundational Mathematics of Neural
Nets

2nd revised and extended edition

DE GRUYTER

Mathematics Subject Classification 2020

47-01, 47A10, 47A30, 47A35, 47A56, 47A60, 47B07, 47B25, 46-01, 46A30, 46A35, 46A45, 46E15

Authors

Prof. Leonid Berlyand
Penn State University
Department of Mathematics
University Park, PA 16802
USA
lvb2@psu.edu

Prof. Pierre-Emmanuel Jabin
Penn State University
Department of Mathematics
109 McAllister
University Park, PA 16802
USA
pejabin@psu.edu

ISBN 978-3-11-914411-7
e-ISBN (PDF) 978-3-11-221821-1
e-ISBN (EPUB) 978-3-11-221838-9

Library of Congress Control Number: 2025947268

Bibliographic information published by the Deutsche Nationalbibliothek

The Deutsche Nationalbibliothek lists this publication in the Deutsche Nationalbibliografie; detailed bibliographic data are available on the Internet at <http://dnb.dnb.de>.

© 2026 Walter de Gruyter GmbH, Berlin/Boston, Genthiner Straße 13, 10785 Berlin
Cover image: metamorworks / iStock / Getty Images Plus
Typesetting: VTeX UAB, Lithuania
Printing and binding: CPI books GmbH, Leck

www.degruyterbrill.com
Questions about General Product Safety Regulation:
productsafety@degruyterbrill.com

Contents

1	About this book — 1
2	Introduction to machine learning: what and why? — 3
2.1	Some motivation — 3
2.2	What is machine learning? — 3
3	Classification problem — 5
4	The fundamentals of artificial neural networks (ANNs) — 7
4.1	Basic definitions — 7
4.2	ANN classifiers and the softmax function — 11
4.3	The universal approximation theorem — 13
4.4	Why is non-linearity in ANNs necessary? — 15
4.4.1	$0 + 0 = 8$? — 15
4.4.2	Non-linear activation functions are necessary in ANNs — 16
4.5	Why do we need biases? — 17
4.6	Exercises — 19
5	Supervised, unsupervised, and semi-supervised learning — 22
5.1	Basic definitions — 22
5.2	Example of unsupervised learning: detecting bank fraud — 24
5.3	Exercises — 25
6	The regression problem — 27
6.1	What is regression? How does it relate to ANNs? — 27
6.2	Example: linear regression in dimension 1 — 29
6.3	Logistic regression as a single neuron ANN — 30
6.3.1	1D example: studying for an exam — 30
6.3.2	2D example of admittance to graduate school: separation of sets and decision boundary — 33
6.3.3	Relation between ANNs and regression — 35
6.3.4	Logistic regression vs. networks with many layers — 36
6.4	Exercises — 39
7	Support vector machine — 43
7.1	Preliminaries: convex sets and their separation, geometric Hahn–Banach theorem — 43
7.2	Support vector machine — 45
7.3	Hard-margin SVM classifiers and support vectors — 47

7.4	Soft margin SVM classifier —	49
7.5	Exercises —	51
8	Kernel methods —	54
8.1	Kernels: what/why? —	54
8.1.1	Recalling the basic principles of linear regression —	54
8.1.2	How a linear kernel arises in linear regression —	56
8.1.3	Linear regression in general dimension —	57
8.1.4	Ridge regression —	58
8.1.5	Solving the ridge regression by the dual problem —	59
8.1.6	How linear kernel arises in ridge regression —	60
8.1.7	Feature space and feature map in classification and regression: examples —	61
8.2	Kernel: definitions and basic properties —	65
8.2.1	Definitions —	65
8.2.2	The “kernel trick”: connecting feature maps and kernels —	66
8.2.3	Choosing the right kernel: the example of the Gaussian kernel —	68
8.3	Exercises —	69
9	Gradient descent method in the training of DNNs —	71
9.1	Deterministic gradient descent for the minimization of multivariable functions —	72
9.2	Additive loss functions —	75
9.3	What are SGD algorithms? When to use them? —	76
9.4	Epochs in SGD —	77
9.5	Weights —	78
9.6	Choosing the batch size through a numerical example —	79
9.7	Exercises —	82
10	Backpropagation —	86
10.1	Computational complexity —	86
10.2	Chain rule review —	91
10.3	Diagrammatic representation of the chain rule in simple examples —	92
10.4	The case of a simple DNN with one neuron per layer —	97
10.5	Backpropagation algorithm for general DNNs —	102
10.6	Exercises —	107
11	Convolutional neural networks (CNNs) —	112
11.1	Convolution —	112
11.1.1	Convolution of functions —	112
11.1.2	Convolution of matrices —	113
11.1.3	Hadamard product and feature detection —	115

11.2	Convolutional layers —	120
11.3	Padding layer —	123
11.4	Pooling layer —	124
11.5	Building CNNs —	126
11.6	Equivariance and invariance —	128
11.7	Summary of CNNs —	138
11.8	Exercises —	139

A Review of the chain rule — 143

Bibliography — 145

Index — 149

1 About this book

This course aims at providing a mathematical perspective to some key elements of the so-called deep neural networks (DNNs). Much of the interest on deep learning has focused on the implementation of DNN-based algorithms. Our hope is that this compact textbook will offer a complementary point of view that emphasizes the underlying mathematical ideas. We believe that a more foundational perspective will help answer important questions that have only received empirical answers so far.

These notes are based on a one-semester course “Introduction to Mathematics of Deep Learning” for senior undergraduate mathematics majors and first-year graduate students in mathematics.

Our goal is to introduce basic concepts from deep learning in a rigorous mathematical fashion, e.g., mathematical definitions of deep neural networks (DNNs), loss functions, the backpropagation algorithm, etc. For each concept, we attempt to identify the simplest setting that minimizes technicalities but still contains the key mathematics.

The book focuses on deep learning techniques and introduces them almost immediately. Other techniques such as regression and SVM are briefly introduced and used as a stepping stone for explaining basic ideas of deep learning. The rigorous definitions and statements are supplemented by heuristic explanations and figures.

The book is organized so that each chapter introduces a key concept. When teaching this course, some chapters could be presented as a part of a single lecture, whereas the others have more material and would take several lectures.

This book came about from two special topic courses taught at Penn State, first for Ph.D. students and two years later for senior undergraduates and master students. The idea behind those courses was to translate some of the computer science concepts for deep learning into some simple mathematical language, more accessible to math students. The lecture notes from these courses became the ground material for this book.

The preparation of this manuscript has been a fun undertaking, involving numerous Penn State students. Here is the list of Ph.D. students to whom we are extremely grateful for many insightful discussions in the preparation of the manuscript: Hai Chi, Robert Creese, Spencer Dang, Jonathan Jenkins, Oleksii Krupchytskyi, Alex Safsten, Yitzchak Shmalo, and Datong Zhou. Alex, Datong, Hai, Robert, Spencer, and Yitzchak took the courses in deep learning and helped in typing the lecture notes. Furthermore, Alex, Datong, Spencer, and Oleksii provided invaluable help in creating figures. It is also a great pleasure to thank Alex and Yitzchak for assisting in the preparation of the exercises for this book. Alex, Jonathan, Oleksii, Spencer, and Yitzchak provided very efficient and timely help in proofreading the manuscript.

We express our special gratitude to Alex Safsten for his contributions to both our teaching and research on deep learning and, in particular, for suggesting several exam-

ples for these lecture notes. The authors have also made the following recent contributions in the research of deep learning foundations and algorithms: [5, 6, 42, 7, 4].

Finally, the authors acknowledge partial support of their work on the mathematics of deep learning from the National Science Foundation.

2 Introduction to machine learning: what and why?

2.1 Some motivation

Machine-learning based artificial intelligence has entered our daily lives in many various ways such as

- Self-driving cars
- Speech and facial recognition
- Google search
- Product (Amazon) and content (Youtube) recommendations

with ever further reaching impacts being developed.

What may be less apparent is the increasing role taken by machine learning algorithms in classical scientific computing questions. A recent seminal example concerned the highly accurate protein structure prediction with AlphaFold in [32]. Proteins are long sequences of amino acids—a type of organic molecule (organic molecules are based on carbon–hydrogen bonds). They are critical to many biochemical processes in living organisms with a prominent example being the DNA replication for cell division.

A long standing problem (since 1960) is the question of how proteins fold. That is, how does the amino acid sequence composing the protein determine its 3D atomic structure? This structure provides the biochemical effects of the protein in the organism and is essential to understand their properties.

Mathematically, the protein folding problem can be formulated either as a dynamical problem or as an energy minimization problem: the protein assumes the shape that minimizes the energy stored in atomic bonds. This problem presents a great computational challenge because proteins are extremely complicated molecules consisting of tens or hundreds of thousands of atoms. Moreover, the problem of cataloging how all proteins fold through laboratory experiments is infeasible because of the many ways a protein can fold and the huge number of proteins. In 2021, Google’s machine learning algorithm *AlphaFold* was the first computational method to achieve accuracy higher than 95 % in predicting protein folding.

2.2 What is machine learning?

We of course refer to several common terms such as artificial intelligence or machine learning. Because there is not necessarily a consensus on the meaning of those terms, we specify below in which sense they are used in this book.

- *Artificial intelligence* (AI) is (according to Wikipedia) “intelligence demonstrated by machines as opposed to natural intelligence demonstrated by animals including humans”. One of the ideas behind AI is to try to mimic the cognitive function of a brain, e. g., “learning” and problem solving.

- *Machine learning* (ML) is a subfield of AI. In a non-learning algorithm, all parameters and methods used to produce the output are predetermined by the programmer. In contrast, machine learning algorithms use at least some parameters that are not fixed. Instead, the algorithms improve themselves (“learn”) by refining their parameters and methods through experience such as comparison of its output with known data. Because of that, a machine learning typically goes through a distinct training phase before it is actually used to solve problems.

3 Classification problem

As a first in-depth example of an important practical problem, consider *the classification problem* of placing objects into categories, also called classes. For example, we may classify images of handwritten digits by determining which digit is depicted.

In a rigorous mathematical formulation of this problem, objects are represented as points $\mathbf{s} \in \mathbb{R}^n$. For example, images of handwritten digits can be represented by a vector of their n pixel values, and the i th pixel is represented by the coordinate $\mathbf{s}_i$. In a simple setting of grayscale (black and white) images, $\mathbf{s}_i$, $i = 1, \dots, n$, take values in $[0, 1]$, which represent the lightness of the pixels from 0 (black) to 1 (white). We denote by $S \subset \mathbb{R}^n$ the set of all objects to be considered; S may be finite or infinite. In practice, we start from some finite subset T of S of objects, which are already classified, and use this information to classify objects from the entire set S , which is typically much larger than T . The set T is called the *training set* because the classification method is “trained” on this set to classify objects that are not in T .

The classes correspond to subsets $S_j \subset S$ indexed by integers $j = 1, \dots, m$. For example, in the above problem of handwritten digits, there are clearly 10 disjoint classes. They are naturally pairwise disjoint:

$$S_j \cap S_k = \emptyset \quad \text{for } j \neq k, \quad (3.1)$$

and together they make up all of S :

$$\bigcup_{j=1}^m S_j = S. \quad (3.2)$$

The function $\hat{\phi} : S \rightarrow \{1, \dots, m\}$ that maps each object to the index of its correct class is called the *exact classifier*. This is an example of a so-called *hard classifier*, which is a function that assigns (maps) each object to a class (if the classifier is not exact, then it need not be the correct class). In other words, the range of the hard classifier function is a collection of classes.

In contrast, it is frequently more convenient to use a *soft classifier*, which assigns to each object $\mathbf{s} \in S$ a vector

$$\mathbf{p}(\mathbf{s}) = (p_1(\mathbf{s}), \dots, p_m(\mathbf{s})), \quad (3.3)$$

where each component $p_i(\mathbf{s})$ represents the predicted probability that $\mathbf{s}$ is in class i . An exact soft classifier would map each object $\mathbf{s}$ in class i to the vector $(p_1(\mathbf{s}), \dots, p_m(\mathbf{s}))$ with $p_{i(\mathbf{s})} = 1$ and $p_j(\mathbf{s}) = 0$ for $j \neq i(\mathbf{s})$, where

$$i(\mathbf{s}) = \hat{\phi}(\mathbf{s}) \quad (3.4)$$

is the index of the correct class (which is known for $\mathbf{s} \in T$) of the object $\mathbf{s}$.

In practical applications, it is typically not possible to find the exact classifier. Instead, the goal is to construct an approximation that depends on a set of parameters α and optimize the choice of these parameters. This can of course lead to a hard or soft classifier. In that regard, it is useful to note that any *soft approximate classifier* $\phi : \mathbb{R}^n \rightarrow \mathbb{R}^m$ can be transformed into a hard classifier $\bar{\phi} : \mathbb{R}^n \rightarrow \{1, \dots, m\}$ by taking $\bar{\phi}(s) = i$ if $p_i(s)$ is the largest component of $\phi(s)$.

4 The fundamentals of artificial neural networks (ANNs)

4.1 Basic definitions

In this course, we focus on *deep learning*, which is a type of ML based on *artificial neural networks* (ANNs) that involve several so-called layers of neuron functions. This kind of architecture was loosely inspired from the biological neural networks in animal brains.

The key building block of ANNs is what we call here a *neuron function*. It is a very simplified mathematical representation of biological neurons dating back to [46]. This naturally leads to various ways of organizing those neuron functions into interacting networks that are capable of complex tasks.

The perceptron, introduced in [56] (see also [47]), is one of the earliest constructions of a one-layer ANN. More recently, multiple layers are typically used instead with the idea that neurons in the same layer are not connected but may interact with neurons in other layers.

We present in this chapter a rather common architecture for classifiers, which is based on so-called *feedforward networks*, networks with no cycles. Here a cycle is when the output of one layer becomes the input of the previous layer. More complex networks can also be used with some cycles between layers (so-called *recurrent* neural networks; see, for example, [19, 26]). We also refer to [8, 16, 18, 58] for a presentation of the main mathematical tools required for machine learning and to [48] for a probabilistic/statistical point of view. The readers interested in practical implementations can, for example, check [71].

Finally, as we readily see, ANNs typically require a very large number of parameters. This makes identifying the best choice for those coefficients delicate and leads to the important question of learning the parameters, discussed later in this book.

Definition 4.1.1. A *neuron function* $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is a mapping of the form

$$f(\mathbf{x}) = \lambda(\mathbf{a} \cdot \mathbf{x} + \beta), \quad (4.1)$$

where $\lambda : \mathbb{R} \rightarrow \mathbb{R}$ is a continuous non-linear function called the *activation function*, $\mathbf{a} \in \mathbb{R}^n$ is a vector of *weights*, and a scalar $\beta \in \mathbb{R}$ is called the *bias*. Here $\mathbf{a} \cdot \mathbf{x}$ is the inner product on $\mathbb{R}^n$.

A typical example of an activation function is *ReLU* (*Rectified Linear Unit*) defined by the following formula (see Figure 4.1):

$$\text{ReLU}(x) = \begin{cases} x, & x > 0, \\ 0, & x \leq 0. \end{cases} \quad (4.2)$$

ReLU is a very simple non-linear function composed of two linear functions that model the threshold between “inactive” and “active” states of the neuron.

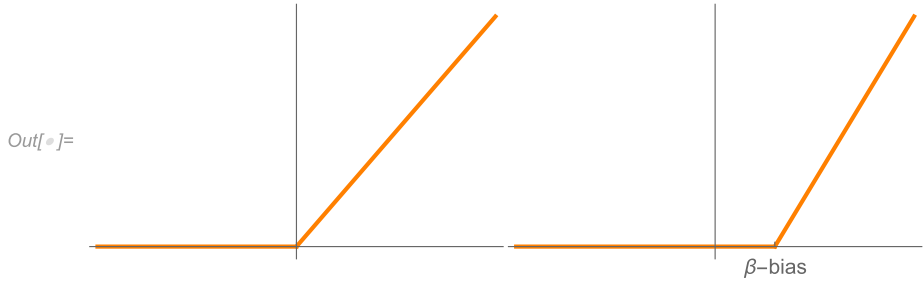


Figure 4.1: ReLU function, β -“bias” constant.

ReLU is a simple way to model a neuron in a brain with two states, resting (resting potential) and active (being fired). Incoming impulses from other neurons can change the state from resting to active, but the impulse must first reach a certain threshold. Thus ReLU’s change from constant 0 to a linear function at $x_{\text{thresh}} = \beta$ reflects the change from resting to active at this threshold.

Since the output of a neuron function is a single number, we can combine several neurons to create a vector-valued function called a *layer function*:

Definition 4.1.2. A *layer function* $g : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is a mapping of the form

$$g(\mathbf{x}) = (f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_m(\mathbf{x})), \quad (4.3)$$

where each $f_i : \mathbb{R}^n \rightarrow \mathbb{R}$ is a neuron function of the form (4.1) with its own vector of parameters $\mathbf{a}_i = (a_{i1}, \dots, a_{in})$ and biases β_i , $i = 1, \dots, m$.

Remark 4.1.1. When $m = 1$ in (4.3), the layer function reduces to a neuron function, as in Figure 4.2.

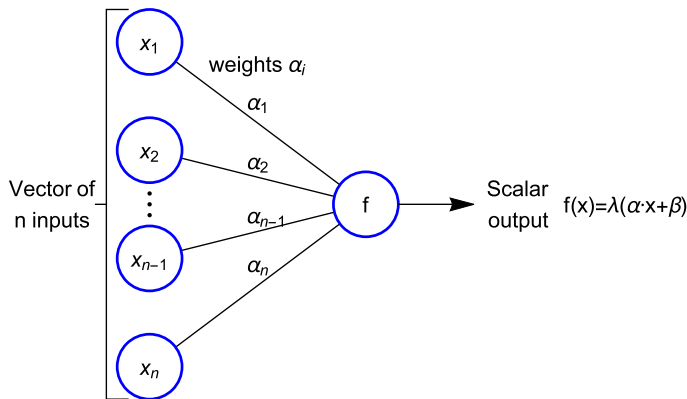


Figure 4.2: A neuron function input $\mathbf{x} \in \mathbb{R}^n$, vector of weights $\mathbf{a} = (a_1, \dots, a_n)$, bias β , output scalar $f(\mathbf{x}) = \lambda(\mathbf{a} \cdot \mathbf{x} + \beta)$.

Remark 4.1.2. When discussing layers, it is important to distinguish between the layer nodes and the layer function that connects them. There are two columns of nodes in Figure 4.3, which are commonly referred to as two layers. However, according to Definition 4.1.2, Figure 4.3 depicts a single layer function. Thus it is important to distinguish between columns of nodes in diagrammatic representations of layers as in Figure 4.3 and the layer function that connects them (defined in (4.3)). This is illustrated by Figure 4.3, where the nodes $\mathbf{x} = (x_1, x_2, x_3)$ can be referred to as the nodes of the input vector, and the output vector of the layer function is given by (4.3). In such a situation, we may commonly refer to two layers of nodes: the input layer composed of the nodes corresponding to the coordinates x_i of x and the output layer composed of the m nodes corresponding to the coordinates y_i of $y = g(x)$. For a general multi-layer network defined in Definition 4.1.3, if we consider M layer functions, then we will have $M + 1$ layers of nodes. We simply write layer for a layer of nodes and often refer to an individual node within a layer as a neuron (vs. the layer function or neuron function connecting those).

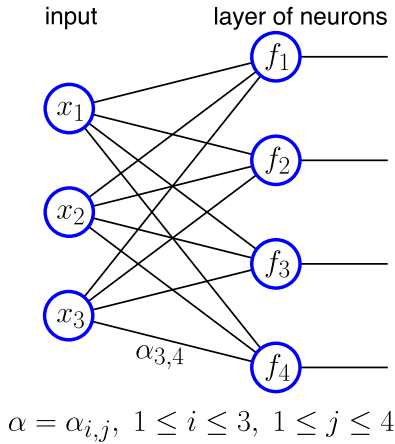


Figure 4.3: Layer of neurons $f_i, 1 \leq i \leq 4$ with vector of input data x_j . Weights a_{ij} assigned to each edge connecting x_j and $f_i, n = 3, m = 4$.

Thus the layer function is determined by a *matrix of parameters*

$$A = \begin{pmatrix} \alpha_{11} & \cdots & \alpha_{1n} \\ \vdots & \ddots & \vdots \\ \alpha_{m1} & \cdots & \alpha_{mn} \end{pmatrix} \quad (4.4)$$

and a *vector of biases*

$$\beta = \begin{pmatrix} \beta_1 \\ \vdots \\ \beta_m \end{pmatrix}. \quad (4.5)$$

Hence (4.3) may be written as

$$g(\mathbf{x}) = \bar{\lambda}(A\mathbf{x} + \boldsymbol{\beta}), \quad (4.6)$$

where $\bar{\lambda} : \mathbb{R}^m \rightarrow \mathbb{R}^m$ is the *vectorial activation function* defined as

$$\bar{\lambda}(x_1, \dots, x_m) = (\lambda(x_1), \dots, \lambda(x_m)) \quad (4.7)$$

for a scalar activation function λ as in Definition 4.1.1.

See Figure 4.3 for a diagram of a layer function. This figure shows a layer as a graph with two columns of nodes. The right column depicts neuron functions in the layer; whereas the left column depicts three real numbers (data), which are input to the layer. The edge connecting the i th input with the j th neuron is multiplied by the parameter value α_{ij} from the matrix of parameters.

Definition 4.1.3. An *artificial neural network* (ANN) is a function $h : \mathbb{R}^n \rightarrow \mathbb{R}^m$ of the form

$$h(\mathbf{x}) = h_M \circ h_{M-1} \circ \dots \circ h_1(\mathbf{x}), \quad M \geq 1, \quad (4.8)$$

where each $h_i : \mathbb{R}^{n_{i-1}} \rightarrow \mathbb{R}^{n_i}$ is a layer function (see Definition 4.1.2) with its own matrix of parameters A^i and its own vector of biases $\boldsymbol{\beta}^i$.

Figure 4.4 shows an ANN composed of two layers. The layer function in Figure 4.4 between input and output is called a *hidden layer* (hidden from the user) because its output is passed to another layer, not directly to the user. The number of neurons n_i in the i th layer is called the layer's *width*, whereas the total number M of layers in an

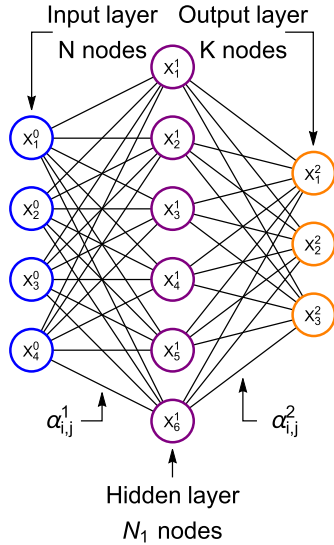


Figure 4.4: Simple network with input layer and two layer functions, $n = n_0 = 4$, $n_1 = 6$, $m = n_2 = 3$.

ANN is the *depth* of the ANN. The numbers $n_1, \dots, n_M$ and M comprise the *architecture* of this network. ANNs with more than one layer are referred to as *deep neural networks* (DNNs). This network is called *fully connected* because for all but the last layer, each neuron provides input to each neuron in the next layer; that is, each node in each column of the graph in Figure 4.4 is connected to each node in the next column. Finally, we mention that the last layer, corresponding to the result of the last layer function h_M , is commonly called the *output layer*. On the other hand, the input vector $\mathbf{x}$ is commonly referred to as the *input layer*. Neither input nor output layer is a layer function in the sense of Definition 4.1.2, and both are node layers; see Remark 4.1.2.

4.2 ANN classifiers and the softmax function

An ANN can be used to build a soft classifier function. Recall that a soft classifier outputs a list of probabilities that the input belongs to each class, that is, if $\phi(\mathbf{x}) = (\phi_1(\mathbf{x}), \dots, \phi_m(\mathbf{x}))$ is a soft classifier, then for every $\mathbf{x}$, we should have

$$0 \leq \phi_i(\mathbf{x}) \leq 1, \quad i = 1, \dots, m, \quad \text{and} \quad \sum_{i=1}^m \phi_i(\mathbf{x}) = 1. \quad (4.9)$$

For ANNs of the form given in Definition 4.1.3, (4.9) is difficult to satisfy. Therefore we define an *ANN classifier* as follows.

Definition 4.2.1. An *ANN classifier* is a function $\tilde{h} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ of the form

$$\tilde{h}(\mathbf{x}) = \sigma \circ h(\mathbf{x}), \quad (4.10)$$

where $h : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is an ANN, and $\sigma : \mathbb{R}^m \rightarrow \mathbb{R}^m$ is the *softmax function*, defined as

$$\sigma(\mathbf{y}) = \left(\frac{e^{y_1}}{\sum_{i=1}^m e^{y_i}}, \dots, \frac{e^{y_m}}{\sum_{i=1}^m e^{y_i}} \right). \quad (4.11)$$

Observe that the softmax function “normalizes” the output of $\tilde{h}$ so that (4.9) is automatically satisfied. Moreover, since $y \mapsto e^y$ is monotone increasing, softmax is order preserving in the sense that if $\mathbf{y} \in \mathbb{R}^m$ and $y_i > y_j$, then $(\sigma(\mathbf{y}))_i > (\sigma(\mathbf{y}))_j$. Thus the largest component of $h(\mathbf{x})$ is also the class with highest probability in $\tilde{h}(\mathbf{x})$.

The softmax function is of course not the only way to normalize the output, but for classification problems it is very commonly used.

In summary, an ANN is a function obtained by an iterative composition of affine and non-linear functions.

Example 4.2.1. Consider an ANN function $h : \mathbb{R}^3 \rightarrow \mathbb{R}^2$ with one hidden layer of five neurons. Suppose that all neuron functions in both function layers use ReLU for their activation functions and have the following matrices of parameters and bias vectors:

$$A^1 = \begin{pmatrix} 4 & 2 & -5 \\ 5 & 4 & 6 \\ 0 & 3 & 2 \\ 2 & 2 & -3 \\ -7 & 1 & -4 \end{pmatrix}, \quad A^2 = \begin{pmatrix} -2 & 6 & 2 & -5 & 0 \\ -3 & 0 & 6 & 1 & 2 \end{pmatrix}, \quad (4.12)$$

$$\mathbf{b}^1 = \begin{pmatrix} -2 \\ 3 \\ 5 \\ -2 \\ 7 \end{pmatrix}, \quad \mathbf{b}^2 = \begin{pmatrix} -3 \\ -1 \end{pmatrix}. \quad (4.13)$$

Calculate the value of the ANN function h at the point $(1, 0, 6)$.

Solution: We may write $h = g_2 \circ g_1$, where g_1 is the first layer, and g_2 is the second layer. The first layer is

$$g_1(\mathbf{x}) = \bar{\lambda}(A^1 \mathbf{x} + \mathbf{b}^1), \quad (4.14)$$

where $\bar{\lambda}$ is the vectorial ReLU activation function. Therefore

$$g_1(1, 0, 6) = \bar{\lambda} \left[\begin{pmatrix} 4 & 2 & -5 \\ 5 & 4 & 6 \\ 0 & 3 & 2 \\ 2 & 2 & -3 \\ -7 & 1 & -4 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 6 \end{pmatrix} + \begin{pmatrix} -2 \\ 3 \\ 5 \\ -2 \\ 7 \end{pmatrix} \right] \quad (4.15)$$

$$= \bar{\lambda} \left[\begin{pmatrix} -26 \\ 41 \\ 12 \\ -16 \\ -31 \end{pmatrix} + \begin{pmatrix} -2 \\ 3 \\ 5 \\ -2 \\ 7 \end{pmatrix} \right] \quad (4.16)$$

$$= \bar{\lambda} \left[\begin{pmatrix} -28 \\ 44 \\ 17 \\ -18 \\ -24 \end{pmatrix} \right] \quad (4.17)$$

$$= \begin{pmatrix} 0 \\ 44 \\ 17 \\ 0 \\ 0 \end{pmatrix}. \quad (4.18)$$

The second layer is

$$g_2(\mathbf{x}) = \bar{\lambda}(A^2\mathbf{x} + \mathbf{b}^2). \quad (4.19)$$

Therefore

$$h(1, 0, 6) = g_2 \circ g_1(1, 0, 6) \quad (4.20)$$

$$= g_2(0, 44, 17, 0, 0) \quad (4.21)$$

$$= \bar{\lambda} \left[\begin{pmatrix} -2 & 6 & 2 & -5 & 0 \\ -3 & 0 & 6 & 1 & 2 \end{pmatrix} \begin{pmatrix} 0 \\ 44 \\ 17 \\ 0 \\ 0 \end{pmatrix} + \begin{pmatrix} -3 \\ -1 \end{pmatrix} \right] \quad (4.22)$$

$$= \bar{\lambda} \left[\begin{pmatrix} 298 \\ 102 \end{pmatrix} + \begin{pmatrix} -3 \\ -1 \end{pmatrix} \right] \quad (4.23)$$

$$= \bar{\lambda} \left[\begin{pmatrix} 295 \\ 101 \end{pmatrix} \right] \quad (4.24)$$

$$= \begin{pmatrix} 295 \\ 101 \end{pmatrix}. \quad (4.25)$$

4.3 The universal approximation theorem

One of the reasons for the success of deep learning methods stems from the fact that ANN functions can approximate a wide class of continuous functions. To illustrate this, we first present the approximation theorem from [29].

Theorem 4.3.1 (Universal approximation theorem). *Let $K \subset \mathbb{R}^n$ be a closed bounded set, and let $f : K \rightarrow \mathbb{R}^m$ be a continuous function such that its components are all non-negative: $f_i(\mathbf{x}) \geq 0$ for all $\mathbf{x} \in K$ and $1 \leq i \leq m$. Then for any $\varepsilon > 0$, there exists an ANN $h : \mathbb{R}^n \rightarrow \mathbb{R}^m$ with two function layers (i. e., three layers with one hidden layer) using the ReLU activation function such that*

$$\sup_{\mathbf{x} \in K} \|h(\mathbf{x}) - f(\mathbf{x})\| < \varepsilon. \quad (4.26)$$

There are several versions of the universal approximation theorem (UAT) in the literature, and we refer again to [29] for a more general version.

Although the UAT guarantees that a positive continuous function may be approximated by an ANN with one hidden layer, it does not make any mention of how large that hidden layer must be. In practice, a good approximation may indeed require this hidden layer to be quite large, too large even for powerful computers to handle. This is where having a deeper network is useful as many hidden layers, even if they are not so wide, may be able to perform the same approximation with far fewer total number of neuron functions.

Another version of the UAT [51] applies to functions of the form $L_2 \circ \bar{\lambda} \circ L_1$, where L_1 and L_2 are affine maps (of the form $A\mathbf{x} + \mathbf{b}$) maps, and $\bar{\lambda} : \mathbb{R}^k \rightarrow \mathbb{R}^k$ is the componentwise application of a non-linear activation function λ (see Definition 4.1.1):

$$\bar{\lambda}(x_1, \dots, x_k) = (\lambda(x_1), \dots, \lambda(x_k)). \quad (4.27)$$

Note that $\bar{\lambda} \circ L_1$ is a layer function, whereas $L_2 \circ \bar{\lambda} \circ L_1$ is not an ANN function (4.8) according to our definitions because L_2 is not composed on the left with an activation function (an ANN would be given by $\bar{\lambda} \circ L_2 \circ \bar{\lambda} \circ L_1$). Thus only a one-layer function with its activation function composed with one affine map is needed for good approximations, but two-layer functions also do the job.

Theorem 4.3.2 (Universal approximation theorem). *Let $K \subset \mathbb{R}^n$ be a closed bounded set, and let $\lambda : \mathbb{R} \rightarrow \mathbb{R}$ be a non-linear function that is not a polynomial. Let $f : K \rightarrow \mathbb{R}^m$ be a continuous function. Then for any $\varepsilon > 0$, there exist $k \in \mathbb{N}$ and affine functions $L_1 : \mathbb{R}^n \rightarrow \mathbb{R}^k$ and $L_2 : \mathbb{R}^k \rightarrow \mathbb{R}^m$ such that*

$$\sup_{\mathbf{x} \in K} \|f(\mathbf{x}) - L_2 \circ \bar{\lambda} \circ L_1(\mathbf{x})\| < \varepsilon, \quad (4.28)$$

where $\bar{\lambda}$ is the component-wise action of λ as in (4.27).

Question: Why can't λ be a polynomial?

Answer: For simplicity, we assume that the dimensions of the input and output are 1, i. e., $n = m = 1$ in Theorem 4.3.2. Suppose that λ is a polynomial of degree $d \in \mathbb{N}$. Then L_1 is an affine map $\mathbb{R} \rightarrow \mathbb{R}^k$ (k is the width of the hidden layer), and L_2 is an affine map $\mathbb{R}^k \rightarrow \mathbb{R}$, so $L_1(x) = \mathbf{a}_1 x + \mathbf{b}_1$ with $\mathbf{a}_1, \mathbf{b}_1 \in \mathbb{R}^k$, and $L_2(\mathbf{y}) = \mathbf{a}_2 \cdot \mathbf{y} + b_2$ with $\mathbf{a}_2 \in \mathbb{R}^k$ and $b_2 \in \mathbb{R}$. Thus $L_2 \circ \bar{\lambda} \circ L_1$ is a linear combination of neuron functions of the form $\bar{\lambda}(\mathbf{a}_1 x + \mathbf{b}_1)$. Clearly, a polynomial of degree d composed with an affine function is still a polynomial of degree d . Furthermore, a linear combination of polynomials of degree d is a polynomial of degree d or less. Therefore $L_2 \circ \bar{\lambda} \circ L_1$ is a polynomial of degree $\leq d$ and can at best approximate any function that can be approximated by d degree polynomials. Recall that the Weierstrass approximation theorem [52] states that arbitrarily close approximations can be achieved by polynomial functions, and in general, arbitrarily large degree may be needed.

We further show that the function $f(x) = \cos(x)$ in the interval $K = [0, 4\pi]$ cannot be approximated by polynomials of degree ≤ 3 . This function has five points where it reaches a local maximum of 1 or minimum of -1: $s_0 = 0$, $s_1 = \pi$, $s_2 = 2\pi$, $s_3 = 3\pi$, and $s_4 = 4\pi$. Note that with this notation, if i is even, then $f(s_i) = 1$, and if i is odd, then $f(s_i) = -1$.

Suppose that $\cos(x)$ can be approximated by a polynomial of degree 3 in the interval K . Then for any $\varepsilon > 0$, there exists a cubic polynomial $p(x)$ such that

$$|\cos(x) - p(x)| < \varepsilon \quad \forall x \in K. \quad (4.29)$$

Let $\varepsilon = 1/2$, and let p be a cubic polynomial such that (4.29) holds. Then p has at most three roots in K . We will consider the case that p has exactly three roots in K (with the case of fewer roots following by the same argument). Let us label $r_1 < r_2 < r_3$ the roots of p . We may decompose the interval K into four subintervals:

$$K = K_1 \cup K_2 \cup K_3 \cup K_4 := [-4\pi, r_1) \cup [r_1, r_2) \cup [r_2, r_3) \cup [r_3, 4\pi]. \quad (4.30)$$

Since $f(x)$ has five local extrema $\{s_0, s_1, s_2, s_3, s_4\}$, at least one of the intervals K_i must contain at least two consecutive extrema s_i and s_{i+1} . Let K_j be this interval. Since p is continuous and has no roots in the interior of K_j , it has constant sign in the interior of K_j . Without loss of generality, suppose $p(x) \geq 0$ for $x \in K_j$. However, since K_j must contain two consecutive extrema s_i and s_{i+1} of f , we have $f(x) = 1$ for one these two, and $f(x) = -1$ for the other. Without loss of generality, suppose $f(s_i) = -1$, so recalling that we assume $p(s_i) > 0$, we conclude that (4.29) does not hold for $x = s_i$ and $\varepsilon = 1/2$. This is a contradiction, so we conclude that $f(x) = \cos(x)$ cannot be approximated by a cubic polynomial on the interval K .

We also observe that fixing any degree $d \geq 1$, we can use the same argument on a sufficiently large interval K to show that $\cos(x)$ cannot be approximated in the sense of (4.29) by polynomials of degree d . Alternatively, we could fix the interval and show that $\cos(kx)$ for k large enough cannot be approximated by polynomials of degree d .

4.4 Why is non-linearity in ANNs necessary?

We now explain why in our case study of classification of handwritten digits, the exact classifier is taken non-linear. Any attempt to approximate the exact classifier by a linear function will usually lead to gross misclassifications; we cannot approximate a curve arbitrarily close by a straight line.

4.4.1 $0 + 0 = 8$?

This example considers the simplified setting of a DNN with no biases and no activation functions, that is, linear maps in place of affine and no activation functions, so the DNN is linear. There is also no softmax layer, but it can be easily added without changing the result.

Consider the problem of classifying handwritten digits. The exact classifier $\hat{\phi}$ maps each image $\mathbf{s}$ of a handwritten digit to the integer between 0 and 9 depicted in the image. Consider the images of digits $\mathbf{s}_1$ and $\mathbf{s}_2$ in Figure 4.5. Both images depict the digit zero. By adding these two images pixel-by-pixel we obtain a new image $\mathbf{s}_1 + \mathbf{s}_2$, which looks very much like an eight. This leads to the provocative name of this example, “ $0 + 0 = 8$ ” [65]. Observe that $\mathbf{s}_1 + \mathbf{s}_2$ is classified as 8:

$$\hat{\phi}(\mathbf{s}_1 + \mathbf{s}_2) = 8, \quad (4.31)$$

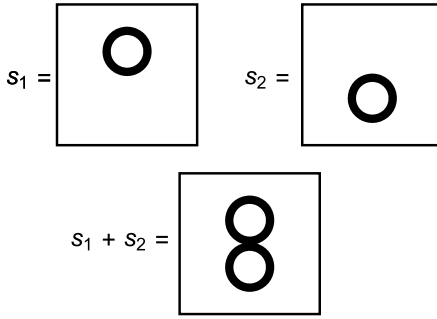


Figure 4.5: Adding together two images of a zero gives an image of an eight.

whereas

$$\hat{\phi}(s_1) + \hat{\phi}(s_2) = 0 + 0 = 0. \quad (4.32)$$

So the sum of the classes is not equal to the class of the sum. Since $\hat{\phi}(s_1) + \hat{\phi}(s_2) \neq \hat{\phi}(s_1 + s_2)$, $\hat{\phi}$ is non-linear, so it cannot be closely approximated by a linear ANN without an activation function.

When attempting tasks that are more complex than recognizing digits, an affine classifier may make even less sense. To quote directly from [65] for example, “recognizing faces instead of numbers requires a great many pixels—and the input–output rule is nowhere near linear”.

4.4.2 Non-linear activation functions are necessary in ANNs

Suppose that we do not use a non-linear activation function λ in a single neuron function, that is, each neuron function (4.1) in our network has the simplified affine form

$$f(\mathbf{x}) = \mathbf{a} \cdot \mathbf{x} + \beta \quad (4.33)$$

for a vector of parameters $\mathbf{a}$. Then a layer function (4.3) has the simplified form

$$g(\mathbf{x}) = A\mathbf{x} + \mathbf{b} \quad (4.34)$$

for a matrix of parameters A and a bias vector $\mathbf{b}$. An ANN (4.8) composed of several linear layers has the form

$$h(\mathbf{x}) = (\mathbf{b}^M + A^M(\mathbf{b}^{M-1} + A^{M-1} \dots (\mathbf{b}^1 + A^1 \mathbf{x}) \dots)), \quad (4.35)$$

where each A^i ($i = 1, \dots, M$) is a matrix of parameters, and each $\mathbf{b}^i$ is a vector. However, the product of several matrices is still a matrix. Therefore, an ANN that is a composition of several affine layers is actually a single-layer function:

$$h(\mathbf{x}) = A\mathbf{x} + \mathbf{b},$$

$$\text{where } A = A^M A^{M-1} \dots A^1, \text{ and } \mathbf{b} = \mathbf{b}^M + A^M \mathbf{b}^{M-1} + A^M A^{M-1} \mathbf{b}^{M-2} + \dots + A^M \dots A^2 \mathbf{b}^1. \quad (4.36)$$

Thus, without non-linear activation functions, the resulting ANN is affine. First of all, this renders all intermediary hidden layers useless. Second, in the complete absence of non-linear activation functions, we cannot approximate exact classifiers that are not affine.

4.5 Why do we need biases?

We can easily find examples where a linear classifier with no bias can misclassify objects, whereas if the classifier includes a bias vector, then all objects are correctly classified.

Introduce two classes

$$S_1 = \{(x, y) \in \mathbb{R}^2 : 2 < y \leq 4\} \quad \text{and} \quad S_2 = \{(x, y) \in \mathbb{R}^2 : 0 \leq y \leq 2\}, \quad (4.37)$$

denoted by red circles and blue crosses, respectively (see Figure 4.6).

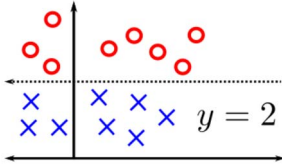


Figure 4.6: Datapoints $(x, y) \in \mathbb{R}^2$ belong to two classes S_1 and S_2 . A linear classifier is insufficient to exactly classify the object since objects near the origin will always be misclassified; see proof below.

Lemma 4.5.1. *Consider a classification problem with two classes S_1 and S_2 defined by (4.37); see Figure 4.6. Then, for any linear classifier $\mathbf{g} : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ with no bias,*

$$\mathbf{g}(\mathbf{x}) := A\mathbf{x}, \quad (4.38)$$

there exists an object $\mathbf{x} \in S_i \subset \mathbb{R}^2$ that will be misclassified.

Proof. Consider a linear classifier $\mathbf{g} : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ with matrix of weights A and no bias:

$$\mathbf{g}(\mathbf{x}) := A\mathbf{x}, \quad A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}. \quad (4.39)$$

Recall that there is no activation function (or $\lambda(x) = x$) for a linear classifier. We now show that the layer function (4.39) (with no bias) misclassifies some objects. Indeed, consider two objects (datapoints)

$$\mathbf{x}_1 = (1, 0), \quad \mathbf{x}_2 = (-1, 0), \quad (4.40)$$

both belonging to the lower set S_2 , the blue crosses. Then the corresponding outputs of the layer function $\mathbf{g}$ are given by

$$A\mathbf{x}_1 = \begin{pmatrix} a_{11} \\ a_{21} \end{pmatrix}, \quad A\mathbf{x}_2 = \begin{pmatrix} -a_{11} \\ -a_{21} \end{pmatrix}. \quad (4.41)$$

Next, we apply the softmax function to the output vectors in (4.41) that normalizes the outputs as probabilities. One important property of the softmax function is order preserving. In particular, the largest component of $A\mathbf{x}_1$ will be the largest component of the output of the softmax function as well. We have

$$\sigma(A\mathbf{x}_1) = \left(\frac{e^{a_{11}}}{e^{a_{11}} + e^{a_{22}}}, \frac{e^{a_{21}}}{e^{a_{11}} + e^{a_{22}}} \right), \quad (4.42)$$

$$\sigma(A\mathbf{x}_2) = \left(\frac{e^{-a_{11}}}{e^{-a_{11}} + e^{-a_{22}}}, \frac{e^{-a_{21}}}{e^{-a_{11}} + e^{-a_{22}}} \right). \quad (4.43)$$

Using the order-preserving property of the softmax function, we get:

- If $a_{11} > a_{21}$, then the vector $\mathbf{x}_1$ is misclassified as an element of class S_1 , that is, the first component of $\sigma(A\mathbf{x}_1)$ (the probability that $\mathbf{x}_1 \in S_1$) is greater than the second component.

The vector $\mathbf{x}_2$ is correctly classified as an element of the class S_2 .

- If $a_{11} < a_{21}$, then the vector $\mathbf{x}_2$ is misclassified as an element of class S_1 , and the vector $\mathbf{x}_1$ is correctly classified as an element of class S_2 . $\square$

Therefore by the counterexample above the *linear* classifier $\mathbf{g}(x) = Ax$ is insufficient for this binary classification problem. However, if we introduce biases, then we have exact classifiers.

Lemma 4.5.2. *There exists an exact classifier of the form $g(\vec{x}) = \lambda(A\vec{x} + \vec{b})$ with the bias vector that will correctly classify all objects $\mathbf{x} \in S_1 \cup S_2$.*

Remark. Observe that this theorem states the existence of an exact classifier with a bias vector depending on the data: not every bias will give an exact classifier as the choice does depend on the data.

Proof. Let us take the matrix of weights

$$A = \begin{pmatrix} 0 & 0 \\ 0 & -1 \end{pmatrix} \quad (4.44)$$

and bias vector

$$\mathbf{b} = \begin{pmatrix} 0 \\ 2 \end{pmatrix}. \quad (4.45)$$

Now we can show that any vector in either class is classified correctly.

- Consider any vector $\mathbf{x}_3 = \begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix} \in S_1$. Compute the output of the affine map:

$$A\mathbf{x}_3 + \mathbf{b} = \begin{pmatrix} 0 \\ 2 - \mu_2 \end{pmatrix}. \quad (4.46)$$

By assumption

$$\mathbf{x}_3 \in S_1 \implies 2 < \mu_2 \leq 4 \iff -2 \leq 2 - \mu_2 < 0.$$

After applying the softmax function, we find that

$$\sigma(A\mathbf{x}_3 + \mathbf{b}) = \left(\frac{e^0}{e^0 + e^{2-\mu_2}}, \frac{e^{2-\mu_2}}{e^0 + e^{2-\mu_2}} \right). \quad (4.47)$$

As $e^0 > e^{2-\mu_2}$, the first entry of (4.47) is greater than the second one, and we conclude that the constructed classifier correctly classifies $\mathbf{x}_3 \in S_1$.

- Now consider any $\mathbf{x}_4 = \begin{pmatrix} \mu_3 \\ \mu_4 \end{pmatrix} \in S_2$. Similarly, for $\mathbf{x}_4$,

$$\mathbf{x}_4 \in S_2 \implies 0 \leq \mu_4 \leq 2 \iff 0 \leq 2 - \mu_4 \leq 2.$$

Applying the affine map results in

$$A\mathbf{x}_4 + \mathbf{b} = \begin{pmatrix} 0 \\ 2 - \mu_4 \end{pmatrix}. \quad (4.48)$$

After applying the softmax function and using the order-preserving property, the classifier also correctly classifies $\mathbf{x}_4 \in S_2$. $\square$

4.6 Exercises

Exercise 1 (A basic example of classifiers). Assume that we have data points in the interval $[0, 1] \subset \mathbb{R}$ belonging to three possible classes. We are looking for possible soft classifiers.

- Explain why such classifiers are functions $p : \mathbb{R} \rightarrow \mathbb{R}^3$.
- Show that the following two functions are acceptable soft classifiers:

$$p(x) = \left(\frac{x}{2}, \frac{x^2}{2}, 1 - \frac{x}{2} - \frac{x^2}{2} \right),$$

$$\tilde{p}(x) = (1 - 4x(1-x) - x(1-2x)^2, 4x(1-x), x(1-2x)^2).$$

- Is the function

$$q(x) = (x, 1-x, (x-1/2)^2)$$

an acceptable soft classifier?

- d. Consider the following data points in $\mathbb{R}$: $x_1 = 0$ in class #1, $x_2 = 1/2$ in class #2, and $x_3 = 1$ in class #3. Calculate the results of the classifiers p and $\tilde{p}$ on those data points. Which of the two has a better performance?

Exercise 2 (Some examples with the softmax function).

- a. Apply the softmax function to the vector

$$y = (1, 0, -1) \in \mathbb{R}^3.$$

- b. Is there a vector $y \in \mathbb{R}^3$ such that

$$\sigma(y) = (1, 0, 0)?$$

- c. Explain why a softmax function cannot directly yield a hard classifier.

Exercise 3 (A basic example of ANN). We consider an artificial neuron network with exactly one hidden layer. All activation functions are ReLU, and there are no biases. The neurons from the input to the hidden layer are given by the matrix

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 4 & 5 \end{bmatrix},$$

whereas the neurons from the hidden layer to the output are given by the matrix

$$B = \begin{bmatrix} 1 & 1 \\ 2 & 2 \\ 1 & 1 \\ 2 & 2 \end{bmatrix}.$$

- Give the dimensions of the input, hidden, and output layers.
- Write an explicit formula for the full network function.
- Give the general condition on two matrices A and B so that there exists an ANN with one hidden layer for which $A = A_1$ is the matrix for the neuron functions between the input and hidden layer and $B = A_2$ is the matrix for the neuron functions between the hidden layer and the output.

Exercise 4 (Representing non-linear functions with ANNs). We only consider in this exercise ANNs with ReLU as an activation function for all neurons.

- a. Construct a neural network with a one-dimensional input layer, one-dimensional output layer and exactly one hidden layer such that the network function is exactly

$$f : x \in \mathbb{R} \rightarrow f(x) = |x| \in \mathbb{R}.$$

What is the minimal dimension of the hidden layer?

- b. Extend the previous construction to obtain the network function

$$f : x \in \mathbb{R}^n \rightarrow f(x) = (|x_1|, |x_2|, \dots, |x_n|) \in \mathbb{R}^n.$$

- c. We now want to prove that it is not possible to construct an ANN that is exactly equal to the function

$$g : x \in \mathbb{R} \rightarrow g(x) = x^2 \in \mathbb{R},$$

even with an arbitrary number of hidden layers.

We break the problem into two parts.

- i. Assume that all biases in the network are 0. Prove that in this case the network function f necessarily satisfies

$$f(\lambda x) = |\lambda| f(x) \quad \text{for all } \lambda \in \mathbb{R},$$

so that $f(x) \neq x^2$.

- ii. In the general case with biases, prove that a network function is necessarily piecewise affine.
- d. Can any piecewise linear function on $\mathbb{R}$ be obtained as a network function?

Exercise 5 (A simple classification problem). We consider a set Ω of objects, i. e., points of $\mathbb{R}^n$ that are in to two different classes or subsets Ω_1 and Ω_2 . We assume that Ω_1 and Ω_2 are separated by a hyperplane: there exist $\alpha \in \mathbb{R}^n$ and $\beta \in \mathbb{R}$ such that

$$x \in \Omega_1 \implies \alpha \cdot x + \beta < 0, \quad x \in \Omega_2 \implies \alpha \cdot x + \beta > 0.$$

- a. Construct an ANN with input of dimension n and output of dimension 2 such that the network function $f(x) = (f_1(x), f_2(x))$ classifies Ω_1 and Ω_2 in the following sense:

$$x \in \Omega_1 \implies f_1(x) > f_2(x), \quad x \in \Omega_2 \implies f_1(x) < f_2(x).$$

- b. Extend the construction above to the case of four classes $\Omega_i, i = 1, \dots, 4$, separated by two non-parallel hyperplanes: there exist $\alpha, \tilde{\alpha} \in \mathbb{R}^n$ and $\beta, \tilde{\beta} \in \mathbb{R}$ such that α and $\tilde{\alpha}$ are not collinear and

$$x \in \Omega_1 \implies \alpha \cdot x + \beta < 0 \text{ and } \tilde{\alpha} \cdot x + \tilde{\beta} < 0,$$

$$x \in \Omega_2 \implies \alpha \cdot x + \beta > 0 \text{ and } \tilde{\alpha} \cdot x + \tilde{\beta} < 0,$$

$$x \in \Omega_3 \implies \alpha \cdot x + \beta < 0 \text{ and } \tilde{\alpha} \cdot x + \tilde{\beta} > 0,$$

$$x \in \Omega_4 \implies \alpha \cdot x + \beta > 0 \text{ and } \tilde{\alpha} \cdot x + \tilde{\beta} > 0.$$

5 Supervised, unsupervised, and semi-supervised learning

5.1 Basic definitions

The training of ML algorithms is commonly subdivided into so-called supervised, unsupervised, and semi-supervised learning. Although those notions are often intuitively clear, there is no commonly accepted agreement in the literature on the definitions of supervised and unsupervised learning. For example, [26] emphasizes that “Unsupervised learning and supervised learning are not formally defined terms. The lines between them are often blurred.”

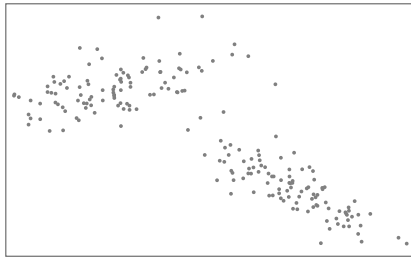
The concept of supervised, unsupervised, or semi-supervised learning was nevertheless recognized early on as useful when considering how the available datasets impact the choice of the algorithms; see [14, 27] or again the more recent [19, 26].

To compare the ideas of *supervised learning* and *unsupervised learning*, we focus on classification and clustering problems where more precise definitions can be given. In both cases, objects must be placed into different classes. However, roughly speaking, supervised learning deals with problems where the classes of the object in the training set are known a priori; we will then say that the training set is labeled. In unsupervised learning, the classes of the objects in the training set are not known a priori, and the training set is said to be unlabeled.

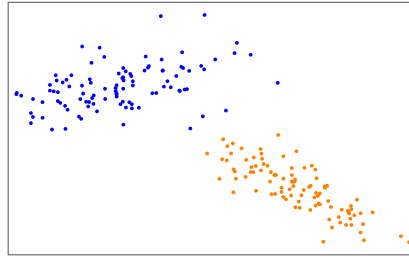
To go back to our example of handwriting recognition, images of handwritten digits need to be placed into one of ten classes, or images of handwritten English letters have to be placed into one of twenty-six classes. When applying supervised learning, we require a labeled training set. The labels (that is, classes) are for example “0”, “1”, ..., “9” for digits. Our training set is a collection T of images s together with the label corresponding to each image.

When training a machine learning algorithm in this supervised setting, we can compare the outputs from the algorithm to the labels that are known to be correct. When the algorithm’s output does not match the correct label, it improves itself (“learns”) and will hopefully produce more accurate outputs in the future. Once training is over, the parameters of the algorithm are fixed, and this algorithm can then be used on a wider range of images outside of T .

In contrast, *unsupervised learning* algorithms learn from unlabeled training sets. In the previous example, that would mean that our training set contains only a collection of images without knowing to which digit each image corresponds. Instead of classification, this is sometimes referred as a *clustering problem*. Like for classification problems, in a clustering problem, we have a set S of objects in $\mathbb{R}^n$ and seek to group together objects in S that are similar in some way. In some cases, we may still know how many groups to form. For example, if we know that we have images of handwritten digits, then we also know that we should obtain 10 groups; but in other situations, even the num-



(a) Unlabeled data



(b) A clustering algorithm groups data into two clusters

Figure 5.1: Result of a clustering algorithm in unsupervised learning.

ber of classes is unknown. Figure 5.1 depicts an example of a simple clustering problem where we start from unlabeled data (left) and end up with two classes (right). We next present two simple clustering examples in unsupervised learning in which the classes are not a priori known.

Example 5.1.1. Suppose we are trying to translate some writing from an unknown ancient language. Since there is no dictionary, the words are unknown. We can only group pieces of writing which may be words by similarity, that is, we can identify clusters of similar objects and assume that they are words in this ancient language.

The following example presents a realistic application of unsupervised learning.

Example 5.1.2. Suppose we visit a tree farm and want to classify various species of a tree that grows there. Before our trip, we were told how many species of trees grow at the farm but nothing more. To classify the trees, we collect a sample of leaves from the farm from every tree for measurement. For each leaf, we can determine the color, size, and shape. With this data, we can group similar leaves together, since leaves from one species of a tree should be similar regardless of which tree the leaf was taken from (i. e., leaves from an oak tree never look like leaves from a pine tree). In this way, we can cluster similar leaves together and assume that each cluster represents one species of a tree.

The architecture of ANNs in unsupervised learning is necessarily somewhat different from that for supervised algorithms. Notably, we cannot simply compare the output of the algorithm with the known label anymore. This observation implies that unsupervised learning requires a structure of ANN that differs from supervised learning. For example, simply adding more layers often widens the class of classifier functions that an ANN can approximate, but such a simple strategy does not work for unsupervised learning. As this is an introductory book, we will not explain how ANNs may be used for unsupervised learning and simply mention the use of so-called auto-encoders or the popular self-organizing maps; see, e. g., [35].

Recall that our set of objects S is divided into mutually disjoint classes, $S = \bigcup_{j=1}^m S_j$, and the exact classifier $\hat{\phi}$ maps objects into the index of their class: $s \mapsto j$. The training set is a subset $T \subset S$ on which the machine learning algorithm is trained to fit its parameters, such as the weights and biases of an ANN in Definition 4.1.3.

This intuitive discussion leads to rigorous definitions adopted in our lectures:

Definition 5.1.1. A machine learning algorithm is called:

- *Supervised* if the exact classifier $\hat{\phi}$ is known on the whole training set T (all data in T are labeled).
- *Unsupervised* if the exact classifier is completely unknown on any object $s \in T$ (no data in T are labeled).
- *Semisupervised* if the exact classifier is known on some strict subset $T_0 \subset T$ (some data in T are labeled).

In general, the decision to use supervised, unsupervised, or semi-supervised algorithms depends on, for example, the availability of appropriate datasets. For instance, if we wish to use supervised learning on some datasets, then it may be necessary to manually classify every object in the training set. This may be very time-consuming or simply impossible to achieve for large, “messy”, training sets. In such circumstances, the use of unsupervised or semi-supervised algorithms may be required.

We also emphasize that unsupervised learning is sometimes useful for making inferences about and identifying patterns from only the training set T , whereas a supervised algorithm is always expected to be used on a larger set of objects.

5.2 Example of unsupervised learning: detecting bank fraud

Consider the following example of a problem that employs unsupervised learning. Banks have an interest in determining whether or not transactions made via their accounts are fraudulent. This is an important practical problem, and the solutions implemented by banks are quite sophisticated. Moreover, banks keep the details of their procedures proprietary to make their security measures harder to circumvent. Therefore we present here a simple example, which illustrates how the problem of bank fraud could hypothetically be addressed by unsupervised learning.

The average account holder makes several transactions every day, and each transaction has several quantifiable properties, which can be used to represent each transaction as a point in $\mathbb{R}^n$: for example, a transaction could be represented in $\mathbb{R}^4$ as

$$s = \begin{pmatrix} \text{transaction amount} \\ \text{transaction time} \\ \text{day of the week} \\ \text{method, e. g., 0 for in-person, 1 for online} \end{pmatrix}. \quad (5.1)$$

An unsupervised clustering algorithm trained on the set of a bank account holder's transactions would find that most account holders have many similar transactions—they tend to spend similar amounts at similar stores and websites at similar times of the day and week, leading to clusters of transactions that are all comparable. Therefore a transaction that does not lie in a cluster with many of the account holder's other transactions is more likely to be fraudulent and warrants closer attention.

5.3 Exercises

Exercise 1 (Basic comparison of supervised vs. unsupervised learning). Consider the points in $\mathbb{R}$: $x_1 = -1$, $x_2 = 1$, and $x_3 = a$ for some $-1 < a < 1$.

- Assume that we know that x_2 and x_3 belong to the same class, whereas x_1 belongs to a different class. Give an example of a classifier that fits the data.
- Assume instead that we do not know the classes, just that there are exactly two classes, so that we instead try to cluster the points x_1, x_2, x_3 into two groups. Explain which is the most logical clustering depending on the value of a .

Exercise 2 (Basic example of k -means algorithm in dimension 1.). The k -means algorithm is a very well-known algorithm to perform clustering tasks in the context of unsupervised learning.

Consider the simplest case where we are given N data points in $\mathbb{R}$: $x_1, x_2, \dots, x_N$. We wish to cluster those points into two separate classes C_1 and C_2 , i. e. k -means with $k = 2$.

- Recall the definition of the variance of the points $x_1, \dots, x_N$.

$$\text{Var}(x_i)_{i \in \{1, \dots, N\}} = \text{Var}(x_1, \dots, x_N) = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2,$$

where

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i.$$

Show that $N \cdot \text{Var}(x_i)$ can also be expressed as

$$N \cdot \text{Var}(x_i)_{i \in \{1, \dots, N\}} = \frac{1}{2N} \sum_{i,j=1}^N |x_i - x_j|^2 = \sum_{i=1}^N x_i^2 - \frac{1}{N} \left(\sum_{i=1}^N x_i \right)^2.$$

- Assume we have $2N$ data points in $\mathbb{R}$. We now divide the indices $i = 1, \dots, 2N$ into two equal subsets C_1 and C_2 with $C_1 \cap C_2 = \emptyset$ and $C_1 \cup C_2 = \{1, \dots, 2N\}$. Give one explicit expression for the following function which is the sum of the variances over each subset

$$L(C_1, C_2) = N \cdot \text{Var}(x_i)_{i \in C_1} + N \cdot \text{Var}(x_i)_{i \in C_2}.$$

c. Show that we can write

$$L(C_1, C_2) = \sum_{i=1}^{2N} |x_i|^2 - \frac{1}{N} \left(\sum_{i \in C_1} x_i \right)^2 - \frac{1}{N} \left(\sum_{i \in C_2} x_i \right)^2.$$

d. We denote

$$x = \frac{1}{2} \sum_{i \in C_1} x_i - \frac{1}{2} \sum_{i \in C_2} x_i.$$

Show that there exists a constant $\bar{L}$ such that

$$L(C_1, C_2) = \bar{L} - \frac{2}{N} x^2.$$

- e. The goal of the k -means clustering algorithm is to choose C_1 and C_2 so as to minimize $L(C_1, C_2)$. Explain what is the best choice in terms of x .
- f. In general finding the optimal C_1, C_2 corresponding to the optimal x is a complex algorithmic problem. But in dimension 1, it is possible to considerably simplify this, even if k -means clustering is not especially useful in dimension 1. First we order the x_i with $x_1 < x_2 < \dots < x_{2N}$. Show that the optimal C_1 and C_2 are necessarily of the type

$$\begin{aligned} C_1 &= \{i \leq i_0\}, & C_2 &= \{i > i_0\} & \text{for some } i_0, \\ \text{or } C_1 &= \{i > i_0\}, & C_2 &= \{i \leq i_0\} & \text{for some } i_0. \end{aligned}$$

6 The regression problem

6.1 What is regression? How does it relate to ANNs?

Regression analysis is a well-known and widely used tool in data science and statistics dating back to the nineteenth century. The term *regression* was coined in [23]; see also [70]. Roughly speaking, the goal of regression analysis is to find a function of some independent variables (called predictors) that best fits the given data; see, e. g., [50]. For example, with one predictor, regression would attempt to find a curve (called a *regression curve*) that provides the best fit to some discrete empirical data, such as a finite set of points in the plane in 2D; see Figures 6.1 and 6.3. In contrast to the well-known problem of interpolation of discrete values of a function, in regression, the desired function does not have to pass through each point but tries to minimize some error, for example, with the method of least squares; see, e. g., [64] for a description of interpolation.

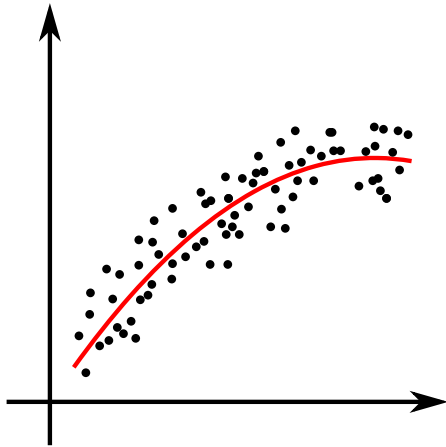


Figure 6.1: An simple example of regression. Black dots represent data points from measurements, a regression curve is in red.

A well-known case concerns linear regression where we are trying to find the “best” linear combination of predictors. Geometrically, this linear combination $c_1x_1 + \dots + c_{n-1}x_{n-1} + b = y$ is a hyperplane where x_i are the predictors and c_i are the coefficients of the linear combination. Again in the case of one predictor, the hyperplane (or linear space) is simply a line $cx + b = y$.

A typical regression problem can be rigorously formulated as follows. We seek to approximate an unknown function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$. In the simplest setting, the value of $f(x)$ is assumed to be known at the sample points $x_1, \dots, x_N$. That is, we have a finite set of pairs (x_i, y_i) in the set

$$T = \{(x_1, y_1), \dots, (x_N, y_N)\} \subset \mathbb{R}^n \times \mathbb{R}^m \quad (6.1)$$

such that $f(x_i) = y_i$ for each $i = 1, \dots, N$.

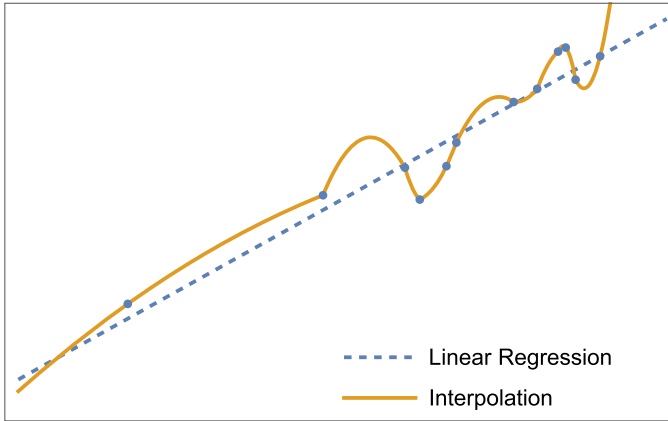


Figure 6.2: Linear regression versus interpolation. The interpolated curve passes through every data point, whereas linear regression does not pass through most of the points. Linear regression captures the trend of the data, whereas interpolation is more sensitive to noise, e. g., errors in measurements.

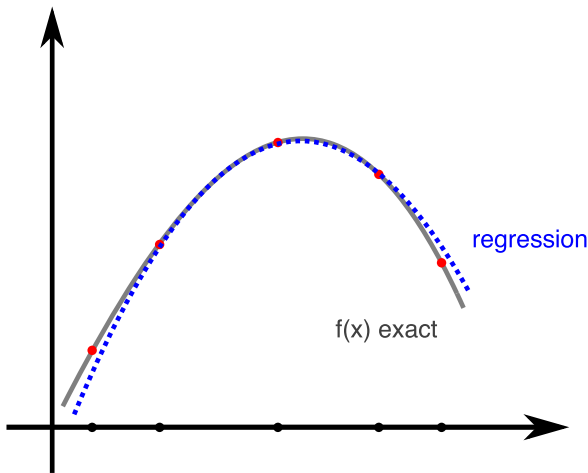


Figure 6.3: The red points are data sampled from the gray curve $f(x)$. The blue dotted line is the regression curve.

However, in many circumstances, we may expect some kind of error or “noise” in the data set T ; for example, measurement errors in y_i or x_i . In those settings, we do not expect to have $f(x_i) = y_i$ for the “actual” function f . This offers another example where pure interpolation may not be the optimal strategy and a best fit from regression may actually be closer to the objective function f , see Figure 6.2.

We seek an approximation of the function f in the form of $\tilde{f}(x, \gamma)$ where γ is a vector of tunable parameters. The regression problem consists of optimizing the parameters γ

so that $f(x) - \tilde{f}(x, \gamma)$ is optimal (e. g., small) in some sense. The most common example of such optimization is simply least squares minimization:

$$\min_{\gamma \in \mathbb{R}^d} \sum_{i=1}^N \|\tilde{f}(x_i, \gamma) - f(x_i)\|^2. \quad (6.2)$$

This optimization is an example of training in a quite simple setting: minimization in (6.2) can be done in iterative steps associated with the steps in the learning process. In some very simple settings, the exact minimization can also be determined and calculated analytically.

In general, regression problems are also a natural framework for DNN. Simply put, we consider functions $\tilde{f}(x, \gamma)$ that is an ANN function $h(x, \gamma)$ as in Definition 4.1.3; γ is the collection of parameters, e. g., weight matrices, as per Figure 6.4.

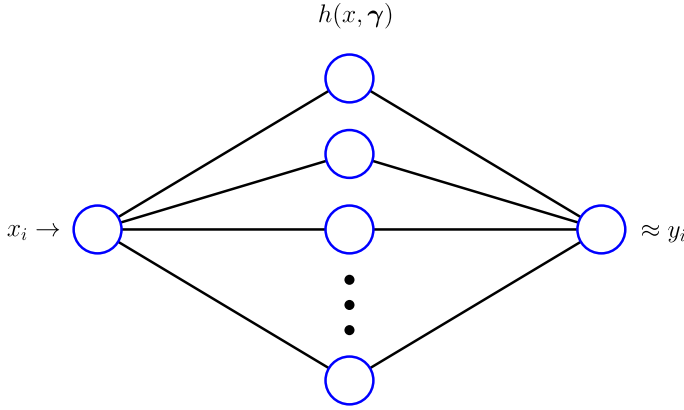


Figure 6.4: Regression ANN with input, one output, and one hidden layer.

6.2 Example: linear regression in dimension 1

The simplest case of the regression problem is a *linear regression* to approximate a function $f : \mathbb{R} \rightarrow \mathbb{R}$ by a linear function $\tilde{f}(x, \gamma)$, i. e., $\tilde{f}(x, \gamma) = ax + b$ with unknown parameters $\gamma = (a, b) \in \mathbb{R}^2$.

In many settings, the optimization of parameters through training is typically a complicated iterative process. In contrast, here we are lucky because the optimization problem (6.2) to find a and b can be solved analytically using calculus, and the resulting formulas are given in terms of datapoints (x_i, y_i) from (6.1):

$$a = \frac{N \sum_{i=1}^N x_i y_i - (\sum_{i=1}^N x_i)(\sum_{i=1}^N y_i)}{N \sum_{i=1}^N x_i^2 - (\sum_{i=1}^N x_i)^2}, \quad b = \frac{\sum_{i=1}^N y_i - a \sum_{i=1}^N x_i}{N}. \quad (6.3)$$

Thus, in this elementary case, finding the optimal parameters for linear regression can be done in a single simple step.

6.3 Logistic regression as a single neuron ANN

As explained above, the goal of regression is to find a curve that fits the given data. The family of such curves with unknown parameters is called a *regression model*. In particular, in the simple case of linear regression in 2D, the curve is a straight line that provides the best fit for pairs of data points on the plane.

In contrast, a *logistic model* [17], also known as a logit model, can describe the probability of events with binary outcomes such as pass/fail in exams or positive/negative in COVID-19 tests. It is possible to generalize this model for more than two outcomes such as the classification of ten handwritten digits, but we will focus here on binary outcomes. Given data and a logistic model that is a parameterized family of probability distributions, logistic regression aims to find the parameters that provide some kind of best fit (e. g., minimizing cross-entropy loss described below) between the data and the probability distribution. For instance, in example below in Figure 6.6 the probability distribution provides a higher probability of passing for the people who actually passed and provides a low probability of passing for the people who actually failed.

We can compare regression with logistic regression in the following way: regression aims to minimize the distance between the data and the approximating curve, whereas logistic regression aims to obtain a probability distribution that maximizes the probability of observing the given data.

The notion of *logistic regression* [34] is commonly used to analyze binary logistic models. Typically, we approximate a function $p(x)$ that gives the probability that an event occurs based on the value of some independent variable x .

We specifically discuss the relations between logistic regression and two areas of machine learning, support vector machines and DNNs, through examples.

6.3.1 1D example: studying for an exam

Consider a group of students preparing for an exam. Each student studies a certain number of hours. Students who study more are clearly more likely to pass the exam than students who study less, though studying for a long time is not a guarantee of passing, and neither not studying is a guarantee of failure. We assume that this situation can be accurately described by a random process, namely that there exists a function $p(x)$ that corresponds to the probability of passing the exam after x hours of study. We can expect $p(x)$ to increase from near 0 when $x = 0$ (students who do not study are unlikely to pass) to near 1 for large x (students who study a lot are very likely to pass). See Figure 6.6 for an example of the curve $p(x)$. A logistic regression algorithm seeks to approximate this function.

The name “logistic” refers to the form of the approximation $\tilde{p}(x, \gamma)$ of $p(x)$ that we are trying to obtain:

$$\tilde{p}(x, \gamma) = \frac{1}{1 + e^{-(ax+b)}}, \quad x \in \mathbb{R}, \quad (6.4)$$

where $\gamma = (a, b) \in \mathbb{R}^2$. The function

$$\sigma(x) = 1/(1 + e^{-x}) \quad (6.5)$$

is called the *logistic function* (which gives the name to logistic regression) or *sigmoid function*. It is often used as an activation function in ANNs, so $\tilde{p}(x, \gamma) = \sigma(ax + b)$ is a composition of a non-linear (activation) function with an affine function, just as in a single neuron function in Definition 4.1.1.

However, $\tilde{p}$ is not a classifier for the following reasons. First, it has only one class, whereas a classifier should have at least two classes (the output vector is in $[0; 1]^n$, $n \geq 2$). Moreover, the output of (6.4) looks like the output of soft classifiers. However, the probabilities for soft classifiers should add up to 1. On top of that, in the classification problem, the students who studied the same amount of hours x will be classified with the same outcome of the exam, whereas logistic regression, by dealing with probabilities, takes into account that students who study the same number of hours can have different outcomes.

In Figure 6.5, we see that $\sigma(x)$ is increasing from 0 as $x \rightarrow -\infty$ to 1 as $x \rightarrow \infty$. Therefore it is a good simple approximation of probabilities evolving from 0 at $-\infty$ (probability of an event that never occurs) to 1 at ∞ (probability of an event that almost surely occurs). The two parameters a and b of the affine transform in (6.4) control the slope of $\tilde{p}(x, \gamma)$ and the location of the point where $\tilde{p}(x, \gamma) = 1/2$. Of course, we cannot expect to fit any such probability function $p(x)$ with only the function $\sigma(x)$ of the form (6.5)

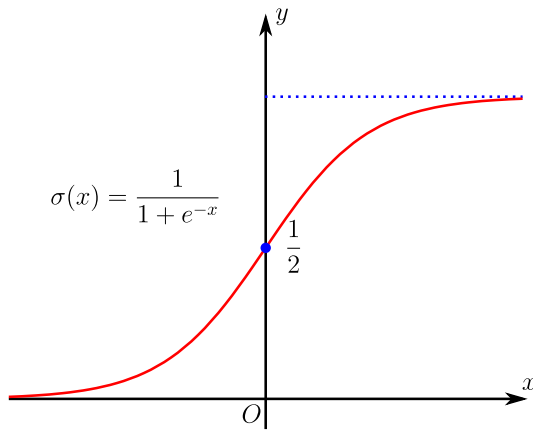


Figure 6.5: Sigmoid red curve $\sigma(x)$. Note that it asymptotically approaches the dotted line $y = 1$ as $x \rightarrow \infty$ and approaches zero as $x \rightarrow -\infty$.

Recall that in the basic regression problem described in Section 6.1, (e.g., (6.1)–(6.2)), we know several exact values taken by the unknown function to be approximated (see (6.1)). In contrast, in the above example of logistic regression, we do not know the exact value of $p(x)$ at any point since we cannot measure the probability of any student passing an exam.

Instead, we have the following data from N students: pairs (x_i, r_i) , $i = 1, \dots, N$, where $x_i \geq 0$ is the number of hours the student spent studying, and r_i is 0 if the student failed and 1 if they passed. We do not expect to be able to predict with perfect accuracy whether any given student passes or fails based only on the number of hours studied because there are many other factors at play, e.g., attendance. Instead, we will use this data to find the optimal parameters $\mathbf{y} = (a, b)$ in the logistic regression function $\tilde{p}(x, \mathbf{y})$ defined in (6.4) to approximate the probability that a student passes the exam.

First, we can arrange the known data pairs (x_i, r_i) so that $r_i = 1$ for $i = 1, \dots, k < N$ and $r_i = 0$ for $i = k+1, \dots, N$, i.e., the first k students passed the exam, and the last $N - k$ students failed. Next, we can, for example, minimize the following function:

$$L(\mathbf{y}) := - \sum_{i=1}^k \log(\tilde{p}(x_i, \mathbf{y})) - \sum_{i=k+1}^N \log(1 - \tilde{p}(x_i, \mathbf{y})). \quad (6.6)$$

The function $L(\mathbf{y})$ in (6.6) is called the *cross-entropy loss function*. The minimization of this function over all parameters $\mathbf{y}$ is a typical example of a training process, which is also used in DNN classifiers.

Since $\tilde{p}(x, \mathbf{y})$ has the logistic form (6.4), we arrive at minimizing the following function over all $(a, b) \in \mathbb{R}^2$:

$$L(a, b) := - \sum_{i=1}^k \log\left(\frac{1}{1 + e^{-(ax_i + b)}}\right) - \sum_{i=k+1}^N \log\left(\frac{e^{-(ax_i + b)}}{1 + e^{-(ax_i + b)}}\right). \quad (6.7)$$

An example with specified numbers (x_i, r_i) of the optimal logistic regression $\tilde{p}(x, \mathbf{y})$ for the exam studying example is shown in Figure 6.6.

In general, of course, the solution to the optimization problem can be intricate. However, there are cases where the solution is simple and understandable, e.g., when we have a steep transition between a probability of succeeding from almost 0 to almost 1. Hence assume that $p(x) \approx 0$ for $x < \bar{x}$ and $p(x) \approx 1$ for $x > \bar{x}$ (here “ $\approx$ ” stands for “close to”). In that case, in the first sum in (6.7) over students who passed, we expect most of them to have had a high probability of passing, i.e., $p(x_i) \approx 1$. Therefore we need to find $\mathbf{y}$ such that the logistic regression $\tilde{p}(x_i, \mathbf{y})$ is also close to 1, that is, we want that $\log(\tilde{p}(x_i, \mathbf{y})) \approx 0$ or that a, b be such that $a x_i + b \gg 1$ for $x_i > \bar{x}$ (here “ $\gg$ ” stands for “much greater than”). Similarly, in the second sum for failing students, we would need $1 - \tilde{p}(x_i, \mathbf{y}) \approx 1$ or, equivalently, that $a x_i + b \ll -1$ for $x_i < \bar{x}$. This naturally leads to $a \gg 1$ and $b = -a \bar{x}$ such that $a x_i + b = a(x_i - \bar{x})$.

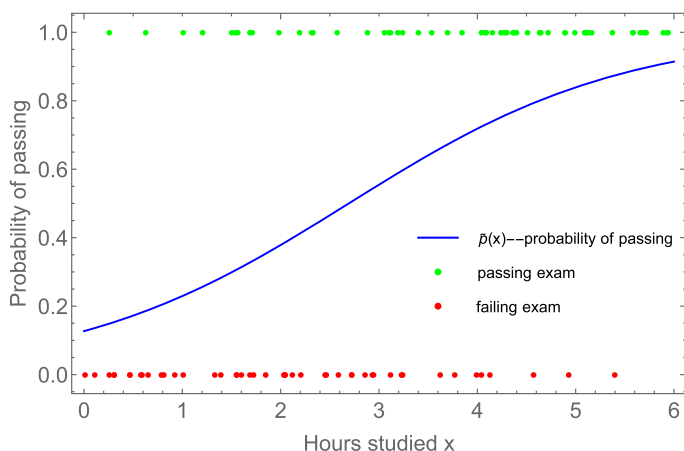


Figure 6.6: 100 students study for an exam for varying amounts of time between 0 and 6 hours. Some students pass (blue dots), other fail (orange dots). The black curve is the composition of the sigmoid with an affine $ax + b$ function (cf., definition of a single neuron 4.1.1). It shows the function $\tilde{p}(x)$ of the form (6.4) that minimizes (6.7). We interpret $\tilde{p}(x)$ as the probability that a student passes the exam after studying for x hours.

6.3.2 2D example of admittance to graduate school: separation of sets and decision boundary

Suppose we are provided with the GPAs and GRE scores of 600 applicants to a graduate program and whether or not each application was accepted. The GPAs and GRE scores are recorded as a vector (x_1, x_2) in $\mathbb{R}^2$. Each vector (x_1, x_2) is assigned the label (class) “1” if the student was admitted and “0” otherwise. See Figure 6.7, where sets of blue and orange dots represent these two classes.

In our previous notations, the set of all possible pairs of GPA and GRE score is the set of objects $S \subset \mathbb{R}^2$. The pool of 600 applicants whose scores we know is the training set T , and it is labeled because we know whether each applicant was accepted or not. The goal is to use logistic regression to predict the probability of acceptance of future applicants, e. g., for $s \in S \setminus T$. Note that in this problem, it is possible that a future applicant has exactly the same GPA and GRE score as one of the 100 applicants in T . However, that does not necessarily mean that the outcome will be the same for these two; two applications can have exactly the same scores with one accepted and one not due to other factors such as the previous research experience, strength of letters of recommendation, etc. The whole point of performing a logistic regression lies in estimating the probabilities of the outcome (rather than the exact outcome) based only on a limited number of factors.

In this example, we use the following logistic regression model:

$$\tilde{p}(x, y) = \sigma(a \cdot x + b), \quad (6.8)$$

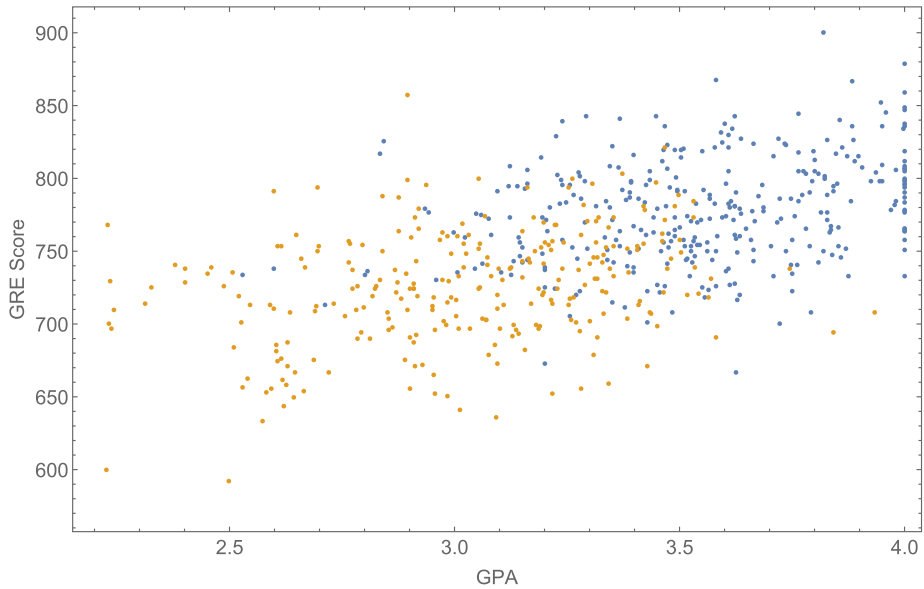


Figure 6.7: Scatter plot of GPAs and GRE scores of grad school applications. Blue points represent applications that were ultimately accepted, whereas orange points represent applications that were rejected.

where $\mathbf{y} = (\mathbf{a}, b)$, $\mathbf{a} \in \mathbb{R}^2$ and $b \in \mathbb{R}$. The graph of the logistic regression function (6.8) is a surface in 3D controlled by three parameters, $\mathbf{a} = (a_1, a_2)$ and b . The line $\mathbf{a} \cdot \mathbf{x} + b = 0$ is the *decision boundary* (see Figure 6.8) for $\tilde{p}(\mathbf{x}, \mathbf{y})$ with $\tilde{p}(\mathbf{x}, \mathbf{y}) > 1/2$ on one side of the line and $\tilde{p}(\mathbf{x}, \mathbf{y}) < 1/2$ on the other.

The decision boundary does not entirely determine the logistic regression model of course. We can easily find other parameters $\tilde{\mathbf{a}}$ and $\tilde{b}$ yielding the same decision boundary, i. e., such that $\{\mathbf{a} \cdot \mathbf{x} + b = 0\} = \{\tilde{\mathbf{a}} \cdot \mathbf{x} + \tilde{b} = 0\}$. We can, for example, just take $\tilde{\mathbf{a}} = \lambda \mathbf{a}$ and $\tilde{b} = \lambda b$ for the same $\lambda \in \mathbb{R}$. In particular, this means that the decision boundary does not give any information on how steep $\tilde{p}(\mathbf{x}, \mathbf{y})$ is around the decision boundary.

In logistic regression as well as in support vector machines (SVM) described below, the decision boundary is a straight line (linear decision boundary), whereas for DNN classifiers, it could be a more complicated curve in 2D (or surface in higher dimensions).

Ideally, all points from one class would be on one side of the decision boundary, and all points from the other would be on the other. This would lead to an exact decision boundary. However, in practical problems, the optimal decision boundary is usually a line that separates classes approximately, i. e., most points from the first class are on one side of the curve and most points from the second class are on the other.

We emphasize that, as a consequence, *the decision boundary is defined by the classifier* (e. g., $\tilde{p}(\mathbf{x}, \mathbf{y})$ for logistic regression) rather than by the purely geometric structure of the classes. In other words, the decision boundary is determined by the classifier's decision about in which class to place each object (be it correct or not). This stands in

contrast with the view at first glance that the decision boundary is determined only by points in the plane from two classes since, given a set of red and blue points, it may not be possible to find a line that separates these two sets.

In the admittance to graduate school example, the parameters $\mathbf{a}$ and b should be chosen such that $\mathbf{a} \cdot \mathbf{x} + b = 0$ roughly separates graduate applications, which are likely to be accepted from those that are not. To find $\mathbf{a}$ and b , we minimize (6.6). Figure 6.8 shows the optimal decision boundary obtained via numerically solving the regression problem.

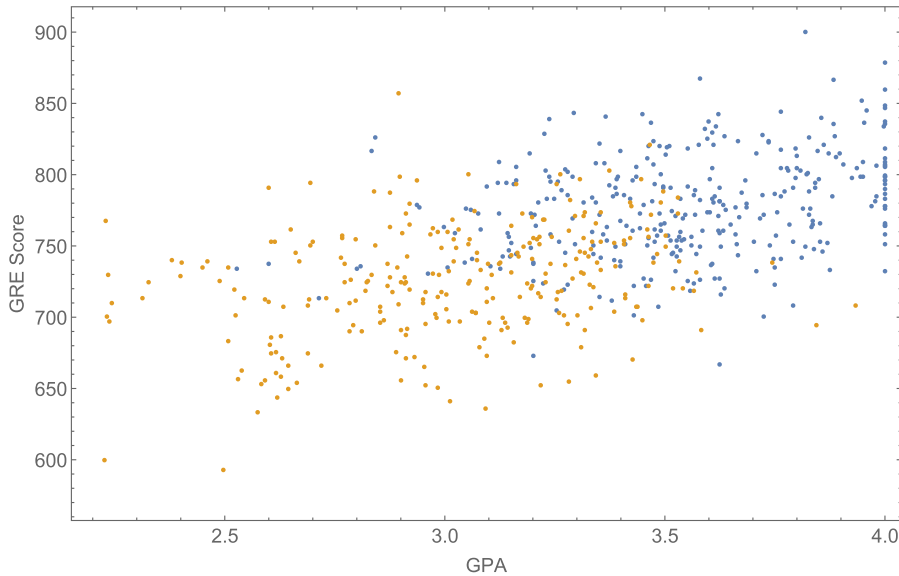


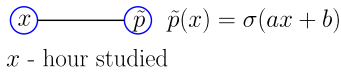
Figure 6.8: Scatter plot of GPAs and GRE scores of grad school applications. Blue points represent applications that were ultimately accepted, whereas orange points represent applications that were rejected. The black line is the decision boundary of the optimal logistic regression, separating applications likely to be accepted from those likely to be rejected.

6.3.3 Relation between ANNs and regression

As we mentioned above, the logistic model (6.8) is a composition of σ with an affine function $\mathbf{a} \cdot \mathbf{x} + b$. We may hence think of the iterative minimization of (6.6) in logistic regression as training an ANN with only one layer function (in fact, only one neuron function, $k = 1$ in Definition 4.1.2 of a layer function), whose activation function is the sigmoid $\sigma(x) = 1/(1 + e^{-x})$, see Figure 6.9.

The sigmoid function is a well-known activation function and was even more commonly used early in the development of ANNs. As an activation function $\sigma(x)$ models a smooth transition between an inactive neuron, which is not firing ($\sigma(x) \approx 0$), and an active (fired) neuron ($\sigma(x) \approx 1$).

Exam-studying example:



Graduate admission example:

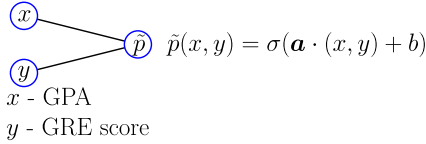


Figure 6.9: Logistic regression examples represented as ANNs with a single neuron function as in Definition 4.1.1.

In recent years, other activation functions such as ReLU have gained popularity relative to the sigmoid function; see, e. g., [25]. This is because the process of training an ANN involves iteratively taking various partial derivatives of the ANN function with respect to its parameters and using these derivatives to improve the parameter values to make better classifications (such as the gradient descent method described later). The derivative $\sigma'(x)$ becomes small for large $x > 0$, whereas $\text{ReLU}'(x) = 1$ for large $x > 0$. If we use the derivative-based optimization methods, then the magnitude of improvement is determined by the magnitude of the derivatives, which is in turn determined by the derivative of the activation function. This means that the improvements made to ANN parameters through the training process are smaller when the activation function is σ than when it is ReLU, leading to longer training when σ is used. This behavior of a sigmoid is known as the “vanishing gradient.”

This allows us to summarize the relations between regression and DNNs as follows.

- Linear regression introduces the concept of affine dependence on parameters, which is a key building block of ANNs.
- Logistic regression introduces a concept of activation function (i. e., sigmoid) and its composition with affine transform, which corresponds to the definition of a single-layer function in DNNs.
- The optimization of parameters in logistic regression leads to the cross-entropy loss function, which is often used in the training of DNNs.

6.3.4 Logistic regression vs. networks with many layers

To keep the discussion as simple as possible, let us return to our 1D example of students working on their final example. In that setting, we see that a logistic regression function $\tilde{p}(x, y)$ is increasing from 0 to 1 (or decreasing from 1 to 0 if $a < 0$). Therefore we can only expect $\tilde{p}(x, y)$ to be a good approximation of the actual (unknown) probability $p(x)$ if $p(x)$ is also monotone (or close enough to monotone, as in Figure 6.10).

The notion of decision boundary is useful again: logistic regression models can be reasonably accurate only when the decision boundary where $p(x) = 1/2$ is *one point*,

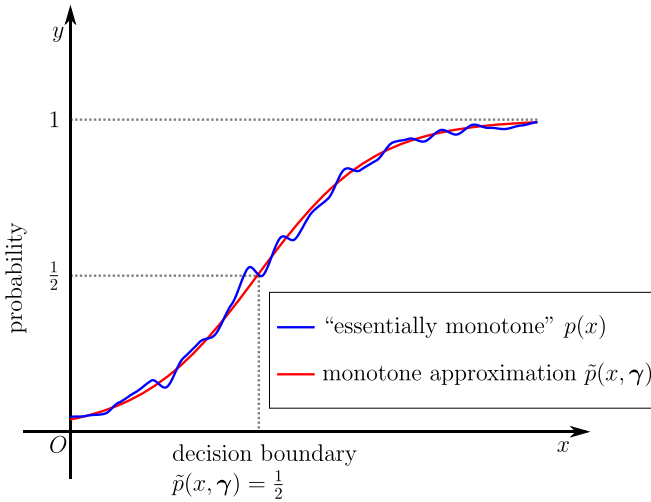


Figure 6.10: The exact probability curve $p(x)$ is not monotone, but it is essentially monotone in that $p(x) > p(y)$ whenever $x - y$ sufficiently large. In this case, it is still possible to approximate $p(x)$ by a monotone function.

or “almost one point” as in the blue curve in Figure 6.10, where the decision boundary consists of several closely spaced points.

A simple example of where a logistic model is likely not appropriate is the following model of growing plants. Every plant requires light of certain intensity. If the intensity is too low, then the plant is not likely to thrive. Likewise, if the light is too intense, then the plant may dry out and wither as well. Thus the probability $p(x)$ that a plant survives after a certain period of time (one week, for example) under light of intensity x is not monotone; see Figure 6.11.

If $p(x)$ represents the probability that a plant survives for a week under light of intensity x , the curve $p(x)$ initially increases from near 0 to near 1 as intensity changes from low to moderate and then decreases from near 1 to near 0 as intensity increases from moderate to high. This curve is not monotone, so it cannot be well approximated by the monotone logistic curve. Note that this curve has not one but two well-separated points where $p(x) = 1/2$.

Recall that the universal approximation theorem states that any continuous function regardless of its monotonicity can be approximated by a DNN with two or more function layers when using the ReLU activation function. This cannot apply to the logistic regression model $\tilde{p}(x, \gamma)$ since it corresponds to a DNN with only one function layer and the sigmoid activation function.

In higher dimensions $n > 1$, as in the 2D graduate admission example, the use of logistic models similarly requires that the decision boundary where $p(x) = 1/2$ is a *single* $(n - 1)$ -dimensional hyperplane, which is a straight line in 2D (or almost a straight line); see Figure 6.8.

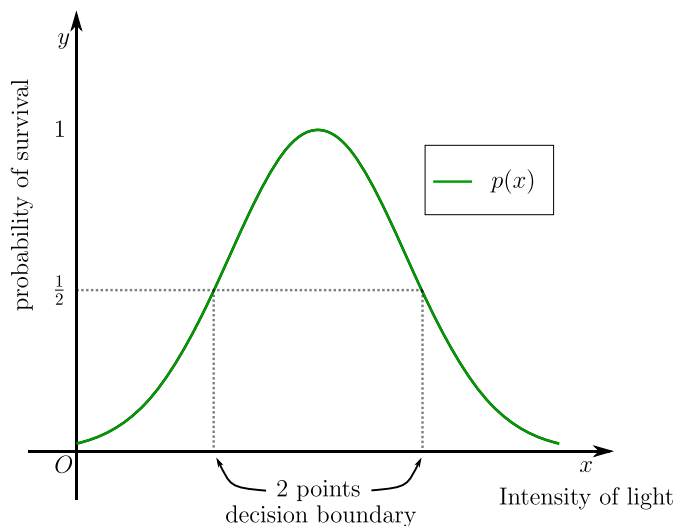


Figure 6.11: Probability distribution of a plant surviving. The points on the x -axis where $p(x) = \frac{1}{2}$ are decision boundaries. Note that this probability distribution cannot be well approximated by a sigmoid function because there is more than one decision boundary.

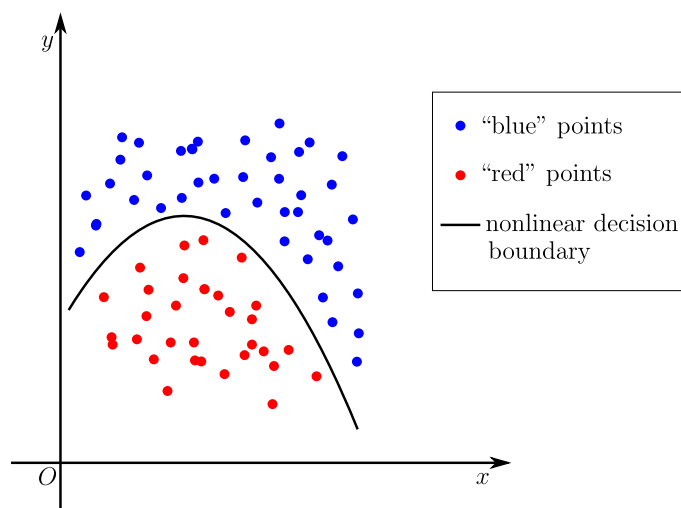


Figure 6.12: An example of a non-linear decision boundary. Note that these two classes are well separated by the curve in black (i. e., each class is on exactly one side of the curve), whereas any straight line decision boundary would be unable to perfectly separate the classes.

Hence in general, when the decision boundary is not almost a straight line (see Figure 6.12) or is expected to consist of several lines, then the use of DNNs with at least two function layers is likely to yield a better approximation of $p(\mathbf{x})$.

6.4 Exercises

Exercise 1 (Basic example of linear regression).

- a. Consider the following data points: $(x_1 = -2, y_1 = 1)$, $(x_2 = 0, y_2 = 2)$, $(x_3 = 1, y_3 = -1)$. Solve the corresponding linear regression problem, that is, find the coefficients $a, b \in \mathbb{R}$ minimizing

$$\sum_{i=1}^3 |a x_i + b - y_i|^2.$$

- b. We now take data points x_i in $\mathbb{R}^2$ and y_i still in $\mathbb{R}$: $x_1 = (-2, 1)$, $y_1 = 1$, $x_2 = (0, -1)$, $y_2 = 2$, $x_3 = (1, 2)$, $y_3 = -1$, $x_4 = (3, 0)$, $y_4 = 0$. Solve the linear regression problem by finding the coefficients $a \in \mathbb{R}^2$ and $b \in \mathbb{R}$ minimizing

$$\sum_{i=1}^4 |a \cdot x_i + b - y_i|^2,$$

where $a \cdot x_i$ is the usual inner product on $\mathbb{R}^2$.

Exercise 2 (Logistic regression in higher dimension). In this chapter, we showed formulas for logistic regression functions $\tilde{p}(x, y)$ where x is one- or two-dimensional.

- a. Write down an analogous formula for a logistic regression function $\tilde{p}(x, a, b)$ with three-dimensional input x , that is, find a formula for a logistic function

$$p : \mathbb{R}^3 \times \mathbb{R}^3 \times \mathbb{R} \longrightarrow \mathbb{R}$$

that maps the input $x \in \mathbb{R}^3$ and parameters $a \in \mathbb{R}^3$ and $b \in \mathbb{R}$ to a probability.

- b. Generalize this by writing a formula for a logistic function with an n -dimensional input x ,

$$p : \mathbb{R}^n \times \mathbb{R}^n \times \mathbb{R} \longrightarrow \mathbb{R}.$$

Exercise 3 (Overfitting in linear regressions). Consider datapoints $\{(x_1, y_1), \dots, (x_N, y_N)\}$ with $x_i \in \mathbb{R}^n$ and $y_i \in \mathbb{R}^m$ on which we apply a linear regression; that is, we are looking for a matrix $A \in M_{m,n}(\mathbb{R})$ and a vector $b \in \mathbb{R}^m$ that minimize the least squares

$$\sum_{i=1}^m \|y_i - A x_i - b\|^2,$$

where $\|\cdot\|$ denotes the Euclidian norm on $\mathbb{R}^m$.

- a. Find explicit linear equations that the matrix A and vector b need to satisfy.
- b. From now on, assume that $m = 1$, that is, y_i are scalar in $\mathbb{R}$, and the matrix A becomes a vector $a \in \mathbb{R}^n$. We assume moreover that $n \geq N$, so that we have more variables than datapoints, that is, the dimensionality n of each predictor/feature x_i is greater than the number N of datapoints, and that the linear system in part a. is

undetermined. Prove that it is possible to choose $a \in \mathbb{R}^n$ and $b \in \mathbb{R}$ such that we have exactly $y_i = a \cdot x_i + b$ for all i . Observe that $y = a \cdot x + b$ determines a hyperplane in $\mathbb{R}^{n+1}$. For example, if $n = 2$ this is a straight line.

- c. (Robustness with respect to measurements). Consider two sets of datapoints (x_i, y_i) , and a second set $(x_i, \tilde{y}_i)$ such that $|y_i - \tilde{y}_i| \leq \varepsilon$. Consider two lines obtained from question b, $y = ax + b$ and $y = \tilde{a}x + \tilde{b}$. Find a constant C such that $|a - \tilde{a}| \leq Cn\varepsilon$. This is called an upper bound for $|a - \tilde{a}|$.

Exercise 4 (Overfitting in 1D polynomial regression). Given datapoints $(x_1, y_1), \dots, (x_n, y_n)$, where $x_i, y_i \in \mathbb{R}$, $i = 1, \dots, n$, it is always possible to find a polynomial $f(x)$ of degree $n - 1$ such that $f(x_i) = y_i$ for each $i = 1, \dots, n$ (as long as $x_i \neq x_j$ whenever $i \neq j$) and thus

$$\sum_{i=1}^n |f(x_i) - y_i|^2 = 0.$$

- a. Find the coefficients a, b, c of the parabola $y = ax^2 + bx + c$ passing through the points $(-1, 4)$, $(0, 2)$, and $(1, -8)$.
- b. Suppose $(x_1, y_1), \dots, (x_n, y_n)$ is a large dataset (that is, $n \gg 1$) and $f(x)$ is an $(n - 1)$ -th-degree polynomial such that $f(x_i) = y_i$ for each $i = 1, \dots, n$.

We now continue measurements to obtain new datapoints. Would the polynomial fit the new data well? Explain why or why not. You can answer by way of a simple example such as fitting four datapoints in the plane with a cubic polynomial versus fitting with a straight line via linear regression.

Hint: Choose three datapoints in a straight line and one more datapoint near the middle one, not on the line but close to it. What does the cubic (3rd-degree polynomial) regression look like? You can use software or an online tool to perform numerical experiments, just specify what tool you used.

This problem illustrates overfitting as follows. Consider a sample of n points $(x_i, y_i) \in \mathbb{R}^2$ and assume that all points except one or two, called outliers, are close to a straight line $y = ax + b$. First, we use as a regression curve polynomials of degree $M > n$. Then we can obtain an ideal regression (loss is zero) in the form of interpolation. We next remove one of the outlier points and compare two regression curves: i) the polynomial of degree M with $M + 1$ parameters that are the coefficients of the polynomial, and ii) the linear regression line $y = ax + b$ that has only two parameters. Clearly, the straight line provides a better approximation on average. This shows that too many parameters can worsen the regression.

Exercise 5 (Understanding the cross-entropy). The purpose of this exercise is to illustrate why the cross-entropy plays an important role in problems such as logistic regression. For this, we revisit the example about groups of students studying for an exam.

We are given N datapoints (x_i, r_i) for each student $i = 1, \dots, N$, where x_i is the number of hours the corresponding student has spent studying, and $r_i = 0$ if they failed or

$r_i = 1$ if they passed. We sort the datapoints so that $r_i = 1$ for $i = 1, \dots, k < N$ and $r_i = 0$ for $i = k + 1, \dots, N$.

Choose any parameterization function $\bar{p}(x, \theta)$ for some parameter θ . We first consider the general optimization problem of minimizing

$$L(\theta) = - \sum_{i=1}^k \log \bar{p}(x_i, \theta) - \sum_{i=k+1}^N \log(1 - \bar{p}(x_i, \theta)).$$

- a. We reformulate the problem by considering for a given number of hours x , the number of students $n(x)$ who studied x hours and passed and the number of students $f(x)$ who studied x hours and failed. We also denote $m(x) = n(x) + f(x)$ the total number of students who studied x hours. Show that we also have

$$L(\theta) = - \sum_{x=0}^{\infty} n(x) \log \bar{p}(x, \theta) - \sum_{x=0}^{\infty} f(x) \log(1 - \bar{p}(x, \theta)). \quad (6.9)$$

- b. Denote $p(x) = n(x)/m(x)$. Show that from (6.9) we may deduce

$$\begin{aligned} L(\theta) = \bar{L} + \sum_{x=0}^{\infty} m(x) & \left(p(x) \log \frac{p(x)}{\bar{p}(x, \theta)} + p(x, \theta) - p(x) \right) \\ & + \sum_{x=0}^{\infty} m(x) \left((1 - p(x)) \log \frac{1 - p(x)}{1 - \bar{p}(x, \theta)} + (1 - p(x, \theta)) - (1 - p(x)) \right), \end{aligned} \quad (6.10)$$

where

$$\bar{L} = - \sum_{x=0}^{\infty} m(x) p(x) \log p(x) - \sum_{x=0}^{\infty} m(x) (1 - p(x)) \log(1 - p(x)).$$

- c. Prove that for all $a, b > 0$, we have

$$a \log \frac{a}{b} + b - a \geq 0.$$

Conclude from (6.10) that $L(\theta) \geq \bar{L}$ and $L(\theta) = \bar{L}$ if and only if $p(x, \theta) = p(x)$ for all $x \geq 0$ where $m(x) > 0$.

- d. Unfortunately, it may not be always possible to find a parameter θ such that $p(x, \theta) = p(x)$ for all $x \geq 0$. From now on, we specifically focus on the case of logistic regression where $\theta = (a, b)$ and

$$\bar{p}(x, \theta) = \frac{1}{1 + e^{-(ax_i + b)}}.$$

Show that if $\theta = (a, b)$ minimizes $L(\theta)$ as given by (6.9), then we have

$$\begin{aligned}
0 &= - \sum_{x=0}^{\infty} n(x) x \frac{e^{-(ax+b)}}{1 + e^{-(ax+b)}} + \sum_{x=0}^{\infty} f(x) x \frac{1}{1 + e^{-(ax+b)}}, \\
0 &= - \sum_{x=0}^{\infty} n(x) \frac{e^{-(ax+b)}}{1 + e^{-(ax+b)}} + \sum_{x=0}^{\infty} f(x) \frac{1}{1 + e^{-(ax+b)}}.
\end{aligned} \tag{6.11}$$

e. Rewrite equations (6.11) as

$$\begin{aligned}
\sum_{x=0}^{\infty} n(x) x &= \sum_{x=0}^{\infty} m(x) x \frac{1}{1 + e^{-(ax_i+b)}}, \\
k &= \sum_{x=0}^{\infty} m(x) x \frac{1}{1 + e^{-(ax_i+b)}}.
\end{aligned}$$

f. Deduce that if (a_1, b_1) and (a_2, b_2) both minimize $L(a, b)$, then we cannot have $a_1 < a_2$ and $b_1 < b_2$.

7 Support vector machine

7.1 Preliminaries: convex sets and their separation, geometric Hahn–Banach theorem

A subset A of $\mathbb{R}^n$ is called *convex* if for any two points $a, b \in A$, it contains the whole line segment joining them. For example, an interval (a, b) in $\mathbb{R}$, squares and disks in $\mathbb{R}^2$, and cubes and balls in $\mathbb{R}^3$ are convex, see the example in Figure 7.1.

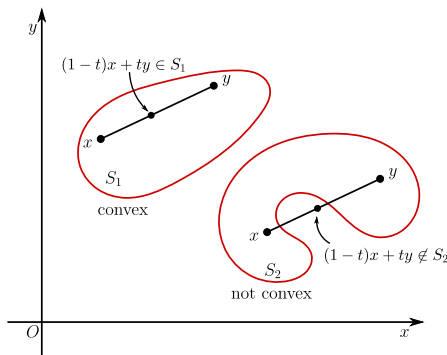


Figure 7.1: Convex and non-convex sets in $\mathbb{R}^2$.

Definition 7.1.1. $A \subset \mathbb{R}^n$ is convex if for all $x, y \in A$, the linear combination $(1-t)x + ty$ is also in A for all $t \in [0, 1]$.

An *affine hyperplane* is a subset H of $\mathbb{R}^n$ of the form

$$H = \{x \in \mathbb{R}^n \mid w \cdot x - b = 0\}, \quad (7.1)$$

where $w \in \mathbb{R}^n$ ($\neq 0$) and $b \in \mathbb{R}$. For example, in $\mathbb{R}^3$, letting $w = (w_1, w_2, w_3)$, H is the plane consisting of all points $x = (x, y, z)$ satisfying

$$w \cdot x - b = w_1x + w_2y + w_3z - b = 0. \quad (7.2)$$

Let A and B be two subsets of $\mathbb{R}^n$. We say that the hyperplane $H = \{x \in \mathbb{R}^n \mid w \cdot x - b = 0\}$ *separates* A and B if

$$w \cdot x - b \leq 0 \quad \forall x \in A \quad (7.3)$$

and

$$w \cdot x - b \geq 0 \quad \forall x \in B. \quad (7.4)$$

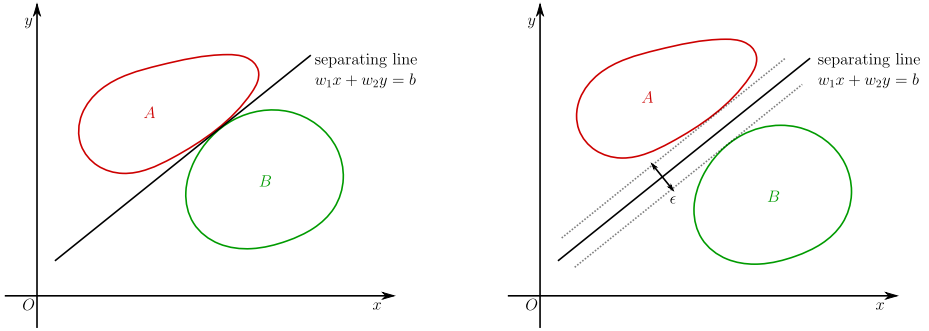
We say that H *strictly separates* A and B if there exists $\varepsilon > 0$ such that

$$w \cdot x - b \leq -\varepsilon \quad \forall x \in A \quad (7.5)$$

and

$$\mathbf{w} \cdot \mathbf{x} - b \geq \varepsilon \quad \forall \mathbf{x} \in B. \quad (7.6)$$

Geometrically, the separation means that A lies in one side of the hyperplane and B lies in the other. For example, see Figure 7.2, where a line $w_1x + w_2y - b = 0$ separates two sets. Strict separation means the sets lie not only on different sides of the hyperplane, but in addition, they are separated by a strip of width 2ε centered on this hyperplane. In such a case, we will often refer to ε as the margin.



(a) Non-strict separation

(b) Strict separation

Figure 7.2: Non-strict and strict separations of two convex sets.

When A and B are convex, the famous Hahn–Banach theorems guarantee that it is possible to separate the sets.

Theorem 7.1.1 (Hahn–Banach, first geometric form: separation). *Let $A \subset \mathbb{R}^n$ and $B \subset \mathbb{R}^n$ be two nonempty convex subsets such that A and B do not intersect, that is, $A \cap B = \emptyset$. Assume that one of them is open. Then there exists a hyperplane $H = \{\mathbf{x} \in \mathbb{R}^n \mid \mathbf{w} \cdot \mathbf{x} - b = 0\}$ that separates A and B [11].*

We also state the version of the theorem ensuring strict separation.

Theorem 7.1.2 (Hahn–Banach, second geometric form: strict separation). *Let $A \subset \mathbb{R}^n$ and $B \subset \mathbb{R}^n$ be two nonempty convex subsets such that $A \cap B = \emptyset$. Assume that A is closed and B is closed and bounded (compact). Then there exists a closed hyperplane $H = \{\mathbf{x} \in \mathbb{R}^n \mid \mathbf{w} \cdot \mathbf{x} - b = 0\}$ that strictly separates A and B .*

Consider a finite training set that can be divided into two subsets $T = T_1 \cup T_2$ corresponding to two classes. Is it possible to apply the Hahn–Banach theorem to ensure that T_1 and T_2 are linearly separable?

We cannot in fact directly apply the Hahn–Banach theorem to T_1 and T_2 since they are both finite. Indeed, convex sets are infinite if they contain more than one point. By

definition, if a convex set T_1 contains two points x_1 and x_2 , then it contains the entire segment connecting these points: all points $\hat{y}$ on the line $\hat{y} = \hat{t}x_1 + (1 - \hat{t})x_2$, $\hat{t} \in [0, 1]$, belong to T_1 . Therefore the sets T_i are only convex if they consist of only one point.

Instead, a natural idea is to embed T_1 and T_2 in convex sets. If A and B are convex with empty intersection and if $T_1 \subset A$ and $T_2 \subset B$, then the Hahn–Banach theorems can provide a decision boundary that separates A and B and hence T_1 and T_2 . We can expect that in this case, a machine learning algorithm will be able to find a classifier that separates T_1 and T_2 with a linear decision boundary.

To make this idea precise, we introduce the notion of a *convex hull*.

Definition 7.1.2. Let $C \subset \mathbb{R}^n$, and let $\mathcal{A}$ be the collection of all convex subsets of $\mathbb{R}^n$ that contain C . Then the *convex hull* $\hat{C}$ of C is the intersection of all the sets in $\mathcal{A}$:

$$\hat{C} = \bigcap_{A \in \mathcal{A}} A. \quad (7.7)$$

The convex hull of C is the smallest convex subset of $\mathbb{R}^n$ that contains C , that is, every convex subset of $\mathbb{R}^n$ that contains C also contains $\hat{C}$. For a 2D bounded subset of the plane, the convex hull may be visualized as the shape enclosed by a rubber band stretched around this set.

Although T_1 and T_2 are not convex, their convex hulls $\hat{T}_1$ and $\hat{T}_2$ are. However, although T_1 and T_2 do not intersect, $\hat{T}_1$ and $\hat{T}_2$ may intersect (imagine, for instance, the convex hulls of the blue and red points in Figure 6.12). Recall that nonintersection of two convex sets is a requirement of Theorems 7.1.1–7.1.2. Therefore by the Hahn–Banach theorem a machine learning algorithm is able to find a linear decision boundary separating T_1 and T_2 if and only if their convex hulls $\hat{T}_1$ and $\hat{T}_2$ are disjoint. For an in-depth presentation of these and related concepts in functional analysis, the reader is referred to, for example, [11, 36, 57].

7.2 Support vector machine

Support-vector machines (SVMs) are straightforward learning algorithms, which can be used to separate objects (data points) into classes. In their most basic form, two classes are separated by finding a separating line in $\mathbb{R}^2$ such that each class falls onto different sides of the line (or in $\mathbb{R}^n$, each class is on different sides of the hyperplane). SVMs are examples of early machine learning algorithms that can be viewed as a precursor to more sophisticated machine learning algorithms such as DNNs.

The first instance of an SVM algorithm can be traced back to the 1963 paper by Vapnik [68], who introduced this algorithm and used it to separate two classes of objects by a hyperplane. Later on, a detailed proof of this result was presented in the paper by Vapnik and Chervonenkis [69]. It is important to note that SVMs have been generalized to higher dimensions using non-linear separating surfaces instead of hyperplanes (see,

for instance, the 1992 paper by Vapnik, Boser et al. [9]). A comprehensive introduction to SVMs and related kernel methods can be found in [61].

In these notes, we consider the simplest case of two classes in the context of supervised learning. For example, consider a set S of objects in $\mathbb{R}^2$ such that each $s \in S$ belongs to one of two classes, e. g., classes “red” and “blue”; see Figure 7.3a. Similarly to logistic regression (e. g., Figure 6.8), an SVM tries to separate the classes through hyperplanes (e. g., straight lines in 2D). However, SVMs can be seen as hard classifiers, that is, they map each object to a class. In contrast, a logistic model can be interpreted as a soft classifier because it maps each object to the probability that the object belongs to a class.

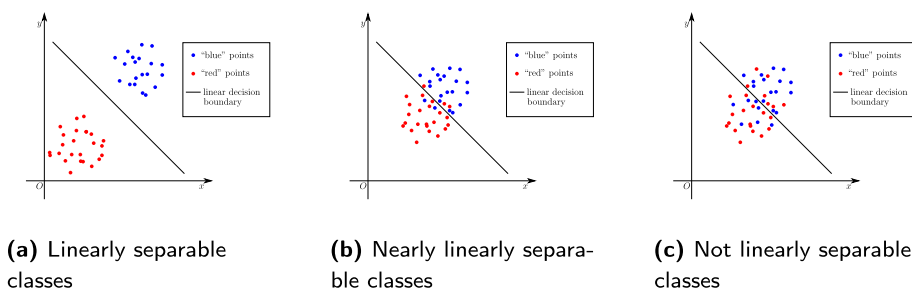


Figure 7.3: Linear decision boundaries in three binary classification problems. In each problem, a classifier attempts to classify all points on one side of the decision boundary as “red” and all points on the other side as “blue.” In the first case, it perfectly classifies all the given points. In the second case, it cannot perfectly classify all points, but it can do so approximately. In the third case, there is much more overlap between the two classes, and the decision boundary can barely classify half the given points correctly.

We can distinguish two types of SVM classifiers: hard margin and soft margin. A *hard-margin SVM classifier* finds a linear decision boundary that perfectly separates points into their two respective classes. This is possible when the two classes are linearly separable as in Figure 7.3a. When the two classes are not linearly separable as in Figures 7.3b–7.3c, we may use a *soft-margin SVM classifier*, which identifies a linear decision boundary approximately separating the classes (see also the discussion on the decision boundary in Section 6.3.2). In that sense, a soft-margin SVM classifier is somewhat similar to a logistic model (cf. Figure 6.8) trained with a different loss function described below (cf. cross-entropy loss (6.7) in regression).

By contrast, a hard-margin SVM classifier further differs from logistic models in that it identifies the linear decision boundary that separates two classes by the *widest margin*, which is a rather different optimization process. To ensure such a widest possible margin, the decision boundary of a hard-margin SVM classifier is chosen in such a way that the nearest point of each class is as far away as possible from the decision boundary; see Figure 7.6. Having a wide margin is obviously beneficial as illustrated in Figure 7.4,

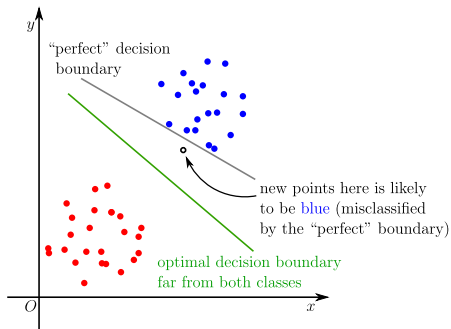


Figure 7.4: A decision boundary that perfectly separates two classes but is not optimal because it is very close to the blue class (small margin). Thus a new data point added close to the decision boundary is likely to be blue, but it could be easily misclassified as red due to the small margin. A perfect decision boundary separates both classes perfectly, but it may not be optimal, whereas the optimal decision boundary is always perfect.

where a decision boundary that is too close to one of the two classes is, for example, more likely to misclassify new data points whose class is unknown.

In a classification problem where two classes can be perfectly separated by a linear decision boundary on training set, a logistic regression algorithm may minimize the cross-entropy loss by making a sharp transition of $\tilde{p}(x, y)$ from 0 to 1. In such a situation the choice of the decision boundary has much less impact on the loss function than the sharpness of the transition; see Figure 7.4. Thus, in such problems, SVM is a better choice because it focuses on identifying the optimal decision boundary and is more likely to correctly classify new objects on the testing set.

7.3 Hard-margin SVM classifiers and support vectors

As mentioned above, we use hard-margin SVM classifiers when we have two classes of objects that can be separated by a linear decision boundary, as in Figure 7.3a; in such a case, we say that the classes are *linearly separable*. However, even in the cases of linearly separable classes, there are of course many lines that separate the classes; see Figure 7.5. Thus the goal of a hard-margin SVM is to find the optimal linear decision boundary in some appropriate sense.

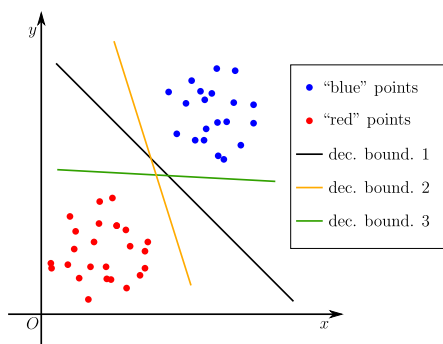


Figure 7.5: When two classes can be separated by a linear decision boundary, there are typically infinitely many such decision boundaries, leading to the question: which is best?

We start with a training set $(x_1, y_1), \dots, (x_N, y_N)$, with the $x_i \in \mathbb{R}^n$ denoting the objects and $y_i \in \{-1, 1\}$ representing the class to which x_i belongs (e.g., 1 for “red” and -1 for “blue” in Figure 7.3a). For simplicity, we will assume that x_i are points in $\mathbb{R}^2$. The goal of hard-margin SVM classifiers is twofold:

1. Separate the classes by a linear decision boundary of the form $\mathbf{w} \cdot \mathbf{x} - b = 0$, where $\mathbf{w} \in \mathbb{R}^2$ and $b \in \mathbb{R}$.
2. Find the parameters $\mathbf{w}$ and b such that the linear decision boundary has the largest possible *margin* that excludes all x_i .

The margin is the set of all points in $\mathbb{R}^2$ between two parallel lines defined as

$$\{\mathbf{x} \in \mathbb{R}^2 : -1 < \mathbf{w} \cdot \mathbf{x} - b < 1\}; \quad (7.8)$$

see Figure 7.6 (cf. strict separation of convex sets in Section 7.1). The distance between these two lines is $\frac{2}{\|\mathbf{w}\|}$. Therefore finding the largest margin that excludes all x_i means minimizing $\|\mathbf{w}\|$ while ensuring that $\mathbf{w} \cdot \mathbf{x}_i - b \geq 1$ if $y_i = 1$ and $\mathbf{w} \cdot \mathbf{x}_i - b \leq -1$ if $y_i = -1$ for each $i = 1, \dots, N$. Thus the optimal $\mathbf{w}$ and b are found by solving the following constrained optimization problem ($y_i = \pm 1$ for red and blue are labels):

$$\min_{\mathbf{w} \in \mathbb{R}^2, b \in \mathbb{R}} \|\mathbf{w}\| \quad \text{such that} \quad y_i(\mathbf{w} \cdot \mathbf{x}_i - b) \geq 1 \quad \forall i = 1, \dots, N. \quad (7.9)$$

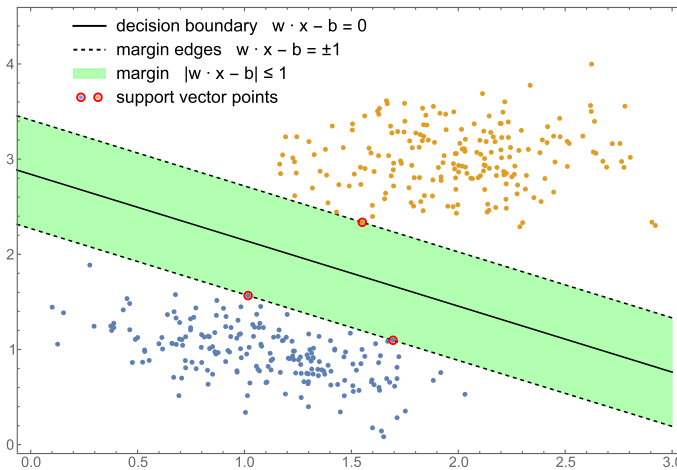


Figure 7.6: An example of classification using SVM. The red circled dots are the support vectors.

If $\mathbf{w}$ and b solve (7.9), then the decision boundary $\mathbf{w} \cdot \mathbf{x} - b = 0$ is optimal in the following sense. First, all datapoints x_i such that $y_i = 1$ are on one side of the line, and all x_i such that $y_i = -1$ are on the other side of the line (i.e., the line perfectly separates the

two classes). Second, among all linear decision boundaries that perfectly separate both classes (of which there are many; see Figure 7.5), this one has the maximum distance from the nearest elements $\mathbf{x}_i$ from each class. This is because all objects $\mathbf{x}_i$ are outside the margin. The distance from the decision boundary to the margin is $1/\|\mathbf{w}\|$, so by minimizing $\|\mathbf{w}\|$ in (7.9) we maximize the distance from the decision boundary to the objects $\mathbf{x}_i$.

Definition 7.3.1. Let $\bar{\mathbf{w}}, \bar{b}$ be a minimizer of (7.9). Then points $(z_i, y_i) \in T$ are called *support vectors* if they satisfy

$$y_i(\bar{\mathbf{w}} \cdot \mathbf{z}_i - \bar{b}) = 1, \quad (7.10)$$

which means that all z_i lie on the edges of the margin. Denote by $Z \subset T$ the set of all support vectors.

In other words, recall that all data points lie outside the margin and thus satisfy $y_i(\mathbf{w} \cdot \mathbf{x}_i - b) \geq 1$, and data points that lie on this margins are called support vectors. The minimization problem (7.9) can be rewritten only in terms of support vectors:

$$\min_{\mathbf{w}, b} \|\mathbf{w}\|^2 \quad \text{s. t.} \quad y_i(\mathbf{w} \cdot \mathbf{z}_i - b) = 1 \quad \forall z_i \in Z. \quad (7.11)$$

7.4 Soft margin SVM classifier

In the somewhat typical case where no linear decision boundary exactly separates two classes, as in Figure 7.3b, a soft-margin SVM classifier can still be used to find a linear decision boundary that best separates the two classes, even if it is not perfect.

Soft-margin SVM classifiers use the *hinge loss function* for individual objects $\mathbf{x}_i$:

$$L_i(\mathbf{w}, b) = \text{ReLU}(1 - y_i(\mathbf{w} \cdot \mathbf{x}_i - b)), \quad i = 1, \dots, N. \quad (7.12)$$

In the soft-margin SVM classifier, we do not require the constraint in (7.9) to hold. However, if $y_i(\mathbf{w} \cdot \mathbf{x}_i - b) \geq 1$ for some $i, \mathbf{w}, b$, that is, if $\mathbf{x}_i$ does satisfy the constraint, then $L_i(\mathbf{w}, b) = 0$ because $\text{ReLU}(x) = 0$ for $x < 0$. Conversely, if $y_i(\mathbf{w} \cdot \mathbf{x}_i - b) < 1$, then $L_i(\mathbf{w}, b) > 0$, increasing linearly with $1 - y_i(\mathbf{w} \cdot \mathbf{x}_i - b)$. Our goal is to find a linear decision boundary such that:

- Most $\mathbf{x}_i$ lie on the correct side of the decision boundary by a distance of at least $1/\|\mathbf{w}\|$ (i. e., $L_i(\mathbf{w}, b) = 0$).
- Some $\mathbf{x}_i$ will be closer to the decision boundary than $1/\|\mathbf{w}\|$ but still on the correct side (i. e., $0 < L_i(\mathbf{w}, b) \leq 1$).
- Even fewer $\mathbf{x}_i$ may even be on the wrong side of the decision boundary (i. e., $L_i(\mathbf{w}, b) > 1$), but not many.

To try to ensure this behavior, we introduce the regularized average hinge loss function

$$L(\mathbf{w}, b) = \lambda \|\mathbf{w}\|^2 + \frac{1}{N} \sum_{i=1}^N L_i(\mathbf{w}, b) \quad (7.13)$$

with an additional term $\lambda \|\mathbf{w}\|^2$.

To understand the meaning of (7.13), it is useful to consider how a soft-margin SVM classifier will be used. Typically, any object located at least $1/\|\mathbf{w}\|$ away from the decision boundary is assigned to the corresponding class. Objects that lie closer to the decision boundary (inside the margin) are not considered to be reliably classified.

Therefore a soft-margin SVM classifier tries to satisfy two somewhat competing objectives. First of all, it should be reliable: most objects that are assigned to a class should indeed belong to that class. Second, it should be reasonably precise: we want to have as few objects as possible that are not reliably classified.

Consider now the cases $\lambda \ll 1$ and $\lambda \gg 1$ in (7.13). If λ is large, then minimizing (7.13) mostly consists of trying to take $\|\mathbf{w}\|$ as small as possible, i. e., trying to have a large margin. However, we then give little weight to the terms involving $L_i(\mathbf{w}, b)$. For this reason, we may have a relatively large number of objects that are not reliably classified. On the other hand, because the margin is quite large, we likely do not have many wrongly classified objects. Such a classifier may be reliable but not precise at all.

On the other hand, if λ is very small, then we focus on minimizing the $L_i(\mathbf{w}, b)$ on the training set. On the training set, this optimization tries to limit the number of misclassified objects but also the number of objects within the margin. Hence it may lead to a very small margin. When looking at new objects outside the training set, this likely creates a large error rate: An object that is at a small distance from a well-classified object in the training set in the same class may end on the wrong side of the decision boundary if the margin is even smaller. We would obtain a precise classifier that is not reliable.

Therefore we must find an appropriate value of λ (not too large or too small) in a trial-and-error process often referred to as “tuning.” The parameter λ is an example of a *hyperparameter*. In machine learning, a hyperparameter is a value that controls the learning process, as opposed to a value determined by the learning process. This can be compared to hyperparameters in DNNs such as the number of layers and their width (which control the learning process versus the parameters of a DNN obtained by training).

Example 7.4.1 (Graduate admissions example revisited with SVM). We now revisit the example of graduate admissions from Section 6.3.2 to better illustrate the similarities and difference between logistic models and SVMs. Recall that we have a dataset consisting of pairs of GPAs and GRE scores for many grad school applications, along with whether each application was accepted or not. We wish to find a good model to try to predict which student will be accepted. In Section 6.3.2, we used logistic regression, and here we present the result for a soft-margin SVM classifier.

To this end, we minimize (7.13) over all $\mathbf{w} \in \mathbb{R}^2$ and $b \in \mathbb{R}$, choosing λ (after some trial and error) to be 0.005. Just as for the logistic model, a key feature of the resulting classifier is the decision boundary shown in Figure 7.7. The decision boundary found by the soft-margin SVM classifier is very similar to that found by logistic regression in Figure 6.8.

Recall from Section 6.3.2 that the logistic regression model provides a predicted probability function for this problem. In contrast, the soft-margin SVM classifier is closer to a hard classifier since it maps objects (e. g., the pairs (GPA, GRE)) to classes (e. g., accepted or not accepted) rather than to probabilities. In addition to identifying the classes, the soft-margin SVM classifier does provide a margin, as can be seen in Figure 7.7. This still offers a natural interpretation: applications that are above the margin are predicted to be accepted, and applications below the margin are predicted to be declined. The SVM classifier typically does not attempt to make a prediction for applications within the margin; indeed, as we can see, the two classes overlap to a considerable extent in the margin.

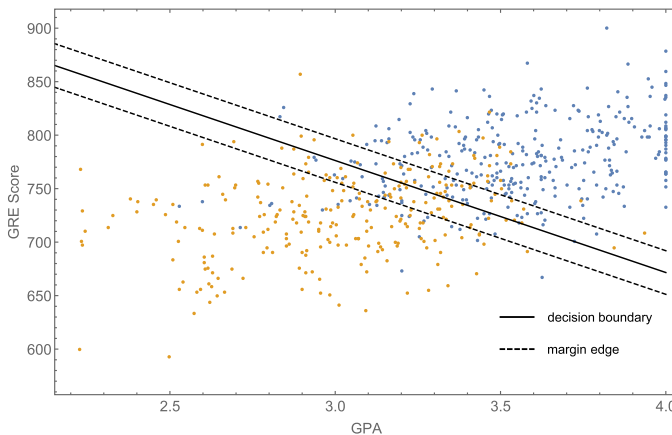


Figure 7.7: Scatter plot of GPAs and GRE scores of grad school applications. Blue points represent applications that were ultimately accepted, whereas orange points represent applications that were rejected. The solid black line is the decision boundary identified by a soft-margin SVM classifier, and the dashed lines bound the associated margin.

7.5 Exercises

Exercise 1 (A basic example of separation). Consider the equation for the line (one-dimensional hyperplane)

$$\mathbf{w} \cdot \mathbf{x} - b = 0, \quad (7.14)$$

where $\mathbf{w} = (1, -1)$ and $b = 1$.

- a. Draw a graph of the line (7.14).
- b. Does the line (7.14) separate the following two circular disks, A and B ? Explain why.

$$A = \{(x, y) \in \mathbb{R}^2 : (x - 1)^2 + (y - 1)^2 < 1/2\},$$

$$B = \{(x, y) \in \mathbb{R}^2 : (x - 2)^2 + y^2 < 1/2\}.$$

Exercise 2 (An example of hard-margin SVM). Consider the following datapoints: $(x_1 = (-1, 0), y_1 = -1)$, $(x_2 = (1, 0), y_1 = 1)$, $(x_3 = (0, 0), y_3 = -1)$, $(x_4 = (2, 0), y_4 = 1)$, where x_i belong to $\mathbb{R}^2$, and y_i belong to $\mathbb{R}$.

- a. Construct the hard margin classifier corresponding to the two classes labeled $y_i = -1$ or $y_i = 1$. Explicitly indicate the support vector.
- b. We add the fifth data point $(x_5 = (-1, 1), y_5 = -1)$. Explain why the hard-margin classifier is identical to the previous question.
- c. We add the sixth data point $(x_5 = (1/2, 1), y_5 = 1)$. Explain why the hard-margin classifier now has to be different and calculate it.

Exercise 3 (Existence and non-existence of hard margin classifiers). Consider the following partial datapoints: $x_1 = (0, 0)$, $x_2 = (1, 0)$, $x_3 = (1, 1)$, $x_4 = (2, 1)$.

- a. Also consider two classes labeled $y_i = -1$ or $y_i = 1$ with $y_1 = 1, y_2 = -1, y_3 = -1, y_4 = 1$.
 - a.1. Show that there does not exist a hard-margin classifier.
 - a.2. Determine the convex hull of each class and their intersection.
- b. We now take $y_1 = 1, y_2 = 1, y_3 = -1, y_4 = -1$.
 - b.1. Show that there exists a hard-margin classifier.
 - b.2. Determine the convex hull of each class and their intersection.
- c. Study all possible choices of y_i and determine for which one a hard-margin classifier exists and for which it does not. Compare this with what the intersection of the convex hull of each class predicts.

Exercise 4 (An example of soft-margin classifier). Consider the following datapoints: $(x_1 = (-1, 0), y_1 = -1)$, $(x_2 = (1, 0), y_1 = 1)$, $(x_3 = (0, 0), y_3 = -1)$, $(x_4 = (2, 0), y_4 = 1)$, where x_i belong to $\mathbb{R}^2$, and y_i are either -1 or 1 corresponding to two separate classes. We wish to find the soft-margin classifier corresponding to this example.

- a. Write the average hinge loss function and the regularized average hinge loss function $L(w, b)$ that we have to minimize.
- b. Recall that the ReLU function is smooth outside 0 and calculate its derivative.
- c. Identify the subdomains where $L(w, b)$ is smooth.
- d. Calculate the gradient of $L(w, b)$ on each such subdomain.
- e. Identify the minimum of $L(w, b)$.

Exercise 5 (Hard-margin classifier vs. soft-margin classifier). We consider $N = 2n$ datapoints $x_1 = \dots = x_n = (0, 0)$ with $y_1 = \dots = y_n = -1$ and $x_{n+1} = \dots = x_{2n} = (1, 0)$ with $y_{n+1} = \dots = y_{2n} = 1$.

- a. Calculate the hard-margin classifier for this dataset.
- b. Calculate the soft-margin classifier for this dataset.
- c. We now add one point to the dataset: $x_{2n+1} = (1/2, 0)$ with $y_{2n+1} = -1$. Calculate the new hard-margin classifier.
- d. Show that for n large, the new soft-margin classifier with the additional (x_{2n+1}, y_{2n+1}) remains close to the classifier identified in question b.

Exercise 6 (Support vectors for hard-margin SVMs). We consider datapoints x_i , $i = 1, \dots, N$ in $\mathbb{R}^2$ belonging to two classes labeled $y_i = -1$ or $y_i = 1$. We assume that there exists at least one optimal hard-margin classifier given by $\bar{w}$ and $\bar{b}$.

- a. Assume that there are three support vectors: two from class -1 corresponding to points x_1 and x_2 and one from class $+1$ corresponding to point x_3 .
 - a.1. Prove that the direction $\bar{w}/\|\bar{w}\|$ is uniquely determined by x_1 and x_2 .
 - a.2. Prove that both $\bar{w}$ and $\bar{b}$ are uniquely determined by x_1 , x_2 , and x_3 .
- b. We now assume that there are only two support vectors: x_1 for class -1 and x_2 for class $+1$.
 - b.1. Show that there exists an infinite set S of possible (w, b) such that $w \cdot x_1 - b = -1$ and $w \cdot x_2 - b = 1$.
 - b.2. Explain that among all (w, b) in S , the element (w_0, b_0) with the minimum norm $\|w_0\|$ is the one such that w_0 is parallel to the line connecting x_1 and x_2 .
 - b.3. Show that there exists a small but still infinite set $S' \subset S$ that perfectly separates the two classes and uses x_1 and x_2 as support vectors.
 - b.4. Of all the possible choices of (w, b) in S' , explain why there is only one that minimizes $\|w\|$ and why this is necessarily the optimal hard-margin classifier $(\bar{w}, \bar{b})$.
 - b.5. Show why we need to have $w_0 = \bar{w}$ if the hard classifier has no other support vectors outside x_1 and x_2 .
- c. Explain why there cannot be only one support vector. Give an example with four support vectors.

8 Kernel methods

We refer, for example, to [15–20] for a nice and more extensive introduction to kernel methods. Among a large literature, we also mention [28] for the properties of positive kernels and their application via SVM algorithms for analyzing data; we also refer to [3] for applying kernel methods to the approximation of operators in Banach spaces, e. g., for solving PDE numerically.

8.1 Kernels: what/why?

8.1.1 Recalling the basic principles of linear regression

Before diving into the complexities of kernel methods, we will start by recalling some basic ideas of linear regression. We will also later introduce regularization, a process that mitigates model overcomplication, e. g., overparameterization. This groundwork of understanding linear regression and regularization will then set the stage for a more comprehensive explanation of kernel methods.

Example 1 (Exact linear regression). Consider a set T of datapoints $(x_i, y_i) \in \mathbb{R}^2, 1 \leq i \leq n$, on the plane $\mathbb{R}^2$ such that the datapoints can be fitted perfectly by a line $y = mx + b$ with $m = 1$ and $b = 1$. The points x_i are the inputs (or predictors), and y_i are the outputs (or targets). In the example on Figure 8.1, all the data set points lie perfectly on the straight line $y = x + 1$, which is easy to find by using two points in the data set.

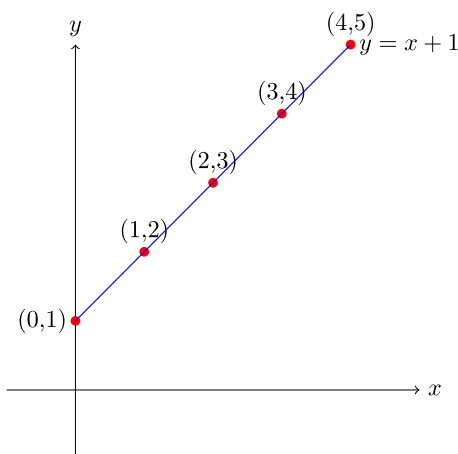


Figure 8.1: Illustration of exact linear regression with additional points for enhanced visualization. The blue line $y = x + 1$ perfectly fits all data points.

To find the line that fits the data using two points, we use the formula $(y_2 - y_1)/(x_2 - x_1)$ for the slope of a line. In this scenario, selecting any two points and applying the formula

will give us the same result, which verifies that the points lie on the same line. The value obtained is the slope of the line ($m = 1$), and substituting one point's coordinates and the slope into the line equation $y = mx + b$, we can solve for b , the y -intercept. This results in the exact line equation that fits the data points.

Example 2 (Approximation with linear regression). In this scenario, unlike the previous example, we will see that the line does not perfectly fit all the datapoints. Instead, we aim to find the best approximate line that represents the overall trend of the data points.

In the real world, data seldom perfectly align on a single line, and hence we often resort to approximate methods like this. Here, on Figure 8.2 the blue line, although not intersecting every point, gives a general representation of the data distribution. The aim in these situations is to minimize the distances between the data points and the regression line, and the line that accomplishes this best is deemed the line of best fit.

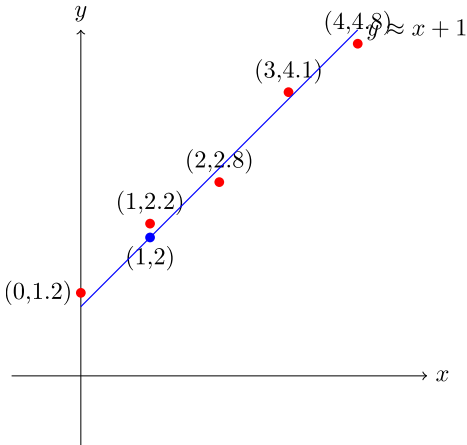


Figure 8.2: Example of linear regression. Data are given by points (x_i, y_i) in red. The blue line $y \approx x + 1$ solves the linear regression and represents the trend of the data, by predicting the output $(x_i, \hat{y}_i)$, a blue point. Observe that although the data do not lie on the predicted line, all predicted values are on the regression line.

To find the line of best fit, we use the method of minimization, specifically minimizing the mean squared error (MSE). The MSE measures the average of squared differences between the actual and predicted values:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n ((x_i w + b) - y_i)^2. \quad (8.1)$$

In this equation, (x_i, y_i) are the actual data points (x_i are inputs, and y_i are outputs/targets), and $(x_i w + b)$ are the y -values predicted by linear regression. We aim to find the values of w (weights) and b (bias) that minimize this error; that is, we want to find

$$\min_{w, b} \text{MSE}.$$

To solve this minimization problem, we take the derivatives with respect to the variables we want to optimize over (w and b). This will allow us to find the values of w and b at which the MSE is the smallest, hence yielding the line that best fits our data.

In terms of the weights w , we start by taking the derivative of the MSE function with respect to w , this as

$$\frac{\partial \text{MSE}}{\partial w}.$$

By setting this derivative to zero and solving for w , we can find the optimal value of the weight that minimizes the MSE.

For w , the derivative is

$$\frac{\partial \text{MSE}}{\partial w} = \frac{2}{n} \sum_{i=1}^n x_i (x_i w + b - y_i) = 0.$$

Rearranging the terms, we get

$$w = \frac{\sum_{i=1}^n x_i (y_i - b)}{\sum_{i=1}^n x_i^2}. \quad (8.2)$$

For b , the derivative is

$$\frac{\partial \text{MSE}}{\partial b} = \frac{2}{n} \sum_{i=1}^n (x_i w + b - y_i) = 0.$$

Solving for b , we get

$$b = \sum_{i=1}^n (y_i - x_i w). \quad (8.3)$$

8.1.2 How a linear kernel arises in linear regression

For scalars inputs x_i , the linear kernel is essentially the product of two input points and is defined as

$$K(x_i, x_j) = x_i x_j. \quad (8.4)$$

The kernel is linear in both variables x_i and x_j in that case. Notice that the denominator of expression (8.2) for w and the right-hand side of expression (8.3) for b can both be rewritten using the linear kernel.

First, we have

$$w = \frac{\sum_{i=1}^n x_i (y_i - b)}{\sum_{i=1}^n K(x_i, x_i)}. \quad (8.5)$$

We can also include the linear kernel in the expression for b using (8.3) and (8.4):

$$b = \sum_{i=1}^n (y_i - K(x_i, w)). \quad (8.6)$$

In fact, the MSE itself can be rewritten as

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n ((K(x_i, w) + b) - y_i)^2. \quad (8.7)$$

At this point, it is natural to wonder why introduce the kernel. A good example where the elegance of the kernel function truly manifests itself is the problem of linear ridge regression. Kernel methods are more attractive in large dimensions, and for this reason, we first recall various formulas for linear regression in higher dimensions.

8.1.3 Linear regression in general dimension

We now recall the case of linear regression in general dimension.

Given a dataset with n samples, each represented by a feature or “object” vector $x_i \in \mathbb{R}^m$ and a corresponding target or “label” value $y_i \in \mathbb{R}$. Hereafter a feature means a function $\phi(x_i)$ defined on the set of objects. We refer to the examples in Section 8.1.7.

We next try to find weights $\mathbf{w}$ and biases b to predict a value $\hat{y}_i$ of the form

$$\hat{y}_i = \mathbf{x}_i^T \mathbf{w} + b, \quad (8.8)$$

where

- $\mathbf{w} \in \mathbb{R}^m$: The weight vector representing the coefficients or weights assigned to each feature.
- b : The bias term, which is a scalar value added to the weighted sum of features. It does not depend on the features and is shared across all samples.
- y_i : The target value or label for the i th sample. It is a scalar value representing the predicted output for the given input $\mathbf{x}_i$.

To measure the quality of the model predictions, we define the loss function as the mean squared error (MSE), which quantifies the average squared difference between the predicted and true values:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - (\mathbf{x}_i^T \mathbf{w} + b))^2. \quad (8.9)$$

8.1.4 Ridge regression

Ridge regression (a. k. a. Tikhonov regularization) is a variant of linear regression where a regularization term is added to the loss function to prevent overfitting. The regularization term helps control the parameters $\mathbf{w}$ and $\mathbf{b}$ of our linear approximation by penalizing large weights. For simplicity, we assume that $\mathbf{b} = 0$ and the only parameters are the weights $\mathbf{w}$.

To introduce the regularization, we add a term to the loss function (8.9). In this case, we consider the L^2 regularization, also known as ridge regression, which penalizes the square of the weights of the parameters $\mathbf{w}$:

$$\text{Loss} = \text{MSE} + \lambda \|\mathbf{w}\|^2,$$

where $\|\cdot\|$ is the L^2 Euclidian norm, and λ is the *regularization parameter* that controls the trade-off between minimizing the MSE and minimizing the magnitude of the parameters $\mathbf{w}$ in L^2 .

To formulate the minimization problem in ridge regression, it is convenient to introduce a matrix form. Define

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & x_{13} & \dots & x_{1n} \\ x_{21} & x_{22} & x_{23} & \dots & x_{2n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ x_{m1} & x_{m2} & x_{m3} & \dots & x_{mn} \end{bmatrix}, \quad (8.10)$$

where

- The matrix $\mathbf{X}$ represents the input data for linear regression.
- It has m rows and n columns, where n is the number of samples, and m is the number of features.
- Each column i of the matrix corresponds to a single sample, the column vector $\mathbf{x}_i$.
- The element x_{ij} represents the value of the i th feature of the j th sample.

The minimization problem in ridge regression can now be expressed as a constrained minimization problem in variables $\mathbf{w}$, $\mathbf{r}$ for some chosen parameter λ (cf. the hyperparameter λ in soft-margin SVM):

$$\min_{\mathbf{w}, \mathbf{r}} \left[\frac{1}{n} \|\mathbf{r}\|^2 + \lambda \|\mathbf{w}\|^2 \right] \quad \text{with the constraint } \mathbf{X}^T \mathbf{w} - \mathbf{r} = \mathbf{y}, \quad (8.11)$$

where $\mathbf{y} = (y_1, \dots, y_n)$ is the known output or target vector, and $\mathbf{X}$ is given by (8.10).

In this formulation, we emphasize the following:

- $\mathbf{w}$ represents the weight vector in the linear regression model. In the simplified example where x_i are scalars, $\mathbf{w}$ represents a scalar. The vector $\mathbf{w}$ contains the parameters of the model that we want to learn.
- $\mathbf{r} = \mathbf{X}^T \mathbf{w} - \mathbf{y}$ represents the residuals or error of our model predictions, defined as the difference between the true target values $\mathbf{y}$ and our predicted values $\mathbf{X}^T \mathbf{w}$.

- The factor $\frac{1}{n}$ represents the averaging over the datapoints.
- λ is the regularization parameter. The value of λ determines the amount of regularization applied. If λ is too large, then the model may underfit the data, as it puts too much emphasis on minimizing the weights rather than fitting the training data. If λ is too small, then the model may overfit the data, as it does not sufficiently penalize large weights, cf. large margins in SVM. Therefore, λ is considered a hyperparameter, chosen before training. Choosing an appropriate value of λ is crucial for achieving a good balance between overfitting and underfitting.

In summary, the objective of ridge regression is to minimize the MSE with additional L^2 regularization.

8.1.5 Solving the ridge regression by the dual problem

Ridge regression is a constrained optimization problem, which can be solved using the method of Lagrange multipliers. Given an objective function $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ and a constraint $g : \mathbb{R}^2 \rightarrow \mathbb{R}$, the method of Lagrange multipliers is useful to solve

$$\min_{(x_1, x_2) \in \mathbb{R}^2} f(x_1, x_2) \quad \text{subject to} \quad g(x_1, x_2) = c.$$

We then define the corresponding Lagrangian

$$L(x_1, x_2; \alpha) = f(x_1, x_2) + \alpha (g(x_1, x_2) - c).$$

We find the critical points of the original problem by solving $\nabla L(x_1, x_2; \alpha) = 0$ where $\nabla L = (\partial_{x_1} L, \partial_{x_2} L, \partial_{\alpha} L)$ so that x_1, x_2, α enter in a symmetric way.

The Lagrangian for our problem (8.11) is

$$L(\mathbf{w}, \mathbf{r}, \boldsymbol{\alpha}) = \frac{1}{2} \|\mathbf{r}\|^2 + \lambda \|\mathbf{w}\|^2 + \boldsymbol{\alpha}^T (\mathbf{r} - \mathbf{X}^T \mathbf{w} + \mathbf{y}), \quad (8.12)$$

where $\boldsymbol{\alpha}$ is the column vector of Lagrange multipliers (vector instead of scalar as we have several constraints).

Setting the derivatives of the Lagrangian with respect to $\mathbf{w}$ and $\mathbf{r}$ to zero, we obtain

$$\nabla_{\mathbf{w}} L(\mathbf{w}, \mathbf{r}, \boldsymbol{\alpha}) = 2\lambda \mathbf{w} - \mathbf{X} \boldsymbol{\alpha} = 0,$$

$$\nabla_{\mathbf{r}} L(\mathbf{w}, \mathbf{r}, \boldsymbol{\alpha}) = \mathbf{r} + \boldsymbol{\alpha} = 0.$$

Solving for $\mathbf{w}$ and $\mathbf{r}$, we get

$$\mathbf{w} = \frac{1}{2\lambda} \mathbf{X} \boldsymbol{\alpha}, \quad (8.13)$$

$$\mathbf{r} = -\boldsymbol{\alpha}. \quad (8.14)$$

Substituting the above solutions of $\mathbf{w}$ and r into the Lagrangian to eliminate $\mathbf{w}$ and r yields the dual function for the unknown vector α of Lagrange multipliers,

$$\begin{aligned} g(\alpha) &= L(\mathbf{w}(\alpha), r(\alpha), \alpha) \\ &= \frac{1}{2}\|\alpha\|^2 + \frac{1}{4\lambda}\|\mathbf{X}\alpha\|^2 + \alpha^T \left(-\alpha - \frac{1}{2\lambda}\mathbf{X}^T\mathbf{X}\alpha + y \right) \\ &= -\frac{1}{2}\|\alpha\|^2 - \frac{1}{4\lambda}\|\mathbf{X}\alpha\|^2 + \alpha^T y = -\frac{1}{\lambda} \left[\frac{1}{4}\alpha^T (\mathbf{X}^T\mathbf{X} + \lambda 2\mathbf{I})\alpha - \lambda \alpha^T y \right]. \end{aligned} \quad (8.15)$$

We can now minimize $g(\alpha)$. Equivalently, due to minus sign in the RHS of (8.15), we can solve the dual maximization problem

$$\max_{\alpha} g(\alpha) = \min_{\alpha} \frac{1}{\lambda} \left[\frac{1}{4}\alpha^T (\mathbf{X}^T\mathbf{X} + \lambda 2\mathbf{I})\alpha - \lambda \alpha^T y \right]. \quad (8.16)$$

The solution for α is obtained as the critical point of (8.16):

$$\alpha = 2\lambda(\mathbf{X}^T\mathbf{X} + \lambda 2\mathbf{I})^{-1}y. \quad (8.17)$$

Then using (8.13), we get an explicit formula for the desired regression parameter $\mathbf{w}$:

$$\mathbf{w} = \mathbf{X}(\mathbf{X}^T\mathbf{X} + \lambda 2\mathbf{I})^{-1}y. \quad (8.18)$$

Regularization parameter λ :

Introducing the regularization parameter λ in the linear regression ensures a unique solution when $\lambda > 0$. This is because the matrix $\lambda \mathbf{I} + \mathbf{X}^T\mathbf{X}$ becomes positive definite, guaranteeing that it is non-singular. However, when $\lambda = 0$, the matrix may become singular, so an inverse may not exist, and there may exist several minima. The name ridge regression comes from the fact that after regularization with positive λ , the minimizer is unique, whereas without the regularization, the set of minimizers can form a ridge.

8.1.6 How linear kernel arises in ridge regression

The formula above can be reformulated by introducing the matrix corresponding to a linear kernel defined by

$$\mathbf{K} = \mathbf{X}^T\mathbf{X} = \begin{bmatrix} x_1 \cdot x_1 & x_1 \cdot x_2 & \cdots & x_1 \cdot x_n \\ x_2 \cdot x_1 & x_2 \cdot x_2 & \cdots & x_2 \cdot x_n \\ \vdots & \vdots & \ddots & \vdots \\ x_n \cdot x_1 & x_n \cdot x_2 & \cdots & x_n \cdot x_n \end{bmatrix}. \quad (8.19)$$

As before in the dual problem, we have that

$$g(\mathbf{a}) = -\frac{1}{\lambda} \left[\frac{1}{4} \mathbf{a}^T (\mathbf{K} + \lambda n \mathbf{I}) \mathbf{a} - \lambda \mathbf{a}^T \mathbf{y} \right].$$

The function $g(\mathbf{a})$ is maximized at

$$\mathbf{a} = 2\lambda(\mathbf{K} + \lambda n \mathbf{I})^{-1} \mathbf{y}.$$

Then using (8.13), we get the explicit formula for the desired regression parameter $\mathbf{w}$:

$$\mathbf{w} = \mathbf{X}(\mathbf{K} + \lambda n \mathbf{I})^{-1} \mathbf{y}, \quad (8.20)$$

and therefore the values predicted by the ridge regression are

$$\mathbf{X}^T \mathbf{w} = \mathbf{X}^T \mathbf{X}(\mathbf{K} + \lambda n \mathbf{I})^{-1} \mathbf{y} = \mathbf{K}(\mathbf{K} + \lambda n \mathbf{I})^{-1} \mathbf{y}. \quad (8.21)$$

Note that (8.21) predicts these values only in terms of the kernel $\mathbf{K}$ and a known target $\mathbf{y}$.

Of course, the formulations with and without the kernel are completely equivalent at the theoretical level. However, there can be several advantages of using the kernel, especially if the size of each input x_i is large. Assume that the number of features (dimension of an input vector) is very large: $m \gg 1$. In this case, $\mathbf{X}$ is an $m \times n$ matrix, and calculating quantities such as $\mathbf{X}^T \mathbf{X}$ is costly numerically, typically, $O(n^2 m)$ operations, see (10.15). Instead, if we have a method to evaluate each entry of $\mathbf{K}$ that does not depend on m , that is $O(1)$ operations we may directly evaluate the $n \times n$ kernel matrix $\mathbf{K}$, which would take $O(n^2)$ operations. Therefore, writing solution of ridge regression the problems in terms of the kernel $\mathbf{K}$, as given in (8.21) reduces computational complexity.

8.1.7 Feature space and feature map in classification and regression: examples

Example 3. We begin by presenting a simple example of a classification problem with two classes, which shows how a feature map from the space where data “lives” into a higher-dimensional space can transform nonlinear separable data into linearly separable data. In the example below these spaces are $\mathbb{R}^2$ and $\mathbb{R}^3$ respectively. Consider red and blue points on the left in Figure 8.3, which are clearly not linearly separable. By introducing the feature map $\phi(x, y) = (x, y, z = x^2 + y^2)$ from $\mathbb{R}^2$ into $\mathbb{R}^3$ the data become separable by a 2D plane.

Example 4. We now show a somewhat trivial example of how feature maps arise in regression and transform nonlinear regression problems into linear regression in the feature space. We consider a regression problem with a straightforward non-linear relation between the output and input, so that linear regression does not perform well. This can be illustrated by Figure 8.4, where the output points y_i can be best approximated by a quadratic function of the input points x_i : $y_i \approx x_i^2$. Obviously, this data cannot be well approximated by any linear function.

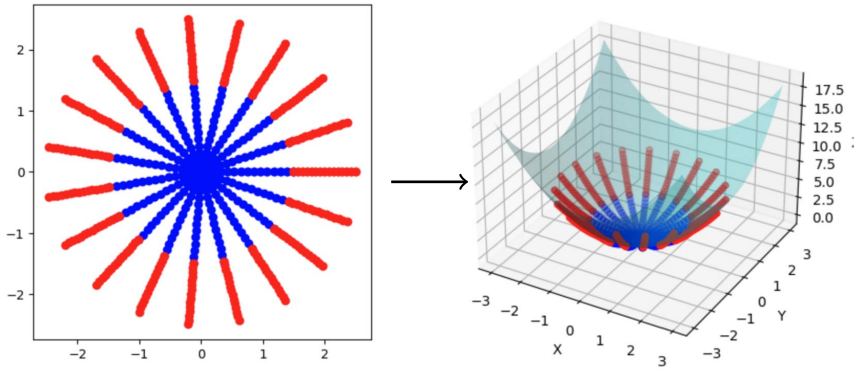


Figure 8.3: Feature map $\phi(x, y) = (x, y, z = x^2 + y^2)$. Feature map transforming a non-linearly separable problem in the original two-dimensional feature space (left) into a linearly separable problem in the three-dimensional feature space (right). Linear separation on the right is obtained by the horizontal cross-section $z = 1.5$.

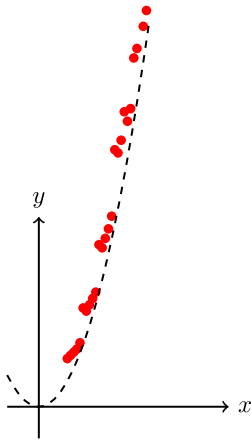


Figure 8.4: Example of non-linear relation between y_i and x_i . The dashed curve is $y = x^2$.

A possible solution in these situations is to use a feature map ϕ : instead of doing linear regression on (x_i, y_i) , we would do a linear regression on $(\phi(x_i), y_i)$, where the function ϕ is called a feature map. In this case, we can naturally choose $\phi(x) = x^2$. However, it is not always possible to choose a feature map with the same image space as the original space of objects, and instead the range of feature maps are often a higher-dimensional space.

Example 5. We now present a more general regression example and calculate explicitly the feature map and regression parameters.

Consider the following data: a set of blue points x_i lying inside a circle of radius 1 centered at the origin and a set of red points x_i lying outside a circle of radius 2; see Figure 8.5. In the context of facial recognition, we can think of the blue points as repre-

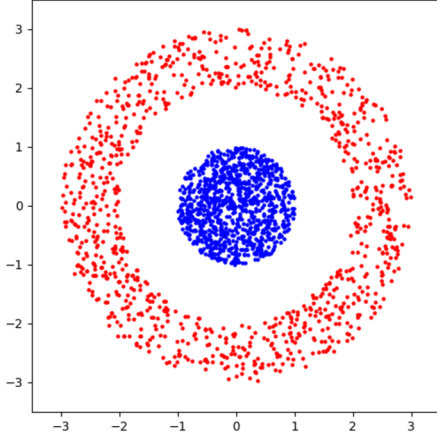


Figure 8.5: An example of feature of interest, the blue “eye”, surrounded by other points.

senting an eye (our feature of interest) and of the red points as the rest of the face. We choose as an output $y_i = x_i$, where x_i is a 2d vector inside the circle, and $y_i = (0, 0)$ for the red points where x_i is outside the circle.

The regression problem now consists in trying to find a function best matching the 2d vectors x_i of the input with the 2d vectors y_i of the output. We can also take a geometric perspective: we are finding a surface of co-dimension 2 closest to the pairs $(x_i, y_i) \in \mathbb{R}^4$. Linear regression here means that this function should be affine or the surface should be a flat plane because the output is two-dimensional; cf. Example on Figure 8.4 with one-dimensional input.

Again, linear regression is therefore not very useful with the original input in the original two-dimensional space. A linear function can match well the blue points of the dataset, for example, $y = x$, where x, y are two-dimensional vectors. Another linear function can match the red points, for example, $y = (0, 0)$. However, because of our choice of the output no linear function can do both at the same time. Furthermore, linear regression for both red and blue data will still not work well if we replace the x_i by $\phi(x_i)$ as long as the $\phi(x_i)$ are still two-dimensional and ϕ is smooth and injective. Indeed, in this case the data is $(\phi(x_i), x_i)$ for blue points and $(\phi(x_i), 0)$ for red points and due to the injectivity $\phi(x_i) \neq \phi(x_j)$, for $i \neq j$. Therefore, if one wants to approximate red points, the only choice is $y(x) \equiv (0, 0)$, which does not approximate blue points. Note that injectivity is helpful because otherwise information about data may be lost, e.g. the red points collapses to a point $(0, 0)$ on Figure 8.5. However, by transforming the data (red and blue points) into a higher-dimensional feature space we can achieve a much better linear regression analogous to the example in Figure 8.3, where blue and red points can be separated by a 2D plane in $\mathbb{R}^3$.

To this end, we introduce the following feature map $\phi : \mathbb{R}^2 \rightarrow \mathbb{R}^4$ that maps our original 2D feature space to a four-dimensional feature space. Specifically, we can define the feature map as

$$\phi(x) = (x, z) = (x, f(\|x\|^2)x) \in \mathbb{R}^4, \quad z \in \mathbb{R}^4, \quad (8.22)$$

with $f(t) : \mathbb{R} \rightarrow \mathbb{R}$ a function equal to 1 for $t \leq 1$ and 0 for $t \geq 2$. This function is chosen to mimic the different behavior of the output on the blue vs. red points, described in Figure 8.5, that is,

$$\phi(x) = \begin{cases} (x, x) & \text{if } |x| \leq 1 \text{ (blue points),} \\ (x, 0) & \text{if } |x| \geq 2 \text{ (red points).} \end{cases} \quad (8.23)$$

We apply this feature map ϕ to the original dataset, which yields a new dataset in the feature space $\mathbb{R}^4$. We then have an exact solution to the linear regression problem through the linear map $(x, z) \rightarrow z$ that maps $\phi(x_i)$ from the input feature space to $z = y_i$. Note that this map is a projection of the four-dimensional vector onto the last two coordinates. Of course, we could just choose $\phi(x) = f(\|x\|^2)x$ with output space $\mathbb{R}^2$, but this is naturally not injective, which can be a big issue as some information is lost. Specifically, we lose positional information of the data, e.g., points with $\|x\| > 2$ get mapped to 0. Thus the linear regression problem in which the target variables y_i are approximated by an affine function is given by the following minimization problem:

$$\min_{A, b} \|A\phi(x_i) + b - y_i\|. \quad (8.24)$$

The solution to (8.24) is

$$A = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad b = \begin{pmatrix} 0 \\ 0 \end{pmatrix}. \quad (8.25)$$

Note that in this problem, the solution of the regression problem is given by the linear map

$$y = A\phi(x). \quad (8.26)$$

Equation (8.26) shows that a nonlinear relationship between input x and output y becomes linear after applying a feature map from $\mathbb{R}^2$ to $\mathbb{R}^4$.

In the case of scalar outputs, linear regression has a simple geometrical interpretation as a hyperplane describing the linear relationship between the input and output. However, in this problem the output is two-dimensional, and there is no single hyperplane that corresponds to the linear map (8.26).

Example 6. We can now revisit the example on Figure 8.4. Consider the feature map $\phi(x) = (\phi_1(x), \phi_2(x)) = (x, x^2)$ from $\mathbb{R}$ to $\mathbb{R}^2$. Then the entire data vector is $(\phi(x), y) = (x, x^2, x^2)$, and the linear regression is given by the inner product $y = (0, 1) \cdot \phi(x)$. Note that this equation defines the hyperplane plane $0 \cdot \phi_1(x) + 1 \cdot \phi_2(x) - 1 \cdot y = 0$ in the space $\mathbb{R}^3$ where the axes are $(\phi_1(x), \phi_2(x), y)$, as illustrated on Figure 8.6, because we now have a scalar output unlike in the example above.

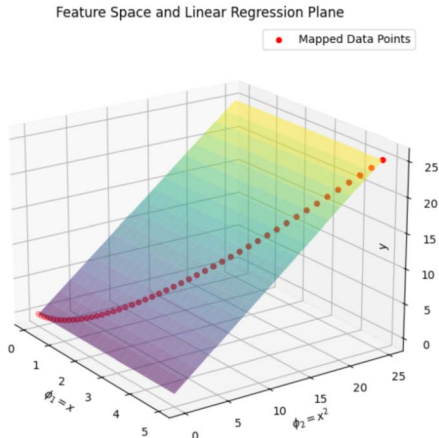


Figure 8.6: Linear regression of the curve $y = x^2$ is given by a 2D plane in $\mathbb{R}^3$.

Depending on the complexity of the problem, it may be necessary to choose a feature map ϕ with an image in a large-dimensional space, potentially infinite dimensional. Because we still want to be able to use inner products, the minimal requirement is that the output space is a Hilbert space. This leads to the following more general definition of feature maps:

Definition 8.1.1. Let $X \subset \mathbb{R}^d$, and let H be a Hilbert space with inner product $\langle \cdot, \cdot \rangle$. A *feature map* is a function $\Phi : X \rightarrow H$.

X is typically the space where the data lie, and H is a higher-dimensional space where the input and output data are linearly related, e. g., linearly separable. We refer to X as the *input feature space* and to H as the *output feature space*.

Feature maps offer the possibility of using linear regression or linear separation, even for nonlinear questions, provided that we can identify an adequate feature map ϕ . This comes at the possible numerical cost of having to work in a much higher-dimensional space. As we will see in the next section, kernel methods are essentially equivalent to feature maps at the theoretical level, with a potentially much reduced computational cost.

8.2 Kernel: definitions and basic properties

8.2.1 Definitions

Directly mapping features to a high-dimensional space can be computationally expensive, especially when the feature map is complex. In this section, we show that kernels offer a way to implicitly map the features to a high-dimensional space, avoiding the explicit computation of the feature map. The drawback is that choosing a kernel can be a complicated question. First, however, we have to make sure that the kernel function has some minimal properties that we list below.

Definition 8.2.1 (Kernel function). A function $K : \mathbb{R}^n \times \mathbb{R}^n \rightarrow \mathbb{R}$ is considered a valid kernel if it is both symmetric and positive definite:

- *Symmetry*: $K(x, z) = K(z, x)$ for all x, z in the input space.
- *Positive definiteness*: For any finite set of points $\{x_1, x_2, \dots, x_n\}$, the kernel matrix $K = [K(x_i, x_j)]_{i,j=1}^n$ is positive semi-definite. This means that for all real vectors $a = (a_1, a_2, \dots, a_n)$, we have the following inequality:

$$\sum_{i=1}^n \sum_{j=1}^n a_i a_j K(x_i, x_j) > 0. \quad (8.27)$$

The linear kernel in equation (8.4) satisfies these conditions, making it a valid kernel function.

8.2.2 The “kernel trick”: connecting feature maps and kernels

A fundamental result is that any kernel function can be obtained as a linear kernel in a higher-dimensional feature space,

Theorem 8.2.1 (Moore–Aronszajn, 1950). *For any set X , a function $K : X^2 \rightarrow \mathbb{R}$ is a valid kernel (e. g., it is symmetric and positive definite) if and only if there exists a mapping Φ from X into a Hilbert space H with a scalar product $\langle \cdot, \cdot \rangle$ such that for all $x_1, x_2 \in X$,*

$$K(x_1, x_2) = \langle \Phi(x_1), \Phi(x_2) \rangle. \quad (8.28)$$

Note that the right-hand side of (8.28) is linear in $\Phi(x_i)$ and therefore (8.28) defines a linear kernel $\bar{K}(\Phi(x_1), \Phi(x_2))$. The proof of Theorem 8.2.1 is beyond the scope of this book and relies on the Riesz representation theorem, which can be found in, e. g., [11].

The direct implication of this theorem shows that any kernel K on X^2 can be written as an inner product in H for some feature map. The converse shows that for any Hilbert space H with an inner product and for any feature map $\Phi : X \rightarrow H$, there is a kernel K such that (8.28) holds. This implies that rather than finding the feature map Φ that provides a better data structure in H , e. g., linearly separable, we can find the right kernel K defined on X^2 that acts like an inner product on the Hilbert space H .

We show that the kernel function $K(x_i, x_j)$ allows one to represent the loss function and the linear regression predictors in terms of the kernel only, see (8.31) and (8.37) below. To derive this formula we start from the MSE loss, (c.f. (8.1)

$$L(w) = \frac{1}{n} \sum_{i=1}^n \left(y_i - \langle w, \Phi(x_i) \rangle \right)^2. \quad (8.29)$$

To prevent overfitting and control the complexity of the solution, we can add a ridge (or Tikhonov) regularization term. For the ridge linear regression, the penalty is $\lambda \|w\|^2$.

By the Representer Theorem [60], w can be expressed in the following form with some unknown coefficients α_i (c.f. (8.13) where X is “replaced” by $\Phi(x)$)

$$w = \sum_{i=1}^n \alpha_i \Phi(x_i). \quad (8.30)$$

Now the loss becomes

$$L(\alpha) = \frac{1}{n} \sum_{i=1}^n \left(y_i - \sum_{j=1}^n \alpha_j \langle \Phi(x_i), \Phi(x_j) \rangle \right)^2 = \frac{1}{n} \sum_{i=1}^n \left(y_i - \sum_{j=1}^n \alpha_j K(x_i, x_j) \right)^2, \quad (8.31)$$

and the predictor function can be written in terms of the kernel as

$$\hat{f}(z) = \sum_{j=1}^n \alpha_j K(x_j, z), \quad z \in X. \quad (8.32)$$

Next, the norm of w can be expressed via kernel $K(x_i, x_j)$ as follows:

$$\begin{aligned} \|w\|^2 &= w^T w = \sum \alpha_i \Phi(x_i)^T \sum \alpha_j \Phi(x_j) = \sum \alpha_i \alpha_j \langle \Phi(x_i), \Phi(x_j) \rangle_{\mathcal{H}} \\ &= \sum_{i,j=1}^n \alpha_i \alpha_j K(x_i, x_j). \end{aligned} \quad (8.33)$$

Thus the regularized loss function for kernel ridge regression is

$$\text{Loss} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{i,j=1}^n \alpha_i \alpha_j K(x_i, x_j) \quad (8.34)$$

with

$$\hat{y}_i = \sum_{j=1}^n \alpha_j K(x_j, x_i). \quad (8.35)$$

By mimicking the derivation of (8.17) for linear ridge regression the closed-form solution for the coefficient vector α is given by

$$\alpha = (\mathbf{K} + n\lambda I)^{-1} \mathbf{y}, \quad (8.36)$$

where $\mathbf{K} = \{K(x_i, x_j)\}_{i,j=1}^n$ is the kernel matrix with entries $K(x_i, x_j)$ coming from the kernel function defined above by the Moore–Aronszajn theorem, I is the identity matrix, and $\mathbf{y} = (y_1, \dots, y_n)$ is a column vector of target values. Consequently, the predictor for any new input z is

$$\hat{f}(z) = \mathbf{y}^T (\mathbf{K} + n\lambda I)^{-1} \tilde{K}(z), \quad z \in X \quad (8.37)$$

with $\tilde{K}(z)$ denoting the vector $[K(x_1, z), K(x_2, z), \dots, K(x_n, z)]^T$.

This approach demonstrates how nonlinear regression via kernel methods naturally extends the ideas of linear regression. By employing feature maps and the kernel trick, we not only recover the linear case as a particular case (with $K(x_i, x_j) = x_i x_j$) but also allow for more general, nonlinear relationships between x_i and y_i , preserving the ridge regularization framework.

8.2.3 Choosing the right kernel: the example of the Gaussian kernel

Choosing an appropriate kernel for a given problem is an active area of research in machine learning. It depends on the nature of the data and the specific task at hand. Various factors like the dimensionality of the data, the underlying distribution, and the computational resources available can influence the choice of the kernel. There are no hard-and-fast rules, and often the selection of the kernel is done empirically.

The Gaussian kernel, also known as the heat kernel, is an example of a radial basis function (RBF) kernel. It is a popular choice in kernel methods as it measures the similarity between two data points based on their Euclidean distance in the feature space. The Gaussian kernel is defined as

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma^2}\right), \quad (8.38)$$

where $\mathbf{x}_i$ and $\mathbf{x}_j$ are the input data points as usual, and σ is a hyperparameter that controls the width of the kernel, chosen prior to the model training process.

We give a simple example of entries of a Gaussian kernel (GK) matrix $\mathbf{K}$ with entries defined in (8.38), where the data consists of two clusters (cluster 1 is blue, and cluster 2 is red). These entries describe the data in Figure 8.7 and are large for points from the same cluster and small for points from different clusters:

$$\mathbf{K} \approx \begin{bmatrix} K(x_i, x_j) \approx 1 & \cdots & K(x_i, x_j) \approx 1 & K(x_i, x_j) \approx 0 & \cdots & K(x_i, x_j) \approx 0 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ K(x_i, x_j) \approx 1 & \cdots & K(x_i, x_j) \approx 1 & K(x_i, x_j) \approx 0 & \cdots & K(x_i, x_j) \approx 0 \\ K(x_i, x_j) \approx 0 & \cdots & K(x_i, x_j) \approx 0 & K(x_i, x_j) \approx 1 & \cdots & K(x_i, x_j) \approx 1 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ K(x_i, x_j) \approx 0 & \cdots & K(x_i, x_j) \approx 0 & K(x_i, x_j) \approx 1 & \cdots & K(x_i, x_j) \approx 1 \end{bmatrix};$$

$K \approx 1$ for datapoints of the same cluster (color), and $K \approx 0$ for datapoints from different clusters.

Finally, we mention that radial basis function kernels are commonly used as their outputs are large for pairs of points (x_i, x_j) that are close to each other and small for pairs of points (x_i, x_j) that are far from each other. In many settings, it is helpful to have close points more correlated than points that are further away. The Gaussian kernel (GK) has

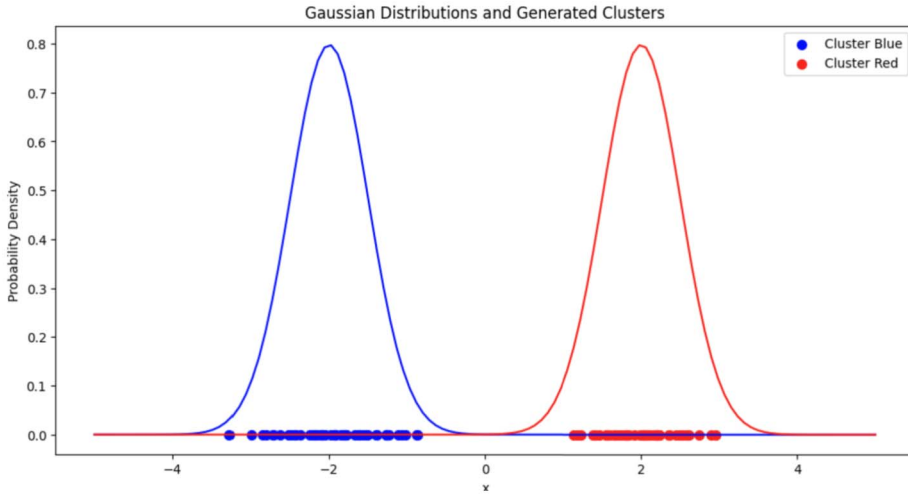


Figure 8.7: This figure illustrates two distinct clusters of data points on a 1D line. Each cluster corresponds to a different Gaussian distribution (blue and red points). This is where the Gaussian kernel becomes particularly effective as it assigns higher similarity to points that are closer (within the same cluster) and lower similarity to points that are farther (from different clusters). In this way, the Gaussian kernel can effectively distinguish two clusters.

other properties, which we will not discuss here, but which make it particularly useful in case of random data obtained from Gaussian distribution.

8.3 Exercises

Exercise 1 (Positive definiteness of the Gaussian kernel). The Gaussian kernel is defined

$$K(x, z) = \exp\left(-\frac{\|x - z\|^2}{2\sigma^2}\right), \quad x, z \in \mathbb{R}^d,$$

with $\sigma > 0$. Prove that K is indeed a kernel by proving the following properties:

- (a) *Symmetric*, that is, $K(x, z) = K(z, x)$ for all x, z , and
- (b) *Positive definite*, that is, for any finite set of points $\{x_1, \dots, x_n\} \subset \mathbb{R}^d$ and any real coefficients $a_1, \dots, a_n$,

$$\sum_{i,j=1}^n a_i a_j K(x_i, x_j) \geq 0.$$

(Hint: Use the following result: A function $f(t)$ is completely monotone for $t \geq 0$ if and only if $f(t^2)$ is positive definite (see [59]). (A function f is completely monotone if $(-1)^k f^{(k)}(t) \geq 0$ for all $k \in \mathbb{N}$ and t .)

Exercise 2 (Invertibility via ridge regularization and the dual formulation). In ridge regression, we add an L^2 penalty to mitigate overfitting. In the dual formulation of ridge regression, after introducing the Lagrange multipliers α , we obtain the solution

$$\alpha = 2\lambda (X^T X + n\lambda I)^{-1} y,$$

where $X \in \mathbb{R}^{m \times n}$ is the data matrix (with each column representing a sample), I is the $n \times n$ identity matrix, and $\lambda > 0$ is the regularization parameter. Here, the matrix

$$M = X^T X + n\lambda I$$

appears in the inverse that provides the solution. It is essential for M is invertible. Prove that M is positive definite (and hence invertible), even if $X^T X$ is singular.

Hint: Show that for any nonzero vector $z \in \mathbb{R}^n$, the quadratic form

$$z^T M z > 0$$

is strictly positive.

Exercise 3 (Computational advantage of the dual formulation). For ridge regression, the primal solution is given (when working with the feature matrix $X \in \mathbb{R}^{m \times n}$ with m features and n samples) by

$$w = (XX^T + \lambda I)^{-1} Xy,$$

whereas the dual (or kernel) formulation leads to the solution

$$w = X(X^T X + n\lambda I)^{-1} y.$$

Assume that the number of features is much larger than the number of samples, i. e., $m \gg n$. Provide an analysis comparing the computational complexity (read Section 10.1) of forming and inverting the matrices in the primal and dual formulations. Conclude which formulation is preferable and why.

9 Gradient descent method in the training of DNNs

In several regards, training is a key feature of ML algorithms, as it is when the algorithms learn from data; see Chapters 3–7. In the context of *supervised learning*, the training of DNNs can often be represented through an optimization process that consists of the minimization of a so-called loss function $L(\mathbf{a})$. Here $\mathbf{a}$ is the vector of DNN parameters introduced in Definition 4.1.3 within the simple framework we consider.

Critical or stationary points of a multivariable function $L(\mathbf{a})$ are the points $\mathbf{a}$ solving the equation $\nabla L(\mathbf{a}) = 0$. Critical points of course include local minima, local maxima, and saddle points, which can be distinguished by the second derivative test (testing the positive definiteness of the Hessian matrix).

For simple functions in low dimension, it is sometimes possible to use this to analytically find local minima or even global minima by comparing the values of $L(\mathbf{a})$ at those local minima. For DNN loss functions, such an analytical approach is impractical in general: The analytical expressions for the functions $L(\mathbf{a})$, $\nabla L(\mathbf{a})$, and $\nabla^2 L(\mathbf{a})$ are not expected to be simple. Moreover, the dimension μ of the parameter space $\mathbb{R}^\mu$ is very large, so that finding the exact solution of the system $\nabla L(\mathbf{a}) = 0$ does not appear feasible.

Minimizing in practice this type of function with a very large number of parameters is hence a difficult problem, for which several numerical approaches have been proposed. We briefly discuss here some common numerical optimization methods for DNN training based on the notion of *gradient descent* (GD). The idea behind such methods is simply that a multivariable function decreases fastest in the direction of the negative gradient of the function. Heuristically, it can be compared to the steps a hiker could take to descend from a forested hillside when they cannot see the bottom of the hill through the trees. In this case the hiker seeks to minimize their elevation while only being able to see what is nearby, that is, locally. Therefore a possible hiker's strategy is to take each step in the direction that leads the fastest downhill locally, without considering future steps to the global minimum. As we can see from this analogy, establishing the convergence of gradient descent methods to a global minimum could be a challenge, especially in the presence of several local minima.

We will consider deterministic and stochastic gradient descent (SGD) methods. We will also very briefly mention a generalization of GD called GD with the *momentum* or *heavy ball method* [54]. The idea is to add inertia to prevent “getting stuck” or “slowing down” in local minima due to oscillations in the loss function $L(\mathbf{a})$. Both SGD and GD with momentum help reduce the computation time and help deal with local minima and saddle points.

9.1 Deterministic gradient descent for the minimization of multivariable functions

We start with the deterministic *gradient descent (GD) method*, which is an iterative process where we update the parameters at each step along the direction of the gradient of the loss function.

The gradient descent method seems to have first been introduced by Cauchy in the mid-nineteenth century and rediscovered by Hadamard at the beginning of the twentieth century. Since then, it has become one of the most extensively studied numerical algorithms and has demonstrated its usefulness for finding local minima of functions; see, for instance, [53]. In the specific case of convex functions, several extensions have been proposed to improve the convergence rate while remaining at first order (e.g., involving only the gradient of the function to be minimized rather than higher-order derivatives), for which we refer in particular to [49].

Consider a point $\mathbf{a}$ together with a direction $\boldsymbol{\gamma}$. We say that a function L is decreasing in the direction of $\boldsymbol{\gamma}$ if the derivative of L in the direction of $\boldsymbol{\gamma}$ is negative, i.e.,

$$\nabla L(\mathbf{a}) \cdot \boldsymbol{\gamma} < 0. \quad (9.1)$$

We recall the elementary identity for the dot product on $\mathbb{R}^\mu$:

$$\nabla L(\mathbf{a}) \cdot \boldsymbol{\gamma} = \|\nabla L(\mathbf{a})\| \|\boldsymbol{\gamma}\| \cos \theta, \quad (9.2)$$

where θ is the angle between the vectors $\nabla L(\mathbf{a})$ and $\boldsymbol{\gamma}$. Therefore the directional derivative (9.2) is maximized when $\theta = 0$ ($\boldsymbol{\gamma}$ is in the direction of the gradient) and minimized when $\theta = \pi$ ($\boldsymbol{\gamma}$ is in the direction of the negative gradient).

This can be used to find another point $\tilde{\mathbf{a}}$ from $\mathbf{a}$ where L is lower. By the Taylor formula, provided that L is smooth,

$$L(\mathbf{a} + \tau \boldsymbol{\gamma}) = L(\mathbf{a}) + \tau \nabla L(\mathbf{a}) \cdot \boldsymbol{\gamma} + O(\tau^2).$$

We give a precise definition of the big $O(\cdot)$ notation in Definition 10.1.1.

If τ is small enough, then our best “guess” for a new point $\tilde{\mathbf{a}}$ naturally consists in taking

$$\tilde{\mathbf{a}} = \mathbf{a} + \tau \boldsymbol{\gamma} \quad \text{with } \boldsymbol{\gamma} = -\nabla L(\mathbf{a}).$$

This is the idea behind GD algorithms: from a random initial point in the parameter space, move to the next point by choosing the appropriate direction of the gradient. Continue iteratively until the desired precision in minimization is achieved.

In more mathematical terms, we begin with a randomly chosen initial parameter vector $\mathbf{a}^{(0)} \in \mathbb{R}^\mu$. Then we iteratively calculate a sequence of parameters $(\mathbf{a}^{(n)})_{n \in \mathbb{N}}$ according to the following rule:

$$\mathbf{a}^{(n)} = \mathbf{a}^{(n-1)} - \tau \nabla L(\mathbf{a}^{(n-1)}). \quad (9.3)$$

The *learning rate* τ (also called step size) governs how big the “step” from $\mathbf{a}^{(n-1)}$ to $\mathbf{a}^{(n)}$ is. It represents the speed of learning in the sense that it controls how much “new information” $\tau \nabla L(\mathbf{a}^{(n-1)})$ is added to the “old information” $\mathbf{a}^{(n-1)}$ to decide on the new parameter value $\mathbf{a}^{(n)}$.

The learning rate has to be small enough because the gradient $\nabla L(\mathbf{a})$ contains only local information about L in a small neighborhood of $\mathbf{a}$. In the example of the hiker, a huge miles-long “step” in the downhill direction could take the hiker to the top of another hill rather than to the bottom of their current hill. However, a learning rate that is *too* small means that we may take many more iterations than necessary, leading to training taking an excessively long time. Therefore it is important to strike a balance when choosing τ .

Example 9.1.1 (Choosing τ). Use GD to approximate the minimum of $F(x) = x^2$ (which is obviously attained at $x = 0$) with initial value $x_0 = 1$ and different values of τ . Table 9.1 illustrates that choosing very small τ leads to convergence taking many iterations, whereas for large τ , the GD algorithm does not converge at all.

Table 9.1: Steps for gradient descent with different τ .

τ	Smallest N such that $ x^n - 0 < 0.001$ for all $n > N$
$\tau = 0.001$	$N = 3451$
$\tau = 0.01$	$N = 342$
$\tau = 0.1$	$N = 31$
$\tau = 0.5$	$N = 1$
$\tau = 1.0$	does not converge

Ideally, the iterative steps of the GD algorithm lead to a minimizer $\bar{\mathbf{a}}$, a point that minimizes $L(\mathbf{a})$, as illustrated in Figure 9.1. This figure also demonstrates a natural geometrical interpretation of the algorithm: At each step of the GD, the gradient is perpendicular to the nearby level set.

More specifically, if $L(\mathbf{a}_0) = c$, then the gradient $\nabla L(\mathbf{a}_0)$ is perpendicular to the level set $\{\mathbf{a} \in \mathbb{R}^\mu : L(\mathbf{a}) = c\}$. This can be easily proved by considering any curve $\mathbf{r} : \mathbb{R} \rightarrow \mathbb{R}^\mu$ such that $\mathbf{r}(0) = \mathbf{a}_0$ and $L(\mathbf{r}(t)) = c$ for all t . Then, on one hand,

$$\left. \frac{d}{dt} L(\mathbf{r}(t)) \right|_{t=0} = \frac{d}{dt} c = 0, \quad (9.4)$$

whereas, on the other hand,

$$\left. \frac{d}{dt} L(\mathbf{r}(t)) \right|_{t=0} = \nabla L(\mathbf{r}(0)) \cdot \mathbf{r}'(0) = \nabla L(\mathbf{a}) \cdot \mathbf{r}'(0). \quad (9.5)$$

Since $\mathbf{r}'(0)$ can be any tangent vector to the level set $L(\mathbf{a}) = c$ at the point $\mathbf{a} = \mathbf{a}_0$, we conclude that $\nabla L(\mathbf{a}_0)$ is perpendicular to the level set.

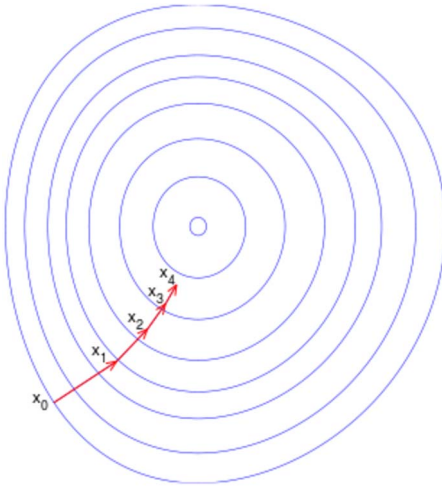


Figure 9.1: How GD works. *Blue curves:* level set of the function $L(\mathbf{a}) : \mathbb{R}^2 \rightarrow \mathbb{R}$ (c -level set = $\{\mathbf{a} \in \mathbb{R}^2 \mid L(\mathbf{a}) = c\}$, c is smaller in the center). *Red arrows:* direction of the negative gradient (perpendicular to the level set).

However, a key question for GD algorithms is whether the sequence $(L(\mathbf{a}_n))_{n \in \mathbb{N}}$ converges to a minimal value $L(\bar{\mathbf{a}})$. There are actually many reasons why convergence to a minimizer would fail: for example, convergence may occur to a local minimum. There are also cases where the algorithm completely fails to converge to any point.

Just as for separating sets, the notion of convexity is very useful to guarantee convergence when applied to functions.

Definition 9.1.1. A function $L : \mathbb{R}^d \rightarrow \mathbb{R}$ is *convex* if

$$L(t\mathbf{a}_1 + (1-t)\mathbf{a}_2) \leq tL(\mathbf{a}_1) + (1-t)L(\mathbf{a}_2) \quad \forall \mathbf{a}_1, \mathbf{a}_2 \in \mathbb{R}^d, t \in [0, 1]. \quad (9.6)$$

Inequality (9.6) means that the line segment joining the points $(\mathbf{a}_1, L(\mathbf{a}_1))$ and $(\mathbf{a}_2, L(\mathbf{a}_2))$ lies above the graph of $L(\mathbf{a})$; see Figure 9.2.

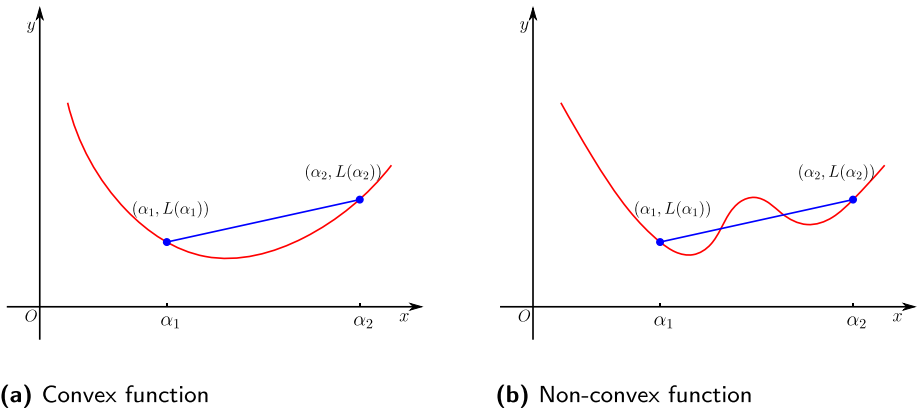


Figure 9.2: Convex and non-convex function.

Convex functions commonly play an important role in optimization, not only for GD algorithms. A first helpful property is that local and global minima are identical for convex functions.

Theorem 9.1.1 (Local vs. global minimum). *Any local minimum of a convex function $L : \mathbb{R}^d \rightarrow \mathbb{R}$ is also a global minimum.*

The next theorem shows that the GD are guaranteed to perform well on convex functions.

Theorem 9.1.2 (Convergence of GD). *Suppose a function $L : \mathbb{R}^d \rightarrow \mathbb{R}$ has the following properties:*

- (i) L is convex and differentiable,
- (ii) L admits a (global) minimum at $\bar{\mathbf{a}} \in \mathbb{R}^d$, and
- (iii) ∇L is Lipschitz with constant d , i. e., $\|\nabla L(\mathbf{a}_1) - \nabla L(\mathbf{a}_2)\| \leq d\|\mathbf{a}_1 - \mathbf{a}_2\|$ for all $\mathbf{a}_1, \mathbf{a}_2$ (where $\|\cdot\|$ is the Euclidean norm).

Assume that $\tau \leq 1/d$. Then the iterations $\mathbf{a}^{(k)}$ of the GD algorithm satisfy

$$L(\mathbf{a}^{(k)}) - L(\bar{\mathbf{a}}) \leq \frac{\|\mathbf{a}^{(0)} - \bar{\mathbf{a}}\|^2}{2\tau k}. \quad (9.7)$$

Intuitively, this theorem means that gradient descent is guaranteed to converge if the learning rate is small enough, and it converges with rate $O(1/k)$.

However, there is no general rule for choosing the optimal τ . Furthermore, we can choose different τ_k for each step in the iteration.

9.2 Additive loss functions

The use of SGD algorithms typically requires a special structure on the loss function. Here we consider so-called additive loss functions, which easily allow us to explain the basic SGD method and have the form

$$L(\mathbf{a}) = \frac{1}{|T|} \sum_{s \in T} \ell(s, \mathbf{a}), \quad (9.8)$$

where $\ell(s, \mathbf{a})$ quantifies the quality of approximation for each individual object $s \in T$.

A simple example of function $\ell(s, \mathbf{a})$ is the squared distance between $\phi(s, \mathbf{a})$ and $\psi(s)$ in the Euclidean norm:

$$\ell(s, \mathbf{a}) = \|\phi(s, \mathbf{a}) - \psi(s)\|^2. \quad (9.9)$$

If (9.9) is used for $\ell(s, \mathbf{a})$ in (9.8), then the loss function $L(\mathbf{a})$ is usually called the *mean square loss*.

In (9.8), $|T|$ is the number of objects in T , and therefore the sum $1/|T| \sum_{s \in T}$ in (9.8) represents the averaging of the loss over all elements in T . The minimization of $L(\mathbf{a})$ hence hopefully leads to minimizing $\ell(s, \mathbf{a})$ for most objects $s \in T$.

For DNNs solving the classification problem, another typical choice is the *cross-entropy loss* functions, which generalize the cross-entropy loss for logistic regression (6.7). Recall that if a DNN ϕ is used for classification, then it has the form given in Definition 4.2.1, and $\phi(s, \mathbf{a}) = (p_1(s, \mathbf{a}), \dots, p_m(s, \mathbf{a}))$, a vector of probabilities that s belongs to each of m classes. We then define the cross-entropy loss as an average over the training set T :

$$L(\mathbf{a}) = -\frac{1}{|T|} \sum_{s \in T} \log(p_{i(s)}(s, \mathbf{a})), \quad (9.10)$$

where $p_{i(s)}(s, \mathbf{a})$ (from (3.3) and (3.4)) is the probability predicted by the DNN that s belongs to its correct class $i(s)$.

Each term $-\log(p_{i(s)}(s, \mathbf{a}))$ in the sum (9.10) is the *loss contributed by object s* . If $p_{i(s)}(s, \mathbf{a}) \approx 0$, then ϕ does not classify s very well, and the loss contributed by s is large. On the other hand, if $p_{i(s)}(s, \mathbf{a}) \approx 1$, then ϕ does a good job classifying s , and the loss contributed by s is small. By choosing $\mathbf{a}$ so that $L(\mathbf{a})$ is minimized we expect that on average, ϕ classifies each object $s \in T$ as well as possible.

9.3 What are SGD algorithms? When to use them?

Methods based on so-called stochastic approximation have become popular in a wide range of problems, as showcased in particular in [63]. The earliest examples of such methods appear to have been introduced in [33, 55]. In the specific context of machine learning, SGD algorithms and their many variants are nowadays the standard method to optimize artificial neural networks when combined with backpropagation; see, for example, [24, 41]. Stochastic gradient descent is a particularly helpful method when the training set is large, as explained, for instance, in [10].

Specifically, minimizing a loss function of the form (9.8) through deterministic GD involves calculating $\nabla \ell(s, \mathbf{a})$ for each object at each iteration step to estimate the sum over all objects. For large training sets, this necessarily results in a high computational cost.

On the other hand, the design of stochastic gradient descent (SGD) algorithms *reduces the number of such operations*. The main idea is to choose a random sample of objects from the training set T , called a *batch*, to calculate the gradient at each iteration step instead of using the entire training set. Thus, whereas GD uses (9.3) for its iterative step, SGD uses

$$\mathbf{a}^{(n)} = \mathbf{a}^{(n-1)} - \tau \nabla \frac{1}{|T_n|} \sum_{s \in T_n} \ell(s, \mathbf{a}), \quad (9.11)$$

where $T_n \subset T$ is the batch used at the n th iteration (a randomly chosen batch for each iteration).

In a simple version of SGD, the number of objects in each batch is fixed over all iterations, and this number is called the *batch size*. When compared with deterministic GD, SGD leads to the following differences:

- Reduction of computational complexity (lowering of the computational cost) since we only use some of the terms in (9.8). We can usually expect the computation time of each iteration to scale linearly with the batch size.
- We do not compute the same gradient at each time step. Ideally, SGD calculates a reasonable approximation of the gradient of L . In practice, the two can be significantly different. Of course, the fewer terms we use (the smaller the batch size), the more likely it is. A first consequence, which is often very noticeable in practice, is that we may have large stochastic fluctuations for small batch sizes: From one iteration to the next, we may choose very different objects and obtain very different gradients of descent. For similar reasons, SGD algorithms may also require more time steps to converge than deterministic GD; the total computational cost may still be lower for SGD as each time step is much less costly. There are also advantages to not calculating the same gradient as in GD, as this can better avoid trapping the loss function in local minima.

For all these reasons, making an appropriate choice of batch size for SGD algorithms may require some trial and errors to get a good compromise between gains in computational times and better behavior.

9.4 Epochs in SGD

The notion of epochs is motivated by the fact a batch does not “cover” the entire training set T and is often much smaller than T . Intuitively, an epoch is the number of batches needed to cover the entire training set. More precisely, an epoch is a family of iterations of the following size:

$$\# \text{ of iterations in an epoch} = \frac{\text{size of training set}}{\text{batch size}} = \frac{|T|}{|T_n|}, \quad (9.12)$$

assuming that we do not change the batch size T_n within an epoch.

The notion of epoch is first useful when comparing the performance of two implementations of the GD or SGD algorithms. To see which algorithm is more effective, for example, we would like to compare the computational cost to achieve some given level of precision. However, when comparing two implementations of SGD with different batch sizes, we cannot just use the number of iterations/steps. This is because for the implementation with smaller batch size, each iteration is faster than in the implementation with larger batch size.

Using the number of epochs instead of steps allows for a more direct comparison. Observe that if an SGD algorithm runs for one epoch, then it has selected a total of $|T|$ elements of the training sets when choosing random batches. Therefore two algorithms with different batch sizes will still perform the same number of calculations of $\nabla \ell(s, \mathbf{a})$ over an epoch. Similarly, an SGD algorithm that ran for one epoch will have taken the same number of gradients as a GD algorithm that ran for one iteration over the entire training set and may therefore be compared fairly.

In some settings, epochs also play a more concrete role in the execution of the algorithm depending on how each batch is randomly selected. A thorough discussion of how to construct batches is beyond the scope of these notes, but to remain at an elementary level, we may distinguish between two cases:

- The objects in the batch T_n are selected randomly, independently of the choices made in the previous batches. This implies that the same object may be selected again in consecutive batches.
- We may instead use entirely distinct batches. This can be done by imposing that we cannot take the same object twice in different batches within the same epoch.

In the second case, we can then guarantee that each object will have been used exactly once within each epoch of the algorithm.

9.5 Weights

A further question when implementing GD or SGD algorithms is whether to weight objects equally or to give more importance to some of them.

There are naturally two occurrences in an SGD algorithm where distinct objects' weights could play a role. First of all, when randomly selecting a subset of T as our batch for each iteration, we could decide that some objects should be more likely to appear in a batch than others.

Similarly, we could decide to weight differently the various contributions in the loss function: Instead of (9.8), we could use

$$L(\mathbf{a}) = \sum_{s \in T} v(s) \ell(s, \mathbf{a}),$$

where $v(s)$ is the weight of the object s , and those weights are normalized so that $\sum_{s \in T} v(s) = 1$.

Weighting directly the loss function does not involve the notion of batch and can also be used in deterministic GD algorithms. For SGD weighting, the loss function or weighting the likelihood of selecting an object may have similar effects: In both cases the gradient $\nabla \ell(s, \mathbf{a})$ will be used more for some objects s than for others.

As is often the case, there is no general rule on whether to weight equally all objects or not. For example, if a training set contains objects from two classes, but there are

significantly more objects from one class than from the other, then we may weigh the objects from the under-represented class so that on average each batch contains as many objects from one class as from the other.

9.6 Choosing the batch size through a numerical example

To conclude our discussion of GD algorithms, we illustrate through a numerical example how we may choose in practice an appropriate batch size for SGD. We consider a relatively simple setting where we can calculate the analytical solution and use it when evaluating the numerical approximations.

We focus on a regression problem to find a function $f : \mathbb{R} \rightarrow \mathbb{R}^n$ that best fits our data. The data set consists of N pairs $(x_i, \mathbf{y}_i)$ with $x_i \in \mathbb{R}$ and $\mathbf{y}_i \in \mathbb{R}^n$ for $i = 1, \dots, N$. We want to approximate $f(x)$ by $\tilde{f}(x, \mathbf{a})$, where $\mathbf{a} \in \mathbb{R}^\mu$ is a vector of parameters. Specifically, we choose $\tilde{f}$ to have the form

$$\tilde{f}(x, \mathbf{a}) = (\tilde{f}_1(x, \mathbf{a}), \dots, \tilde{f}_n(x, \mathbf{a})), \quad (9.13)$$

where each $\tilde{f}_i(x, \mathbf{a})$ is a polynomial of degree d in x . Since each of the n polynomials of degree d has $d + 1$ coefficients (parameters), the parameter vector $\mathbf{a}$ has $n(d + 1)$ components, that is, the dimension μ of parameter space is $n(d + 1)$.

We can first find analytically the parameters $\mathbf{a}$ that yield the best fit $\tilde{f}$ by minimizing the mean square loss (6.2) from Section 6.1,

$$L(\mathbf{a}) = \frac{1}{N} \sum_{i=1}^N \|\tilde{f}(x_i, \mathbf{a}) - \mathbf{y}_i\|^2. \quad (9.14)$$

Observe that each polynomial $\tilde{f}_i(x, \mathbf{a})$ depends linearly on its coefficients. Therefore $\tilde{f}(x_i, \mathbf{a})$ depends linearly on $\mathbf{a}$. It is convenient to rearrange the vector $\mathbf{a}$ as an $n \times (d + 1)$ matrix A with the i th row containing the coefficients of the polynomial $\tilde{f}_i$. Then

$$L(\mathbf{a}) = L(A) = \frac{1}{N} \sum_{i=1}^N \|A\hat{\mathbf{x}}_i - \mathbf{y}_i\|^2 \quad (9.15)$$

with $\hat{\mathbf{x}}_i = (1, x_i, x_i^2, \dots, x_i^d) \in \mathbb{R}^{d+1}$, so that $A\hat{\mathbf{x}}_i$ is a vector containing the values of the n polynomials $\tilde{f}_i$ at the point x_i .

The gradient $\nabla L(A)$ is now explicitly given by

$$\nabla L(A) = \frac{2}{N} \sum_{i=1}^N (A\hat{\mathbf{x}}_i - \mathbf{y}_i)\hat{\mathbf{x}}_i^T. \quad (9.16)$$

Recall that the minimizer $A_{\min}$ of $L(A)$ satisfies $\nabla L(A_{\min}) = 0$. Therefore $A_{\min}$ can be calculated from (9.16) as

$$A_{\min} = \left(\sum_{i=1}^N \mathbf{y}_i \hat{\mathbf{x}}_i^T \right) \left(\sum_{i=1}^N \hat{\mathbf{x}}_i \hat{\mathbf{x}}_i^T \right)^{-1}, \quad (9.17)$$

where $\mathbf{y}_i \hat{\mathbf{x}}_i^T$ and $\hat{\mathbf{x}}_i \hat{\mathbf{x}}_i^T$ are matrices with $(\mathbf{y}_i \hat{\mathbf{x}}_i^T)_{jk} = (\mathbf{y}_i)_j (\hat{\mathbf{x}}_i)_k$ and $(\hat{\mathbf{x}}_i \hat{\mathbf{x}}_i^T)_{jk} = (\hat{\mathbf{x}}_i)_j (\hat{\mathbf{x}}_i)_k$, respectively. The minimizing matrix $A_{\min}$ of $L(A)$ corresponds to a minimizing vector $\mathbf{a}_{\min}$ of $L(\mathbf{a})$.

We may expect that since it is possible to explicitly calculate the minimizer $\mathbf{a}_{\min}$, there is no need to use numerical methods like SGD to approximate it. However, first of all, if N , n , and d are large, then calculating (9.17) is computationally expensive, particularly due to the need to invert the matrix $\sum_{i=1}^N \hat{\mathbf{x}}_i \hat{\mathbf{x}}_i^T$. In this case, it may still be faster to use SGD, particularly because the batch size may be much smaller than N .

Second, we also stress again that in more complex examples, it may be impossible to calculate an exact analytical solution. This example was specifically chosen so that the analytical solution exists and can be compared to numerical implementations of SGD.

We now turn to the implementation of SGD algorithms to minimize (9.14) for $N = 2^{13}$, $n = 64$, and $d = 3$. Our objective is to show how the batch size in the SGD algorithm changes the rate of convergence. We run SGD twelve times with batch sizes 2^k for $k = 0, 1, 2, \dots, 11$. At each run, we train for 200 epochs, recording the loss and computation time (in seconds) at the end of each epoch.

Let $\hat{\mathbf{a}}$ be the minimizer calculated by (9.17), and let $L_{\min} = L(\hat{\mathbf{a}})$ be the global minimum of the loss function. We introduce the *relative loss*

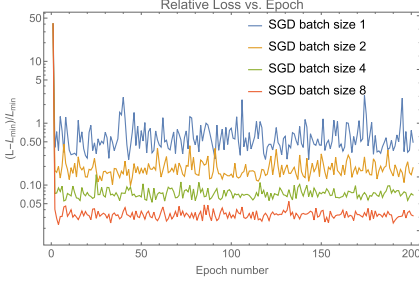
$$\bar{L}(\mathbf{a}) = \frac{L(\mathbf{a})}{L_{\min}} - 1, \quad (9.18)$$

which shows how close $L(\mathbf{a})$ is to $L_{\min}$.

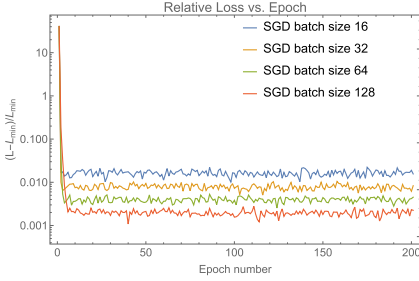
Figures 9.3a–9.3c show how the relative loss changes with the epoch number for each batch size. We observe from Figure 9.3 that for each batch size b , the loss function first rapidly decreases with the number of epoch number, before roughly leveling out with rapid oscillations about some value of relative loss $\bar{L}_b$. Observe that $\bar{L}_b$ decreases as b increases, but the number of epochs required to reach $\bar{L}_b$ increases with b . Thus larger batch sizes lead to higher precision than smaller batch sizes but may take longer to level out (see, in particular, Figure 9.3c).

Figure 9.4 shows the amount of computation time¹ (in seconds) required for SGD algorithm to reach the relative loss value of 0.04 for each batch size. Observe that (i) the smallest batch sizes never even reach this level of relative loss value, (ii) large batch sizes take a comparatively longer time, (iii) the optimal batch size (shortest time to reach this loss) appears to be about 32.

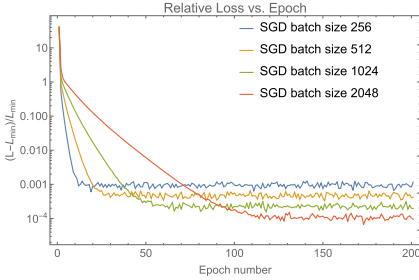
¹ The code for this example was written in Mathematica 11.0 and ran on an 11th Gen Intel(R) Core(TM) i7-1165G7 @ 2.80 GHz.



(a) Loss vs. epoch for small batch sizes.



(b) Loss vs. epoch for medium batch sizes.



(c) Loss vs. epoch for large batch sizes.

Figure 9.3: Log plots of loss vs. epoch for several batch sizes of SGD and deterministic GD. Note that the vertical axis is not the actual mean square loss but the relative difference between loss and the absolute minimum loss: $\bar{L} = (L - L_{\min})/L_{\min}$. Observe that each curve decreases rapidly before approximately reaching a minimum and then rapidly oscillates near this minimum.

Figure 9.5a supports our previous findings. It shows the relative loss averaged over the last 50 epochs of training. Clearly, larger batch sizes lead to smaller relative loss. Figure 9.5b shows the average computation time per iteration, which is, as expected, linear in terms of the batch size.

To summarize,

- The SGD algorithms with small batch sizes b_{small} very quickly reach a higher relative loss $\bar{L}_{b_{\text{small}}}$ and do not really improve afterward.
- The SGD algorithms with large batch sizes b_{large} reach the relative loss $\bar{L}_{b_{\text{large}}}$ slower; that is, it requires more epochs, but the resulting loss $\bar{L}_{b_{\text{large}}}$ is smaller: $\bar{L}_{b_{\text{large}}} < \bar{L}_{b_{\text{small}}}$.
- For a fixed value of relative loss $\bar{L}_0$, there is an optimal batch size b_0 , which is not so small that SGD does not reach $\bar{L}_0$ but is not so large that SGD takes many epochs to reach $\bar{L}_0$. In our example, $b_0 \approx 32$, as follows from Figure 9.4.

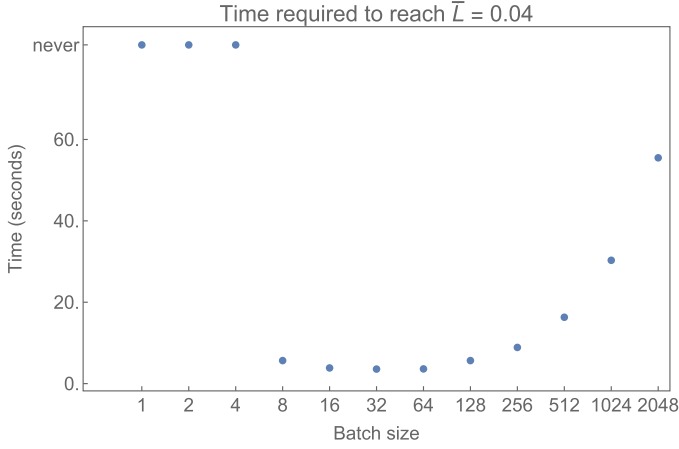
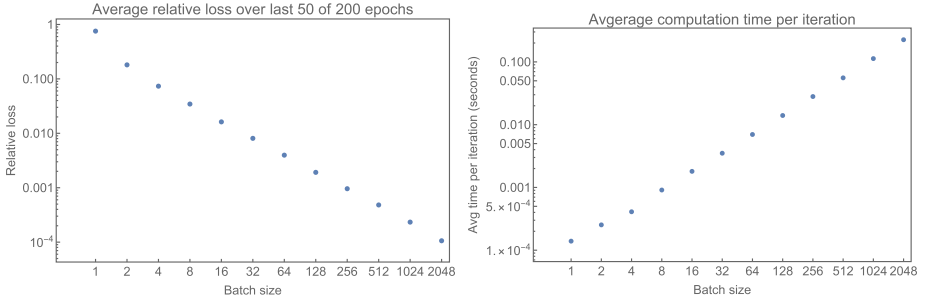


Figure 9.4: The amount of time required to reach the loss.



(a) Average relative loss over last 50 epochs (out of 200).

(b) Average computation time per iteration.

Figure 9.5: Average time and relative loss.

9.7 Exercises

Exercise 1 (Elementary considerations on the learning rate). We consider the function

$$x \in \mathbb{R} \longrightarrow f(x) = \lambda x^2$$

for a fixed constant $\lambda > 0$.

- Calculate the derivative of f .
- For given $\tau > 0$ and x , calculate

$$y = x - \tau f'(x).$$

- Determine τ for which we have

$$f(y) < f(x).$$

Exercise 2.

- a. Find the minimum value of $f(x, y) = x^2/16 + y^2/25$.
- b. Now try to approximate the minimizer of the same function $f(x, y) = x^2/16 + y^2/25$ via the GD. Consider the initial point $(x_0, y_0) = (1, 2)$. Fix the step size $\tau = 10$.
 - Perform one step of the gradient descent and find the next point (x_1, y_1) . What is the value of f at this point?
 - Compute one more step of the gradient descent to find (x_2, y_2) . What is the value of $f(x_2, y_2)$?

Exercise 3 (Finding the best direction). This exercise shows that to minimize some function $f(x)$, the direction $-\nabla f(x)$ may not always be the optimal one for a fixed τ . Consider

$$x \in \mathbb{R}^2 \longrightarrow f(x) = x_1^2 + 2x_2^2.$$

For a fixed τ and $x \in \mathbb{R}^2$, we wish to find the best direction $y \in \mathbb{R}^2$ with $\|y\| = 1$, that minimizes $f(x - \tau y)$.

- a. Explain why we can take $y = (\cos \theta, \sin \theta)$ and study

$$g(\theta) = f(x_1 - \tau \cos \theta, x_2 - \tau \sin \theta).$$

- b. Calculate $g'(\theta)$ and show that the minimum is obtained for

$$y_1 = \frac{x_1 y_2}{2x_2 - \tau y_2}.$$

- c. Calculate $\nabla f(x)$.
- d. Show that we can take y parallel to $\nabla f(x)$ if $\tau = 0$ but that y and $\nabla f(x)$ are not colinear if $\tau > 0$.

Exercise 4 (Oscillations in gradient descent algorithms). We are using a gradient descent algorithm with constant learning rate on the function

$$x \in \mathbb{R} \longrightarrow f(x) = (\sin 10x)^2.$$

Naturally, f has a minimum at $x = 0$.

- a. Explain why if the learning rate $\tau > 1/19$, then we cannot expect in general to have convergence to the minimum at $x = 0$.
- b. Verify this result numerically by testing various learning rates τ by writing your own code.

Exercise 5 (The role of weights in the limit of gradient descent algorithms). We consider the following quadratic loss function, corresponding to a simple case with one parameter and three objects,

$$\alpha \in \mathbb{R} \longrightarrow L(\alpha) = (\alpha - s_1)^2 + (\alpha - s_2)^2 + \nu (\alpha - s_3)^2,$$

where $\nu > 0$ is a weight that can be chosen. The values s_1, s_2, s_3 correspond to the three objects, and we assume that

$$s_1 = 0, \quad s_2 = 0.01, \quad s_3 = 1.$$

- Calculate $L'(\alpha)$.
- For each ν , find the value α_ν that minimizes $L'(\alpha)$.
- Explain how α_ν changes in terms of ν and interpret this in terms of how well classified each object is.

Exercise 6. Let $F(x, y)$ be a convex differentiable function such that

$$\|\nabla F(x_1, y_1) - \nabla F(x_2, y_2)\| \leq 2\|(x_1, y_1) - (x_2, y_2)\| \quad (9.19)$$

for all $(x_1, y_1), (x_2, y_2) \in \mathbb{R}^2$. Suppose that F reaches its minimum at an unknown point $(\bar{x}, \bar{y})$ somewhere in the disk centered at the origin with radius 10. We apply the gradient descent algorithm starting from the point $(x_0, y_0) = (0, 0)$ to approximate the minimum.

- What is the largest value of the step size τ such that the gradient descent algorithm is guaranteed to converge based on a theorem from this chapter? Which theorem is that?
- Let (x_k, y_k) be obtained after the k th iteration of gradient descent starting from $(x_0, y_0) = (0, 0)$. Estimate the minimum value of k such that $F(x_k, y_k) - F(\bar{x}, \bar{y}) < 0.0001$.

Exercise 7 (Convergence of gradient descent for non-convex functions). Consider a non-convex smooth function $f : \mathbb{R}^n \rightarrow \mathbb{R}_+$. We assume that all second-order partial derivatives of f are bounded and recall that this implies, through Taylor expansion, the following inequality: There exists $\kappa > 0$ such that for all α and β in $\mathbb{R}^n$,

$$|f(\beta) - f(\alpha) - (\beta - \alpha) \cdot \nabla f(\alpha)| \leq \kappa \|\alpha - \beta\|^2. \quad (9.20)$$

We implement a gradient descent algorithm with non-constant learning rate. It consists in calculating the sequence α_k by

$$\alpha_{k+1} = \alpha_k - \tau_k \nabla f(\alpha_k).$$

- Use (9.20) to show that if $\tau_k \kappa \leq 1/2$, then $f(\alpha_{k+1}) \leq f(\alpha_k)$ with, in particular,

$$f(\alpha_{k+1}) \leq f(\alpha_k) - \tau_k \|\nabla f(\alpha_k)\|^2.$$

- b. Deduce that if $\tau_k \kappa \leq 1/2$, then

$$\sum_{k=1}^{\infty} \tau_k \|\nabla f(\alpha_k)\|^2 < \infty.$$

- c. Deduce further that if $\sum_{k=1}^{\infty} \tau_k = +\infty$, then $\|\nabla f(\alpha_k)\| \rightarrow 0$ as $k \rightarrow \infty$.
 d. We finally assume that f has only a finite number of critical points: There only exist finitely many points $y_1, \dots, y_N$ such that $\nabla f(y_i) = 0$. Conclude that the sequence α_k must converge to one of the y_i .

Exercise 8 (Some advantages of stochastic gradient descent). Consider a finite number of differentiable functions $\ell_i(\alpha) : \mathbb{R}^n \rightarrow \mathbb{R}$, each corresponding to a given object s_i , $i = 1 \dots N$, in the training set T . We attempt to minimize the full loss function

$$L(\alpha) = \frac{1}{N} \sum_{i=1}^N \ell_i(\alpha)$$

through a stochastic gradient descent algorithm with constant learning rate and batch size S .

- a. Recall the definition of stochastic gradient descent.
 b. We choose a batch size of 1 and assume that the random sequence α_k obtained from SGD converges almost surely to some $\bar{\alpha}$. Prove that

$$\nabla \ell_i(\bar{\alpha}) = 0 \quad \text{for all } i.$$

Explain why this is better than if we had convergence for deterministic gradient descent.

- c. Show that the same is true for any batch size.

10 Backpropagation

The training of DNNs requires to efficiently compute $\nabla L(\alpha)$ when the vector of parameters $\alpha = (\alpha_k) \in \mathbb{R}^\mu$ has large dimension $\mu \gg 1$; currently, for the largest DNNs, μ can be more than half a trillion! Calculating individually all partial derivatives for each parameter μ is time consuming, even for high-performance computers. Therefore techniques for expediting the calculation of derivatives are valuable tools for training DNNs.

One of the most effective of these techniques is *backpropagation*, which takes advantage of the fact that $L(\alpha)$ is not an arbitrary function of many parameters. On the contrary, the dependence of $L(\alpha)$ on the parameters α is determined by the compositional structure of a DNN; see Definition 4.1.3 and Figure 4.4. Then computing $\nabla L(\alpha)$ using the compositional structure of (4.8) leads to applying the chain rule to each layer iteratively. The key observation here is that in this iterative process from layer to layer of the DNN, most of the derivatives appear more than once. The computational savings come from eliminating repetitive computations of such derivatives.

10.1 Computational complexity

To understand how one algorithm is more efficient than another, we need to define how to “measure” efficiency. To this end, we briefly review the basic concepts of computational complexity. Roughly speaking, the notion of complexity corresponds to the total number of operations (e. g., addition and multiplication) required to execute a computational algorithm. The following example in an everyday context gives an idea of computational complexity.

Example 10.1.1 (Comparing lists of names). Suppose you are given two lists of names A and B that you have to compare: for example, the list of the names of students registered to Calculus 1 with the list of the names of students registered to Linear Algebra. The goal is to mark all the names that are on both lists A and B. For simplicity, the computational complexity in this example will be the total number of times that we read a name, e. g., reading one name twice counts as two operations, and we assume that list A and B both have n names.

We are going to consider three cases of how the lists can be ordered.

- (i) The names in both lists were entered in alphabetical order.
- (ii) The names in both lists were entered in random order, for example, by writing the name of each student in the order when they registered for the course.
- (iii) The names in one list, for example, A, are in alphabetical order, whereas the names in list B are in random order.

Intuitively, randomness in the list brings more complexity than an ordered list. Therefore the first case should be the easiest, and the second the most time consuming. We can quantify this by evaluating the computational complexity in each of the cases.

Let us start with example (i). Take, for instance, $A = [aa, bc, fc, ii, zy, zz]$ and $B = [aa, cf, fc, iz, zy, zz]$. We start by comparing the two first elements. Since aa match, we add it to list C . We choose the next element, bc , in A and compare it with the next element in B . We go until the element in list B is at least greater than the element in A ; this guarantees that there is no match. We further continue with the next element in A . We go sequentially through each list marking the names in list A that are not in list B . We need to read each name in both lists only once. Thus, in the worst case, we read both lists once for a total of $n + n = 2n$ names read.

In the second case, we can go through the names in list B in the order in which they are entered, but for each name, we need to go through list A and check whether the corresponding name is in A . This may require going one-by-one through list A for each name in list B . To be more mathematically precise, we can number the names on list B from 1 to n ; for each i between 1 and n , denote by p_i the number of names we must read in list A before finding name i (or we read all the names in list A and deduce that name i is not in this list, in which case $p_i = n$). The total cost is then

$$\sum_{i=1}^n (1 + p_i), \quad (10.1)$$

where $(1 + p_i)$ corresponds to first reading a name in list B (which has cost 1) and then reading p_i names in list A (which has the cost p_i).

The exact total depends on the exact values of p_i . However, by considering the best and worst case random orderings we can easily provide lower and upper bounds. In the best case scenario, our two lists contain the same elements with perhaps different ordering, and hence each p_i has to be between 1 and n and must be different, giving us a lower bound (see (10.2)). In the worst case scenario, $p_i = n$ for all i because lists A and B share no elements. Hence we obtain that

$$\frac{n(n+3)}{2} = \sum_{j=1}^n (1+j) \leq \sum_{i=1}^n (1+p_i) \leq \sum_{j=1}^n (1+n) = n(n+1). \quad (10.2)$$

The third case is somewhat more complex, and the computational complexity here ends up being intermediate between the fastest case 1 and the slowest case 2.

We now explain an algorithm called the *binary search algorithm*, which uses the alphabetical structure of list A to efficiently mark all names on list A that are also on list B . The algorithm works as follows: for any given name $\#i$ in list B , we find whether this name is in list A or not. To this end, we begin by looking at the name in the middle of list A . If this name matches our desired name $\#i$, then our algorithm terminates. Otherwise, since A is in alphabetical order, we will know if name $\#i$ is in the first half of A or in the second half, in the same way that we figure out whether we have passed a word in the dictionary. We then repeat this process on the half of the list that may contain name $\#i$ until we find it in list A (or determine that the name $\#i$ is not in list A).

In this case, for each student in list B, we perform a binary search algorithm to see if their name appears in list A. Each binary search algorithm requires at most $\log_2(n)$ operations. To understand why, we consider the maximum number of times that a list of n elements can be split in half, and then one of the resulting halves is split into half and so on. In the first step, we cut the second list by half, resulting in $\frac{n}{2}$ names left in an updated list. Then, in the second step, we once again cut the new updated list in half, resulting in $\frac{n}{4}$ names in the updated list. We continue doing this until we have one name left, and so we need to find x such that $2^x = n$. Thus $x = \log_2(n)$, and so we divide list A into equal halves a maximum of $\log_2(n)$ many times. Finally, because there are n names in list B, each of which requires this binary search algorithm, the total maximum number of operation we perform is $n + n \log_2(n)$.

As we can see, the cost in this case, $n + n \log_2(n)$, is much lower than in the second case but still higher than in the first case. Of course, it is possible to be more efficient in the third case, for example, by marking where is the first name starting with letter A, B, C, . . .

When it comes to computational complexity, we are typically concerned about how the algorithms behave for a large number of operations. However, it is often not clear how to compare exactly individual operations: For example, the time to perform a multiplication vs. an addition may depend on the processor. It may also be tedious or not simple to calculate the exact number of operations, as in the example above. For this reason, we often wish to focus on the order of magnitude without considering precise numerical constants. The following definition provides rigorous mathematical meaning to this idea.

Definition 10.1.1 (Landau symbol $O(\cdot)$). Consider two functions $f, g : \mathbb{N} \rightarrow \mathbb{R}$. If there exist a constant $C > 0$ and an integer N such that

$$|f(n)| \leq C|g(n)| \quad (10.3)$$

for all $n > N$, then f is said to be of at most order g as $n \rightarrow \infty$. We denote this by

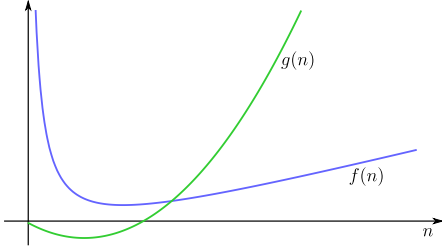
$$f(n) = O(g(n)) \quad \text{as } n \rightarrow \infty. \quad (10.4)$$

In Figure 10.1, there is a graphical example of $f(n) = O(g(n))$, where inequality (10.3) is satisfied for large n . Notice that $g(n)$ may in fact be much larger than $f(n)$ as $n \rightarrow \infty$.

The following proposition shows how $O(\cdot)$ can be determined using a limit.

Proposition 10.1.1. Let $f, g : \mathbb{N} \rightarrow \mathbb{R}$. If $\lim_{n \rightarrow \infty} f(n)/g(n)$ exists and is finite, then $f(n) = O(g(n))$ as $n \rightarrow \infty$. Moreover, if the limit is nonzero, then $g(n) = O(f(n))$ as $n \rightarrow \infty$.

Proof. Suppose $\lim_{n \rightarrow \infty} f(n)/g(n) = \ell < \infty$. Then there exists N such that if $n > N$, then $|f(n)/g(n) - \ell| < 1$, which leads to the following sequence of inequalities:

Figure 10.1: Example of $f(n) = O(g(n))$.

$$\left| \frac{f(n)}{g(n)} - \ell \right| < 1 \quad (10.5)$$

$$\Rightarrow \left| \frac{f(n)}{g(n)} \right| - |\ell| < 1 \quad (\text{triangle inequality}) \quad (10.6)$$

$$\Rightarrow \frac{|f(n)|}{|g(n)|} < 1 + |\ell| \quad (10.7)$$

$$\Rightarrow |f(n)| < (1 + |\ell|)|g(n)| \quad (10.8)$$

for all $n > N$. Letting $C = (1 + |\ell|)$, we observe that f, g satisfy (10.3) for all $n > N$. Therefore $f(n) = O(g(n))$ as $n \rightarrow \infty$.

If $\ell \neq 0$, then we observe that $\lim_{n \rightarrow \infty} g(n)/f(n) = 1/\ell$, so we may repeat the above argument to see that $g(n) = O(f(n))$ as $n \rightarrow \infty$. $\square$

It is sometimes easier to calculate a limit than to show that inequality (10.3) holds directly, which is when Proposition 10.1.1 is useful.

Example 10.1.2. Let $f(n) = n$ and $g(n) = n^2$. The function $f(n)$ is of order $g(n)$ because $n \leq n^2$ (set $C = 1$ for $n \geq 1$).

However, $g(n)$ is not of order $f(n)$ because

$$\lim_{n \rightarrow \infty} \frac{n^2}{n} = \infty. \quad (10.9)$$

There are some common orders that are worth mentioning. For instance,

$$O(1) \leq O(\log n) \leq O(n^s) \leq O(\exp n), \quad s \in \mathbb{Z}, n \rightarrow \infty. \quad (10.10)$$

We often refer to $O(1)$ functions as bounded in n as $n \rightarrow \infty$, to $O(\log n)$ as logarithmic growth, $O(n^s)$ as polynomial growth, and $O(\exp n)$ as exponential growth.

If $f(n)$ gives the number of operations that are sufficient to complete the computation and $g(n)$ is such that $f(n) = O(g(n))$, then we can also say that the order of complexity of the computation is $O(g(n))$. Generally, we try to choose g as simple as possible, ignoring constant factors and smaller-order terms. Whenever possible, we also try to ensure that $g(n) = O(f(n))$.

For example, if $f(n) = \pi n^2 + 3n$, then we would typically choose $g(n) = n^2$ and explain that the algorithm is of order n^2 . Technically speaking, it would also be correct to write that $f(n) = O(n^3)$, but this is obviously much less useful.

Clearly, algorithms whose computational complexity grows linearly, namely $O(n)$, take much less time to execute than algorithms whose complexity grows quadratically, that is, $O(n^2)$.

Hereafter, counting the number of operations will play a key role in determining the order of computational complexity. As mentioned above, depending on the computing platform, some elementary operations may take proportionally longer than others. To keep the discussion simple and independent of the platform, we however count all those operations identically. This leads to the following definition.

Definition 10.1.2. In a algorithm, an *operation* is any one of the following:

- Addition.
- Subtraction.
- Multiplication.
- Division.
- Exponentiation.
- Comparison of the values of two numbers (e. g., checking whether $x < y$ for $x, y \in \mathbb{R}$).

Now we apply Definitions 10.1.1 and 10.1.2 to two simple computations.

Example 10.1.3 (Computational complexity of the dot product). Let $a = (a_1, \dots, a_n)$ and $b = (b_1, \dots, b_n) \in \mathbb{R}^n$ be two vectors. The dot product of a and b is given by

$$a \cdot b = a_1 b_1 + a_2 b_2 + \dots + a_n b_n. \quad (10.11)$$

First, we need to perform n multiplications: $a_1 b_1, a_2 b_2, \dots, a_n b_n$. Then we need to perform $n-1$ additions: $a_1 b_1 + a_2 b_2, (a_1 b_1 + a_2 b_2) + a_3 b_3, \dots$. The total number of operations according to our definitions is $n + (n-1) = 2n-1$. Since

$$\lim_{n \rightarrow \infty} \frac{2n-1}{n} = 2, \quad (10.12)$$

the computational complexity of the dot product in $\mathbb{R}^n$ is $O(n)$.

Recall the algorithm to multiply two matrices. Since it involves additions and multiplications, we can find its complexity in a similar way.

Example 10.1.4 (Computational complexity of matrix multiplication). Let $A = (a_{ij})$, $B = (b_{ij}) \in \mathbb{R}^{n \times n}$ be two $n \times n$ matrices. Their product is given, component-wise, by

$$(AB)_{ij} = \sum_{k=1}^n a_{ik} b_{kj}, \quad i, j \in \{1, \dots, n\}. \quad (10.13)$$

First, fix a component of the resulting matrix. To compute this component, we perform $2n - 1$ operations (n multiplications and $n - 1$ additions), since it is the dot product of a row of A and a column of B . This occurs with each of the n^2 components. Therefore, we require $n^2(2n - 1)$ operations. Since

$$\lim_{n \rightarrow \infty} \frac{n^2(2n - 1)}{n^3} = 2, \quad (10.14)$$

the complexity of multiplication of $n \times n$ matrices is $O(n^3)$.

It is actually possible to do better in terms of computational complexity, since the pioneering work [66]. The computational complexity of matrix multiplication has been recently reduced to approximately $O(n^{2.3728596})$ in [1]. See also the recent work [21]. Strassen algorithms from [66] are mostly only used in practice for very large matrices. Moreover, on modern computers, memory access or parallelizing often proves to be more critical for the final computing time.

It will be useful to note that multiplying a $k \times m$ rectangular matrix by an $m \times n$ rectangular matrix requires

$$O(kmn) \text{ operations.} \quad (10.15)$$

Observe that if $k = m = n$, then the matrices are square, and this agrees with the $O(n^3)$ complexity calculated above.

Finally, note that although in the above examples the operations were multiplications and additions, in backpropagation, other operations are involved, such as determining whether a number is positive or negative (e.g., for ReLU).

10.2 Chain rule review

There is a standard definition of chain rule that can be found in calculus textbooks (see Appendix). Here we present two simple examples for the single- and multivariable cases.

Example 10.2.1. Let $f : \mathbb{R} \rightarrow \mathbb{R}$ and $g : \mathbb{R} \rightarrow \mathbb{R}$ be defined by $f(x) = \cos(x)$, $g(x) = e^x + 2x$. Find

$$\left. \frac{d}{dx} f \circ g(x) \right|_{x=0}. \quad (10.16)$$

The chain rule for one-variable functions gives

$$\left. \frac{d}{dx} f \circ g(x) \right|_{x=0} = \left. \frac{d}{dy} f(y) \right|_{y=g(0)} \cdot \left. \frac{d}{dx} g(x) \right|_{x=0}. \quad (10.17)$$

Since $f'(y) = -\sin(y)$, $g'(x) = e^x + 2$, and $g(0) = 1$, the desired derivative is

$$\left. \frac{d}{dx} f \circ g(x) \right|_{x=0} = -\sin(y)|_{y=1} \cdot (e^x + 2)|_{x=0} = -\sin(1) \cdot (1 + 2) = -3 \sin(1). \quad (10.18)$$

Example 10.2.2. Let $f : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ and $g : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ be functions defined by

$$f(u, v) = (u^2 + 3uv, \exp(u + v)), \quad (10.19)$$

$$g(x, y) = (xy, x^2 + y^2). \quad (10.20)$$

Find all partial derivatives at $(x, y) = (1, 0)$,

$$\left. \frac{\partial(f \circ g)}{\partial(x, y)} \right|_{(x, y)=(1, 0)}. \quad (10.21)$$

As a result of the chain rule, this is equal to the following matrix product:

$$\left(\begin{array}{cc} \frac{\partial f_1}{\partial u}(u, v) & \frac{\partial f_1}{\partial v}(u, v) \\ \frac{\partial f_2}{\partial u}(u, v) & \frac{\partial f_2}{\partial v}(u, v) \end{array} \right) \bigg|_{(u, v)=g(1, 0)} \cdot \left(\begin{array}{cc} \frac{\partial g_1}{\partial x}(x, y) & \frac{\partial g_1}{\partial y}(x, y) \\ \frac{\partial g_2}{\partial x}(x, y) & \frac{\partial g_2}{\partial y}(x, y) \end{array} \right) \bigg|_{(x, y)=(1, 0)}, \quad (10.22)$$

where $f_1(u, v) = u^2 + 3uv$, $f_2(u, v) = \exp(u + v)$, $g_1(x, y) = xy$, $g_2(x, y) = x^2 + y^2$, and $g(1, 0) = (0, 1)$. The matrices in (10.22) are called the *Jacobians* or *Jacobian matrices* for the functions $f(u, v)$ and $g(x, y)$, respectively.

The first matrix in (10.22) is

$$\left(\begin{array}{cc} 2u + 3v & 3u \\ \exp(u + v) & \exp(u + v) \end{array} \right) \bigg|_{(u, v)=(0, 1)} = \begin{pmatrix} 3 & 0 \\ e & e \end{pmatrix}, \quad (10.23)$$

and the second matrix in (10.22) is

$$\left(\begin{array}{cc} y & x \\ 2x & 2y \end{array} \right) \bigg|_{(x, y)=(1, 0)} = \begin{pmatrix} 0 & 1 \\ 2 & 0 \end{pmatrix}. \quad (10.24)$$

Therefore the desired derivative is given by the product of matrices

$$\begin{pmatrix} 3 & 0 \\ e & e \end{pmatrix} \cdot \begin{pmatrix} 0 & 1 \\ 2 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 3 \\ 2e & e \end{pmatrix}. \quad (10.25)$$

Thus the application of the chain rule leads to matrix multiplication. Since the chain rule is a building block of the backpropagation algorithm, the computational complexity of such multiplication for large n plays an important role in the evaluation of the complexity of backpropagation.

10.3 Diagrammatic representation of the chain rule in simple examples

In this subsection, we introduce a diagrammatic interpretation of the chain rule, which is helpful for understanding backpropagation methods for DNN training. This interpretation is also natural when representing DNNs via diagrams, e. g., Figure 4.4.

In the following example, we introduce a toy *backpropagation algorithm* in the simple context of calculating the gradient of a sigmoid-type function.

In this example, the single-variable chain rule is not sufficient, and the multivariable chain rule is used because several variables are present. However, it is used in a simple form, which does not require matrix multiplication of Jacobians. In the next example, the multivariable chain rule involving matrix multiplications is necessary.

Despite the simplicity of Example 10.3.1, it *introduces some of the key ideas of the backpropagation algorithm*.

Example 10.3.1 (Backpropagation algorithm for sigmoid-type function). We want to compute the gradient of the function $\sigma : \mathbb{R}^4 \rightarrow \mathbb{R}$ given by

$$\sigma(x, y, z, w) = \frac{1}{w + \exp(z + \cos(yx^2))} \quad (10.26)$$

at $(x, y, z, w) = (1/2, \pi, 1, 1)$. As we will see, there is an obvious way to avoid repeating calculations which pre-figures the idea behind backpropagation. This example is a building block for the explanation of backpropagation in the case of the simplest DNN with one neuron per layer; see Proposition 10.4.1.

We begin by representing $\sigma(x, y, z, w)$ as a composition of simpler functions. To this end, we define the following functions, which cannot be written as a composition of simpler functions. We call such functions *elementary functions*.

$$p(x) = x^2, \quad (10.27)$$

$$q(y, p) = y \cdot p, \quad (10.28)$$

$$r(q) = \cos(q), \quad (10.29)$$

$$s(z, r) = z + r, \quad (10.30)$$

$$t(s) = e^s, \quad (10.31)$$

$$u(w, t) = w + t, \quad (10.32)$$

$$v(u) = 1/u. \quad (10.33)$$

Using p, q, r, s, t, u , and v , we may write σ as a composition

$$\sigma(x, y, z, w) = v(u(w, t(s(z, r(q(y, p(x))))))). \quad (10.34)$$

The purpose of writing σ as a composition of elementary functions is that it is very easy to compute the derivative(s) of each elementary function, and the chain rule provides a way to write the derivative of a composition of elementary functions as a product of the derivatives of the elementary functions. This composition is represented in the diagram in Figure 10.2.

We now compute $\nabla\sigma(1/2, \pi, 1, 1)$ using the diagram in Figure 10.2 in several steps. We will compute the gradient using two methods. The first method is straightforward,

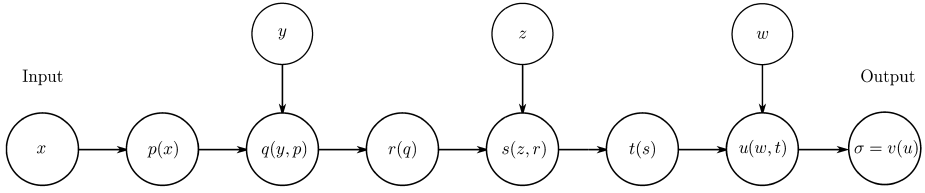


Figure 10.2: Diagrammatic representation of the composition of functions in (10.26).

but the second reduces the order of complexity. We will show where this reduction in order of complexity comes from.

Step 1. Write the function in each node of each layer of the diagram in terms of the functions in the previous layer. These functions are also called the label of the corresponding node. For example, the first node in the second layer has label $q(y, p) = y \cdot p(x)$, where y and $p(x)$ are labels in the first layer. In this diagram, the elementary functions correspond to multiplication, addition, exponentiation, etc.

Step 2. Forward propagation. Given values of x, y, z, w , calculate the corresponding values for the functions p, q, r, s, t, u, σ . For the point $(x, y, z, w) = (1/2, \pi, 1, 1)$, these values are

$$p = 1/4, \quad q = \pi/4, \quad r = \sqrt{2}/2, \quad s = 1 + \sqrt{2}/2, \quad t = e^{1+\sqrt{2}/2}, \quad (10.35)$$

$$u = e^{1+\sqrt{2}/2} + 1, \quad \sigma = \frac{1}{e^{1+\sqrt{2}/2} + 1}. \quad (10.36)$$

In the following Steps 3–8, we introduce the idea of *backpropagation algorithm* in the context of this example.

Step 3. Compute the derivative function of the functions in each layer with respect to the variable in the previous layer. For example, $\partial t / \partial s = e^s$.

Step 4. Evaluate the derivatives in (10.37)–(10.40) at the point $(1/2, \pi, 1, 1)$. We now show two ways of evaluating these derivatives.

Step 5. Naive calculations of derivatives and their products. We may first naively apply the chain rule from calculus, separately each time for each partial derivative. To compute $\nabla \sigma$, we need to compute all nineteen derivatives on the right-hand sides of (10.37)–(10.40). Furthermore, to apply the chain rule straightforwardly, we need additional 16 multiplications of the derivatives; e.g., in (10.37), we need six operations to compute the product of seven derivatives, leading to 16 total multiplications in (10.37)–(10.40):

$$\frac{d\sigma}{dx} = \left(\frac{d\sigma}{du} \frac{du}{dt} \frac{dt}{ds} \frac{ds}{dr} \frac{dr}{dq} \right) \frac{dq}{dp} \frac{dp}{dx}, \quad (10.37)$$

$$\frac{d\sigma}{dy} = \left(\frac{d\sigma}{du} \frac{du}{dt} \frac{dt}{ds} \frac{ds}{dr} \frac{dr}{dq} \right) \frac{dq}{dy}, \quad (10.38)$$

$$\frac{d\sigma}{dz} = \left(\frac{d\sigma}{du} \frac{du}{dt} \frac{dt}{ds} \right) \frac{ds}{dz}, \quad (10.39)$$

$$\frac{d\sigma}{dw} = \left(\frac{d\sigma}{du} \right) \frac{du}{dw}. \quad (10.40)$$

Thus computing $\nabla\sigma(1/2, \pi, 1, 1)$ using straightforward chain rule leads to computing 19 derivatives and 15 multiplications.

For the next two steps, recall that complexity comes from two sources: calculation of derivatives and multiplications of derivatives.

Step 6. Avoiding repetitive calculation of derivatives. Observe that (10.37)–(10.40) have many repeated terms shown in parentheses. Indeed, in (10.37)–(10.40), there are only ten distinct derivatives. Therefore the number of derivatives we need to compute can be reduced from 19 in Steps 5 to 10, which is the total number of edges in Figure 10.2. The diagram in Figure 10.2 shows that the savings come from the fact that we traverse each edge only once from any path from right to left.

Step 7. Clever backward calculation of the product of derivatives. To complete the application of the chain rule, we need to find the products of the distinct derivatives obtained in Step 6. Observe that again many products are repeated in (10.37)–(10.40), and we wish to perform as few multiplications of derivatives as possible. To this end, we view σ as a function of all variables $u, t, s, r, p, q, w, z, y, x$ in the diagram in Figure 10.2. We then calculate the derivatives of σ with respect to each of these variables starting with u and working *backward*, right to left in the diagram in Figure 10.2. We reuse any product of derivatives already calculated to avoid repetitions (cf. Step 6, where we reuse the derivatives already calculated). For example, the product in parentheses in (10.43) was already calculated in (10.42):

$$\frac{\partial\sigma}{\partial u} = -\frac{1}{u^2}, \quad (10.41)$$

$$\frac{\partial\sigma}{\partial t} = \frac{\partial\sigma}{\partial u} \frac{\partial u}{\partial t}, \quad (10.42)$$

$$\frac{\partial\sigma}{\partial s} = \left(\frac{\partial\sigma}{\partial u} \frac{\partial u}{\partial t} \right) \frac{\partial t}{\partial s}. \quad (10.43)$$

This reduces the necessary number of multiplications of derivatives from 15 in Steps 5 to 9, which is the total number of edges minus one on the diagram in Figure 10.2. The number of edges is equal to the number of derivatives, and the product of k derivatives requires $k - 1$ multiplications.

Step 8. We now use the method described in Step 7 to calculate $\nabla\sigma(1/2, \pi, 1, 1)$.

$$\frac{\partial\sigma}{\partial u} = -\frac{1}{u^2} = -\frac{1}{(1 + e^{1+\sqrt{2}/2})^2}, \quad (10.44)$$

$$\frac{\partial\sigma}{\partial w} = \frac{\partial\sigma}{\partial u} \frac{\partial u}{\partial w} = -\frac{1}{(1 + e^{1+\sqrt{2}/2})^2} \cdot 1, \quad (10.45)$$

$$\frac{\partial \sigma}{\partial t} = \frac{\partial \sigma}{\partial u} \frac{\partial u}{\partial t} = -\frac{1}{(1 + e^{1+\sqrt{2}/2})^2} \cdot 1, \quad (10.46)$$

$$\frac{\partial \sigma}{\partial s} = \frac{\partial \sigma}{\partial t} \frac{\partial t}{\partial s} = -\frac{1}{(1 + e^{1+\sqrt{2}/2})^2} \cdot e^s = -\frac{e^{1+\sqrt{2}/2}}{(1 + e^{1+\sqrt{2}/2})^2}, \quad (10.47)$$

$$\frac{\partial \sigma}{\partial z} = \frac{\partial \sigma}{\partial s} \frac{\partial s}{\partial z} = -\frac{e^{1+\sqrt{2}/2}}{(1 + e^{1+\sqrt{2}/2})^2} \cdot 1, \quad (10.48)$$

$$\frac{\partial \sigma}{\partial r} = \frac{\partial \sigma}{\partial s} \frac{\partial s}{\partial r} = -\frac{e^{1+\sqrt{2}/2}}{(1 + e^{1+\sqrt{2}/2})^2} \cdot 1, \quad (10.49)$$

$$\frac{\partial \sigma}{\partial q} = \frac{\partial \sigma}{\partial r} \frac{\partial r}{\partial q} = -\frac{e^{1+\sqrt{2}/2}}{(1 + e^{1+\sqrt{2}/2})^2} \cdot (-\sin(q)) = \frac{\sqrt{2}e^{1+\sqrt{2}/2}}{2(1 + e^{1+\sqrt{2}/2})^2}, \quad (10.50)$$

$$\frac{\partial \sigma}{\partial y} = \frac{\partial \sigma}{\partial q} \frac{\partial q}{\partial y} = \frac{\sqrt{2}e^{1+\sqrt{2}/2}}{(1 + e^{1+\sqrt{2}/2})^2} \cdot p = \frac{\sqrt{2}e^{1+\sqrt{2}/2}}{8(1 + e^{1+\sqrt{2}/2})^2}, \quad (10.51)$$

$$\frac{\partial \sigma}{\partial p} = \frac{\partial \sigma}{\partial q} \frac{\partial q}{\partial p} = \frac{\sqrt{2}e^{1+\sqrt{2}/2}}{8(1 + e^{1+\sqrt{2}/2})^2} \cdot y, \quad (10.52)$$

$$\frac{\partial \sigma}{\partial x} = \frac{\partial \sigma}{\partial p} \frac{\partial p}{\partial x} = \frac{\pi \sqrt{2}e^{1+\sqrt{2}/2}}{8(1 + e^{1+\sqrt{2}/2})^2} \cdot 2x = \frac{\pi \sqrt{2}e^{1+\sqrt{2}/2}}{4(1 + e^{1+\sqrt{2}/2})^2}. \quad (10.53)$$

In (10.44)–(10.53), we explicitly see 10 distinct derivatives, which must be calculated, and 9 distinct multiplications, which must be made.

In summary, the backward direction of propagation is given by the fact that we need to compute derivatives of σ (rightmost) with respect to variables x, y, z, w (leftmost). There are two sources of savings in this elementary example of backpropagation.

- First, reusing derivatives already calculated; see Step 6.
- Second, reusing any product of derivatives already calculated; see Step 7.

We note that in this example, we reduced the total number of operations from $2n^2+3 = 35$ to $4n+3 = 19$, where $n = 4$ corresponds to the number of parameters (x, y, z, w) . This shows a reduction of computational complexity by an order of magnitude from $O(n^2)$ to $O(n)$.

Following Steps 1–4 and 6–7, we obtain the following gradient for $\sigma(1/2, \pi, 1, 1)$:

$$\sigma_x(1/2, \pi, 1, 1) = \frac{\pi \sqrt{2}e^{1+\sqrt{2}/2}}{4(1 + e^{1+\sqrt{2}/2})^2}, \quad (10.54)$$

$$\sigma_y(1/2, \pi, 1, 1) = \frac{\sqrt{2}e^{1+\sqrt{2}/2}}{8(1 + e^{1+\sqrt{2}/2})^2}, \quad (10.55)$$

$$\sigma_z(1/2, \pi, 1, 1) = -\frac{e^{1+\sqrt{2}/2}}{(1 + e^{1+\sqrt{2}/2})^2}, \quad (10.56)$$

$$\sigma_w(1/2, \pi, 1, 1) = -\frac{1}{(1 + e^{1+\sqrt{2}/2})^2}. \quad (10.57)$$

Example 10.3.2. Note that the diagram in Example 10.3.1 is rather simple because every node provides output to only one other node (left to right). However, the same steps apply to functions whose diagrams contain nodes with multiple outputs. For example, consider the function $f : \mathbb{R}^3 \rightarrow \mathbb{R}$ defined by $f(x, y, z) = \sin(xy + xz + yz)$, which has the computation diagram in Figure 10.3.

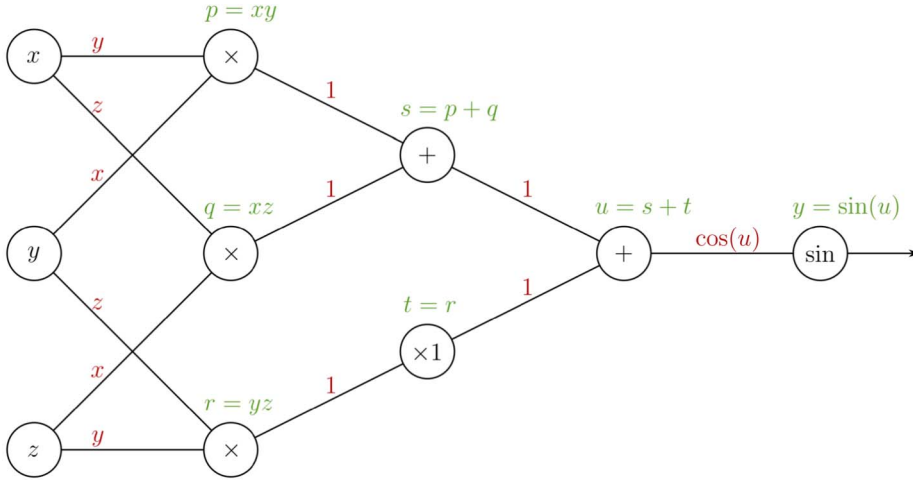


Figure 10.3: Diagram for Example 10.3.2.

10.4 The case of a simple DNN with one neuron per layer

In the previous section, we illustrated how backpropagation reduces the number of operations when applying the chain rule to functions of many variables. We used a diagrammatic representation of the chain rule because of the analogy with DNN diagrams. In this section, we extend these backpropagation savings to DNNs and quantify its computational complexity. For simplicity of presentation, we first consider a trivial network with one neuron per layer. The same approach works for general networks (with many neurons per layer), but the calculations are more technical, e. g., by computing gradient with respect to matrices.

If μ is the dimension of the parameter space, then we show that the calculation of the gradient of the loss function $L(\mathbf{a})$ takes $O(\mu)$ operations (see Definition 10.1.2). Note that if one naively approximates the gradient using finite differences, then the calculation of $L(\mathbf{a})$ takes $O(\mu^2)$ operations. Indeed, the gradient in $\mathbf{a}$ has μ components. Furthermore, each component requires at least one computation of the ANN function, and evaluating the ANN function takes $O(\mu)$ computations, because, e. g., each of μ layers requires $O(1)$ computations; see Figure 10.4. We now contrast this naive approach with the backpropagation algorithm.

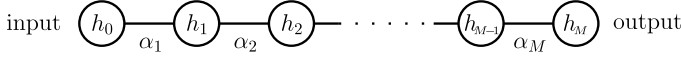


Figure 10.4: A DNN with many layers and one neuron per layer, h_k is the k th layer function, defined recursively in (10.64).

Proposition 10.4.1. *Let $\phi : \mathbb{R} \times \mathbb{R}^\mu \rightarrow \mathbb{R}$ be a neural network as in Definition 4.1.3 with M layers and one neuron in each layer and no biases (i. e., ϕ depends on $\mu = M$ parameters). Let $T \subset \mathbb{R}$ be a training set for ϕ , and let $L : \mathbb{R}^\mu \rightarrow \mathbb{R}$ be a loss function for this DNN of the form (cf. (9.8))*

$$L(\mathbf{a}) = \frac{1}{|T|} \sum_{s \in T} \ell(s, \phi(s, \mathbf{a})). \quad (10.58)$$

Then for given $\mathbf{a} \in \mathbb{R}^\mu$, there exists an algorithm, called backpropagation, that calculates $\nabla L(\mathbf{a})$ in $O(|T|\mu)$ operations.

Remark 10.4.1. In this proposition the number of layers M is also the number μ of parameters because there is only one neuron without bias per layer and therefore only one parameter per layer.

Remark 10.4.2. As we saw before, an example of a specific form of the loss function (10.58) is the mean square approximation of an unknown function $f(x)$ (e. g., exact classifier function) such that

$$\ell(s, \phi(s)) = (\phi(s) - f(s))^2. \quad (10.59)$$

We assume that $f(s)$ is already known for each s , e. g., the result of some measurement, and therefore requires no operations to calculate.

Remark 10.4.3. Although Proposition 10.4.1 concerns the gradient of the loss calculated over the entire training set T , the same result holds if we replace T with a batch $T_n \subset T$ used in, e. g., SGD.

Proof of Proposition 10.4.1. We begin from rewriting formula (10.58) in terms of the layer functions h_k , $k = 1, \dots, \mu$, to obtain the following formula for $\nabla L(\mathbf{a})$:

$$\frac{\partial L}{\partial a_k} = \frac{1}{|T|} \sum_{s \in T} \frac{\partial \ell}{\partial a_k} = \frac{1}{|T|} \sum_{s \in T} \frac{\partial \ell}{\partial h_k} \frac{\partial h_k}{\partial a_k}, \quad k = 1, \dots, \mu. \quad (10.60)$$

Next, we note that the number of elements $|T|$ in the training set does not depend on μ , so that we may perform all calculations independently for each separate object s . Therefore it suffices to consider a training set with a single element s . Then

$$\frac{\partial L}{\partial a_k} = \frac{\partial \ell}{\partial h_k} \frac{\partial h_k}{\partial a_k}, \quad k = 1, \dots, \mu. \quad (10.61)$$

The proof can be done in the following four steps:

- (i) Showing that calculating h_k for $k = 1, \dots, \mu$ requires a total of $O(\mu)$ operations due to the recursive definition of h_k .
- (ii) Showing that calculating $\partial\ell/\partial h_k$ requires a total of $O(\mu)$ operations because of Step (i). Otherwise, there would be $O(\mu^2)$ operations. This is the *backpropagation* step when we calculate $\partial\ell/\partial h_k$ when we start with $k = \mu$ and work recursively backward to $k = 1$. The key difference between (i) and (ii) is that h_k is defined by the forward recursive formula (10.64), whereas $\partial\ell/\partial h_k$ is defined via the backward recursive formula (10.68).
- (iii) Showing that calculating $\partial h_k/\partial \alpha_k$ for $k = 1, \dots, \mu$ requires a total of $O(\mu)$ operations instead of $O(\mu^2)$ operations, again because of Step (i).
- (iv) Showing that calculating $\partial L/\partial \alpha_k$ for $k = 1, \dots, \mu$ using (10.61) requires μ multiplications of the numbers already calculated in steps (ii) and (iii).

Finally, since each step requires $O(\mu)$ operations, the total number of operations to calculate $\nabla L(\mathbf{a})$ for a training set with a single element s is indeed $O(\mu)$ for each object.

Step 1. In this step, we calculate the values of all layer functions $h_1, \dots, h_\mu$ in the one-neuron-per-layer network and prove that

$$\text{the calculation of the values of } h_k \text{ for } k = 1, \dots, \mu \text{ requires } O(\mu) \text{ operations in total.} \quad (10.62)$$

Hereafter, we refer to h_k as the k th output for brevity.

First, we derive a recursive formula for each h_k . The output of the last neuron h_μ is equal to $\phi(s, \mathbf{a})$ for the object s :

$$\phi(s, \mathbf{a}) = \lambda(\alpha_\mu \lambda(\alpha_{\mu-1} \cdots \alpha_2 \lambda(\alpha_1 s) \cdots)), \quad (10.63)$$

where λ is an activation function, and α_k is the scalar parameter of the k th layer. Since each layer has one neuron, we write the k th layer function $h_k : \mathbb{R} \rightarrow \mathbb{R}$ in the recursive form

$$h_k = \lambda(\alpha_k h_{k-1}), \quad k = 1, \dots, \mu \quad (10.64)$$

with

$$h_0 = s \quad \text{and} \quad h_\mu = \phi(s, \mathbf{a}). \quad (10.65)$$

Since the output of h_μ is equal to $\phi(s, \mathbf{a})$, we may write

$$\phi(s, \mathbf{a}) = \phi(h_0, \mathbf{a}) = h_\mu \circ h_{\mu-1} \circ \cdots \circ h_1(h_0) \quad (10.66)$$

due to (10.64). Therefore ϕ and ℓ may be viewed as depending on each h_k , justifying the derivatives in (10.60).

Next, we now show that for each k , if h_{k-1} is known, then calculating h_k recursively via (10.64) requires $O(1)$ operations. To this end, fix $h_0 \in T$. If h_{k-1} is known, then calculating h_k requires a fixed number of operations that does not depend on μ . For example, if $\lambda = \text{ReLU}$ in (10.64), then calculating h_k requires two operations: one operation to calculate the product $a_k h_{k-1}$ and one operation to determine whether $a_k h_{k-1}$ is positive or negative.

Finally, observe that calculating all the outputs $h_1, \dots, h_\mu$ requires $O(\mu)$ operations. Indeed, since each h_k requires a fixed number of operations independent of μ , calculating all the μ outputs $h_1, \dots, h_\mu$ requires repeating this fixed number of operations μ times, leading to a total of $O(\mu)$ operations. Thus the claim of Step 1 is proved.

Step 2. In this step, we prove that

$$\text{the calculation of the values of } \frac{\partial \ell}{\partial h_k} \text{ for } k = 1, \dots, \mu \text{ requires } O(\mu) \text{ operations in total.} \quad (10.67)$$

To this end, we apply the chain rule to $\ell(s, \phi(s, \alpha))$ using (10.63) and (10.64). This yields a formula for $\partial \ell / \partial h_k$, which provides recursion in the *backward direction* as opposed to forward recursion (10.64):

$$\frac{\partial \ell}{\partial h_k} = \frac{\partial \ell}{\partial h_{k+1}} \frac{\partial h_{k+1}}{\partial h_k}, \quad k = 1, \dots, \mu - 1. \quad (10.68)$$

For $k = \mu$, the derivative $\partial \ell / \partial h_\mu$ is calculated directly since $s = h_0$ and $\phi(s, \alpha) = h_\mu$ as in (10.65), and therefore $\ell(s, \phi(s, \alpha)) = \ell(h_0, h_\mu)$. For example, if ℓ is the mean square loss as in (10.59), then there is a function $f : \mathbb{R} \rightarrow \mathbb{R}$ such that $\ell(h_0, h_\mu) = (h_\mu - f(h_0))^2$ (see Remark 10.4.2) and

$$\frac{\partial \ell}{\partial h_\mu} = 2(h_\mu - f(h_0)). \quad (10.69)$$

This derivative requires two operations to calculate: one for the subtraction and one for multiplication by 2. Note that h_μ was already calculated in Step 1, so no further operations are required to calculate h_μ . Since the total number of operations does not depend on μ , the calculation of $\frac{\partial \ell}{\partial h_\mu}$ takes $O(1)$ operations.

Now we show that the calculation of the remaining derivatives $\partial \ell / \partial h_k$ for $1 \leq k \leq \mu - 1$ also requires $O(1)$ operations for each k . To this end, we apply the *backward* recursive formula (10.68) for $k \leq \mu - 1$. Note that the backward recursion formula (10.68) contains the derivative $\partial h_{k+1} / \partial h_k$, which can be calculated by differentiating (10.64):

$$\frac{\partial h_{k+1}}{\partial h_k} = \lambda'(a_{k+1} h_k) a_{k+1}. \quad (10.70)$$

If h_k has already been calculated (see Step 1), then $\frac{\partial h_{k+1}}{\partial h_k}$ can be calculated in $O(1)$ operations: two multiplications and one evaluation of λ' . Similarly, if $\frac{\partial \ell}{\partial h_{k+1}}$ has already been calculated (and since $\partial h_{k+1}/\partial h_k$ requires $O(1)$ operations), then the number of operations to calculate $\frac{\partial \ell}{\partial h_k}$ is $O(1)$ due to (10.68). Therefore, since the last derivative $\frac{\partial \ell}{\partial h_\mu}$ has already been calculated in $O(1)$ operations, each of the previous derivatives $\frac{\partial \ell}{\partial h_k}$, $k = 1, \dots, \mu - 1$, can be calculated in a backward recursive process with $O(1)$ operations in each of the $\mu - 1$ recursive steps, leading to a total of $O(\mu)$ operations to calculate all the derivatives $\frac{\partial \ell}{\partial h_k}$, $k = 1, \dots, \mu$.

Step 3. Due to (10.60), it remains to show that

$$\text{the calculation of the values of } \frac{\partial h_k}{\partial a_k} \text{ for } k = 1, \dots, \mu \text{ requires } O(\mu) \text{ operations in total.} \quad (10.71)$$

Differentiating (10.64), we obtain

$$\frac{\partial h_k}{\partial a_k} = \lambda'(a_k h_{k-1}) h_{k-1}. \quad (10.72)$$

Since h_{k-1} was already calculated in Step 1, $\frac{\partial h_k}{\partial a_k}$ can be calculated in $O(1)$ operations for each k : two multiplications and a fixed number of operations to calculate λ' . For example, if λ is ReLU, then

$$\lambda'(x) = \begin{cases} 0, & x \leq 0, \\ 1, & x > 0. \end{cases} \quad (10.73)$$

In this case, calculating $\lambda'(a_k h_k)$ requires only one operation: determining whether $a_k h_k$ is positive or negative. This leads to a total of $O(\mu)$ operations to calculate $\frac{\partial h_1}{\partial a_1}, \dots, \frac{\partial h_\mu}{\partial a_\mu}$.

Step 4. In this step, we use the quantities calculated in Steps 2 and 3 to obtain the gradient $\nabla L(\alpha)$ and show that it can be done in $O(\mu)$ operations.

From (10.61) we have

$$\frac{\partial L}{\partial a_k} = \frac{\partial \ell}{\partial h_k} \frac{\partial h_k}{\partial a_k}, \quad k = 1, \dots, \mu. \quad (10.74)$$

Since $\partial \ell / \partial h_k$ and $\partial h_k / \partial a_k$ have already been calculated in Steps 2 and 3 for $k = 1, \dots, \mu$, we only need to perform one multiplication for each $k = 1, \dots, \mu$, leading to a total of μ operations to compute (10.74).

Since each of the Steps 1–4 requires $O(\mu)$ operations and since $4 \times O(\mu) = O(\mu)$, we conclude that the calculation of the gradient can be done in $O(\mu)$ operations. $\square$

Important remark

The savings in the backpropagation come from the recursive formulas (10.64) and (10.68). While calculating h_μ requires μ operations, in the process of this calculation, we also calculate h_k for $k = 1, \dots, \mu - 1$ due to (10.64). Similarly, due to (10.68), calculating $\partial\ell/\partial h_1$ requires $O(\mu)$ operations, but $\partial\ell/\partial h_k$ for $k = 2, \dots, \mu$ are calculated along the way. Thus each of the four steps of the backpropagation algorithm described in the proof above requires $O(\mu)$ operations and also generates μ pieces of information used in the calculation of the gradient.

10.5 Backpropagation algorithm for general DNNs

In this section, we describe the steps of the backpropagation algorithm applied to DNNs having M layers with no restrictions on their size. We index layers starting from $k = 0$ and denote by n_k the number of neurons in layer $\#k$. We index layer functions starting from 1 so that the k th layer function is $h_k : \mathbb{R}^{n_{k-1}} \rightarrow \mathbb{R}^{n_k}$ with n_0 being the number of inputs of the DNN.

From Definition 4.1.2, the layer function is expressed as $h_k(\mathbf{x}) = \bar{\lambda}(A_k \mathbf{x} + \mathbf{b}_k)$, where A_k is an $n_k \times n_{k-1}$ weight matrix, $\mathbf{b}_k \in \mathbb{R}^{n_k}$ is a bias vector (see Figure 10.5), and $\bar{\lambda}$ is an activation function (recall that $\bar{\lambda} : \mathbb{R}^{n_k} \rightarrow \mathbb{R}^{n_k}$ is the component-wise action of a scalar activation function $\lambda : \mathbb{R} \rightarrow \mathbb{R}$). Let $\mu_k = n_k(n_{k-1} + 1)$ be the total number of parameters in the k th layer, and let $\mathbf{a}_k \in \mathbb{R}^{\mu_k}$ be a vector containing all the parameters in both A_k and $\mathbf{b}_k$ in the k th layer as follows:

$$\mathbf{a}_k = ((A_k)_{11}, (A_k)_{12}, \dots, (A_k)_{n_k n_{k-1}}, (\mathbf{b}_k)_1, \dots, (\mathbf{b}_k)_{n_k}). \quad (10.75)$$

Denote by $\mu = \mu_1 + \dots + \mu_M$ the total number of parameters of the DNN and by $\mathbf{a} \in \mathbb{R}^\mu$ the vector of all parameters of the DNN:

$$\mathbf{a} = (\mathbf{a}_1, \dots, \mathbf{a}_M). \quad (10.76)$$

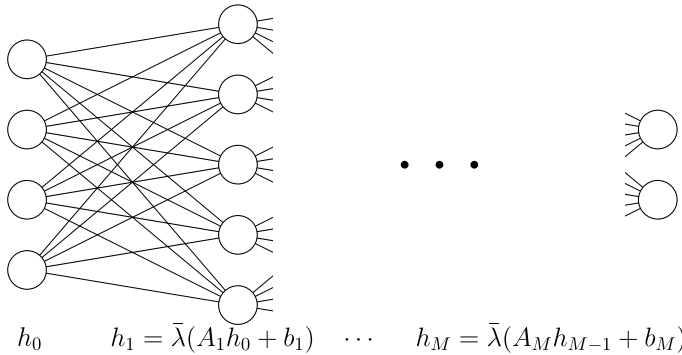


Figure 10.5: Diagrammatic representation for a general DNN.

Let $T \subset \mathbb{R}^{n_0}$ be a training set for the DNN (or a batch of the training set used in SGD), and let $L : \mathbb{R}^\mu \rightarrow \mathbb{R}$ be a loss function of the form (10.58). We now describe how to implement the backpropagation algorithm to calculate $\nabla L(\mathbf{a})$ for a given $\mathbf{a} \in \mathbb{R}^\mu$:

$$\nabla L(\mathbf{a}) = \left(\frac{\partial L}{\partial \mathbf{a}_1}, \dots, \frac{\partial L}{\partial \mathbf{a}_M} \right), \quad (10.77)$$

where each $\partial L / \partial \mathbf{a}_k$ is a vector corresponding to the sum of $\partial \ell / \partial \mathbf{a}_k(s, \cdot)$ for each object s in the batch. In turn, each $\partial \ell / \partial \mathbf{a}_k(s, \cdot)$ has μ_k components (see Figure 10.6) given by

$$\underbrace{\frac{\partial L}{\partial \mathbf{a}_k}}_{1 \times \mu_k \text{ matrix}} = \underbrace{\frac{\partial \ell}{\partial h_k}}_{1 \times n_k \text{ matrix}} \underbrace{\frac{\partial h_k}{\partial \mathbf{a}_k}}_{n_k \times \mu_k \text{ matrix}}. \quad (10.78)$$

$$1 \left\{ \underbrace{\left[\frac{\partial L}{\partial \mathbf{a}_k} \right]}_{\mu_k} \right\} = 1 \left\{ \underbrace{\left[\frac{\partial \ell}{\partial h_k} \right]}_{n_k} \underbrace{\left[\frac{\partial h_k}{\partial \mathbf{a}_k} \right]}_{\mu_k} \right\}_{n_k}$$

Figure 10.6: Dimensions of matrices in (10.80).

- Forward propagation. Forward propagation (also referred to as a forward pass) is not technically part of the backpropagation algorithm, though it is a necessary preliminary. This is why we denote it by “step 0”. To this end, we calculate the value of each layer function h_k , $k = 1, \dots, M$, for each input $s \in T$ using the recursive formula

$$h_k = \bar{\lambda}(A_k h_{k-1} + \mathbf{b}_k), \quad (10.79)$$

where $h_0 = s \in T$.

- Calculation of $\partial \ell / \partial h_k$. Using the fact that $\phi(s, \mathbf{a})$ in (10.58) is equal to h_M , calculate $\frac{\partial \ell}{\partial h_M}$ for each input $s \in T$. This requires at most $O(n_M)$ operations, as seen in the example of mean square loss: if $\ell(s, h_M) = \|h_M - f(s)\|^2$, then $\partial \ell / \partial h_M = 2(h_M - f(s))$, leading to $O(n_M)$ operations as $h_M, f(s) \in \mathbb{R}^{n_M}$.

Then for each $s \in T$, use the recursive formula

$$\underbrace{\frac{\partial \ell}{\partial h_k}}_{1 \times n_k \text{ matrix}} = \underbrace{\frac{\partial \ell}{\partial h_{k+1}}}_{1 \times n_{k+1} \text{ matrix}} \underbrace{\frac{\partial h_{k+1}}{\partial h_k}}_{n_{k+1} \times n_k \text{ matrix}} \quad (10.80)$$

to calculate $\frac{\partial \ell}{\partial h_k}$ for each k , starting from $k = M - 1$ and proceeding backward to $k = 1$. The backward recursive formula (10.80) shows how to calculate the deriva-

tives backward starting from the last layer and proceeding to the first; thus allowing us to reuse previously calculated derivatives. The matrices in the right-hand side of (10.80) come from the multivariable chain rule (A.8). Specifically,

$$\frac{\partial h_{k+1}}{\partial h_k} = \frac{d\bar{\lambda}}{dx} \Big|_{x=A_{k+1}h_k + \mathbf{b}_{k+1}} \underbrace{A_k}_{n_{k+1} \times n_k \text{ matrix}}, \quad (10.81)$$

$n_{k+1} \times n_{k+1} \text{ matrix}$

where $\frac{d\bar{\lambda}}{dx} \Big|_{x=A_{k+1}h_k + \mathbf{b}_{k+1}}$ is the diagonal matrix with the i th diagonal entry $\lambda'((A_{k+1}h_k + \mathbf{b}_{k+1})_i)$. For example, if λ is ReLU, then the i th diagonal entry of $\frac{d\bar{\lambda}}{dx} \Big|_{x=A_{k+1}h_k + \mathbf{b}_{k+1}}$ is 1 or 0, depending on whether the i th component of $A_{k+1}h_k + \mathbf{b}_{k+1}$ is positive or negative. We first apply straightforwardly the computational complexity formula (10.15) to matrix multiplication in (10.81) without taking into account specific structure of the matrices (e.g., diagonal). This results in $O(n_{k+1}^2 n_k)$ operations. However, since $\frac{d\bar{\lambda}}{dx} \Big|_{x=A_{k+1}h_k + \mathbf{b}_{k+1}}$ is a diagonal matrix, the matrix multiplication in (10.81) only requires $O(n_{k+1} n_k)$ operations.

Formula (10.80) involves other $O(n_{k+1} n_k)$ operations to estimate the product.

2. Calculation of $\partial h_k / \partial \mathbf{a}_k$. Take the derivative of (10.79) in $\mathbf{a}_k$ to get

$$\frac{\partial h_k}{\partial \mathbf{a}_k} = \frac{d\bar{\lambda}}{dx} \Big|_{x=A_k h_{k-1} + \mathbf{b}_k} \underbrace{\frac{\partial (A_k h_{k-1} + \mathbf{b}_k)}{\partial \mathbf{a}_k}}_{n_k \times n_k (n_{k-1} + 1) \text{ matrix}}. \quad (10.82)$$

As in Step 1, the matrix $\frac{d\bar{\lambda}}{dx} \Big|_{x=A_k h_{k-1} + \mathbf{b}_k}$ is diagonal with the i th entry $\lambda'((A_k h_{k-1} + \mathbf{b}_k)_i)$. For each $k = 1, \dots, M$ and each $s \in T$, calculate the $n_k \times (n_k \cdot n_{k-1})$ matrix¹ $\frac{\partial h_k}{\partial A_k}$ and the $n_k \times n_k$ matrix $\frac{\partial h_k}{\partial \mathbf{b}_k}$ using (10.79). Then combine these to form the $n_k \times \mu_k$ matrix $\frac{\partial (A_k h_{k-1} + \mathbf{b}_k)}{\partial \mathbf{a}_k}$. Although not diagonal, this matrix is sparse, meaning that most of its entries are zero. It has the following form:

$$\begin{aligned} & \frac{\partial (A_k h_{k-1} + \mathbf{b}_k)}{\partial \mathbf{a}_k} \\ &= \begin{pmatrix} (h_{k-1})_1 & \cdots & (h_{k-1})_{n_{k-1}} & \cdots & 0 & \cdots & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & \cdots & (h_{k-1})_1 & \cdots & (h_{k-1})_{n_{k-1}} & 0 & \cdots & 1 \end{pmatrix}. \end{aligned} \quad (10.83)$$

Observe from (10.83) that the first n_{k-1} columns of $\frac{\partial (A_k h_{k-1} + \mathbf{b}_k)}{\partial \mathbf{a}_k}$ have the elements of h_{k-1} in the first row and zeros in all other rows. The next n_{k-1} columns have

¹ Though A_k is a matrix, here treat it as a vector in $\mathbb{R}^{n_k n_{k-1}}$.

the elements of h_{k-1} in the second row. The pattern continues until columns $(n_k - 1)n_{k-1} + 1$ through $n_k n_{k-1}$, which has the elements of h_{k-1} in the last row. The last n_k columns are the identity matrix. The first $n_k n_{k-1}$ columns of $\frac{\partial(A_k h_{k-1} + \mathbf{b}_k)}{\partial \mathbf{a}_k}$ represent the derivative of $A_k h_{k-1} + \mathbf{b}_k$ with respect to the elements of A_k , and the last n_k columns represent the derivative of $A_k h_{k-1} + \mathbf{b}_k$ with respect to the elements of $\mathbf{b}_k$. In general, the multiplication of an $n_k \times n_k$ matrix by an $n_k \times n_k(n_{k-1} + 1)$ matrix as in (10.82) is expected to take $O(n_k^3(n_{k-1} + 1))$ operations due to (10.15). However, in (10.82), both matrices are sparse, so the number of operations required is only $O(n_k(n_{k-1} + 1))$. Since both factors in (10.82) are sparse, the product $\partial h_k / \partial \mathbf{a}_k$ is also sparse, having only one nonzero element in each of its μ_k columns.

3. Calculation of $\partial \ell / \partial \mathbf{a}_k$. For each $k = 1, \dots, M$ and each $s \in T$, calculate $\frac{\partial \ell}{\partial \mathbf{a}_k}$ using

$$\underbrace{\frac{\partial \ell}{\partial \mathbf{a}_k}}_{1 \times \mu_k \text{ matrix}} = \underbrace{\frac{\partial \ell}{\partial h_k}}_{1 \times n_k \text{ matrix}} \underbrace{\frac{\partial h_k}{\partial \mathbf{a}_k}}_{n_k \times \mu_k \text{ matrix}}, \quad (10.84)$$

where $\frac{\partial \ell}{\partial h_k}$ is obtained in Step 2, and $\frac{\partial h_k}{\partial \mathbf{a}_k}$ is obtained in Step 3.

4. Sum the derivatives of individual loss $\frac{\partial \ell}{\partial \mathbf{a}_k}$ over T for each $k = 1, \dots, M$ to obtain the derivatives with respect to parameters in the k th layer:

$$\frac{\partial L}{\partial \mathbf{a}_k} = \frac{1}{|T|} \sum_{s \in T} \frac{\partial \ell}{\partial \mathbf{a}_k}. \quad (10.85)$$

5. Combine all the μ_k -dimensional vectors $\frac{\partial L}{\partial \mathbf{a}_k}$, $k = 1, \dots, M$ into the single μ -dimensional vector

$$\nabla L(\mathbf{a}) = \frac{dL}{d\mathbf{a}} = \left(\frac{\partial L}{\partial \mathbf{a}_1} \mid \frac{\partial L}{\partial \mathbf{a}_2} \mid \dots \mid \frac{\partial L}{\partial \mathbf{a}_M} \right). \quad (10.86)$$

Finally, we can compute the computational complexity of the above steps to obtain the total complexity of the backpropagation algorithm.

0. For each k , computing $A_k h_{k-1}$ requires $O(n_k n_{k-1})$ operations (see (10.15)). Adding $\mathbf{b}_k$ and evaluating the activation function $\bar{\lambda}$ are both $O(n_k)$ operations. Therefore the computational complexity of this step is

$$\sum_{k=1}^M O(n_k n_{k-1}) + O(n_k) = \sum_{k=1}^M O(n_k n_{k-1} + n_k) = \sum_{k=1}^M O(\mu_k) = O(\mu). \quad (10.87)$$

1. As observed above, calculating $\frac{\partial h_{k+1}}{\partial h_k}$ requires $O(n_k n_{k+1})$ operations for each $k < M$. Furthermore, by (10.15) the matrix multiplication (10.80) also requires $O(n_k n_{k+1})$ operations. Finally, we observed above that the calculation of $\partial \ell / \partial h_M$ requires $O(n_M)$ operations. Therefore the total computational complexity of this step is

$$O(n_M) + \sum_{k=1}^{M-1} \mu(n_k n_{k+1}) = \sum_{k=1}^{M-1} O(\mu_{k+1}) = O(\mu). \quad (10.88)$$

2. The multiplication of sparse matrices in (10.82) requires only $O(n_k(n_{k-1}+1)) = O(\mu_k)$ operations since each of the two factor matrices has only one nonzero entry in each column. Therefore the computational complexity of this step is

$$\sum_{k=1}^M O(\mu_k) = O(\mu). \quad (10.89)$$

3. For each k , the matrix $\frac{\partial \ell}{\partial \mathbf{a}_k}$ only has one nonzero entry in each column as a result of Step 3. Therefore the matrix multiplication (10.84) requires only $O(\mu_k)$ operations, and the total complexity for this step is

$$\sum_{k=1}^M O(\mu_k) = O(\mu). \quad (10.90)$$

Thus the total computational complexity the backpropagation algorithm for one object $s \in T$ is $O(\mu)$. In Step 4, we calculate $\partial L / \partial \mathbf{a}_k$ for each k by repeating Steps 0–3 for each $s \in T$. Finally, in Step 5, these $\partial L / \partial \mathbf{a}_k \in \mathbb{R}^{\mu_k}$ are assembled into a single vector $\partial L / \partial \mathbf{a} \in \mathbb{R}^\mu$. Because $|T|$ does not depend on μ , repeating Steps 0–3 $|T|$ times has the computational complexity as completing Steps 0–3 $|T|$ times. Furthermore, assembling $\partial L / \partial \mathbf{a} \in \mathbb{R}^\mu$ requires no additional operations. Thus the full backpropagation algorithm requires $O(|T|\mu)$ operations.

The key step in backpropagation is the backward recursive step (10.80). If we calculate each derivative $\partial \ell / \partial h_k$ independently without this recursive step, then we would replace (10.80) with

$$\frac{\partial \ell}{\partial h_k} = \frac{\partial \ell}{\partial h_M} \prod_{j=k}^{M-1} \frac{\partial h_{j+1}}{\partial h_j}. \quad (10.91)$$

For fixed k , calculating $\partial \ell / \partial h_k$ requires

$$\sum_{j=k+1}^M O(n_{j+1}n_j) = \sum_{j=k+1}^M O(\mu_j). \quad (10.92)$$

Now repeating the calculation of (10.91) for each $k = 1, \dots, M-1$ requires

$$\sum_{k=1}^M \sum_{j=k+1}^M O(\mu_j) = \sum_{k=2}^M (k-1)O(\mu_k) = MO(\mu) = O(M\mu) > O(\mu) \quad (10.93)$$

operations, considerably more than backpropagation.

10.6 Exercises

Exercise 1 (Basic computational complexity). Assume that the computational cost of additions, multiplications, and exponentials are comparable. Give the number of operations necessary to calculate

$$f(x) = e^x + 6e^{3x} - 5,$$

taking an exponential as one operation.

Exercise 2 (Computational complexity and order of operations). Assume that we are given a function $f: \mathbb{R} \rightarrow \mathbb{R}$ that is costly to calculate with respect to additions or multiplications.

a. Consider

$$g(x) = f(x) + 2(3 + f(x)).$$

Show two different manners of calculating g , one that involves calculating f only once and one that involves calculating f twice.

b. Consider now the more complex formula

$$g(x) = \sum_{k=-5}^5 (2+k)f(k^2 x).$$

- i. Explain why a straightforward calculation of $g(x)$ calculates f 11 times.
- ii. Show how it is possible to calculate $g(x)$ by calculating f only 6 times.

Exercise 3 (Computational complexity of the multiplication between a sparse matrix and a full vector). Consider a matrix $M \in M_{N \times N}(\mathbb{R})$. The matrix M has n non-zero entries with $n \ll N$. Consider a vector $u \in \mathbb{R}^N$ with all non-zero entries.

- a. Show that the total number of multiplications to calculate Mu in a straightforward manner is always n .
- b. Give two examples of $N \times N$ matrices M_1 and M_2 , both with $n \ll N$ non-zero entries, but such that the number of additions needed to calculate $M_1 u$ is different from the number of additions needed to calculate $M_2 u$.

Exercise 4 (Another example of chain rule). We define the function of three variables $\sigma: \mathbb{R}^3 \rightarrow \mathbb{R}$ by

$$\sigma(x, y, z) = \cos(x + 2 \sin(z + \exp(-y))).$$

Explain how to efficiently calculate all three partial derivatives of σ following the steps spelled out in Section 10.3.

Exercise 5 (An example of indifferent order of differentiating). We define the function of four variables $\sigma: \mathbb{R}^3 \rightarrow \mathbb{R}$ by

$$\sigma(x, w, y, z) = w^2 + 2x^4 + 5y + 3z^3.$$

Explain why the order in which we calculate the partial derivatives of σ does not affect the computational cost.

Exercise 6 (An example of backpropagation with two neurons per layer). We consider here how backpropagation works on a simple network with two neuron functions per layer and a total of M layer functions.

- a. Explain why the network can be described by a sequence of layer functions $h_i: \mathbb{R}^2 \rightarrow \mathbb{R}^2$ such that our full classifier is a function $h: \mathbb{R}^2 \rightarrow \mathbb{R}^2$ with

$$h(x) = h_M \circ \dots \circ h_1(x).$$

- b. We use fully connected layers without bias for each h_i . Explain why this means that each h_i depends on four parameters $a_{i,1}, \dots, a_{i,4}$ that form a matrix

$$A_i = \begin{bmatrix} a_{i,1} & a_{i,2} \\ a_{i,3} & a_{i,4} \end{bmatrix},$$

so that the function h_i is given by

$$h_i(x) = \bar{\lambda}(A_i x)$$

for some non-linear fixed activation function $\bar{\lambda}$.

- c. Because of the parameters, we now denote $h_i = h_i(x, a_{i,1}, \dots, a_{i,4})$ by explicitly including the parameters. Give the formula for the partial derivatives

$$\frac{\partial h_i}{\partial x}, \frac{\partial h_i}{\partial a_{i,1}}, \frac{\partial h_i}{\partial a_{i,2}}, \frac{\partial h_i}{\partial a_{i,3}}, \frac{\partial h_i}{\partial a_{i,4}}$$

and explain the computational cost to calculate each of them.

- d. Explain how we may use the chain rule to calculate any partial derivative $\partial h / \partial a_{i,j}$ for any $i = 1, \dots, M$ and $j = 1, \dots, 4$.
- e. Calculate the number of additions and multiplications to calculate one partial derivative $\partial h / \partial a_{i,j}$ in terms of i .
- f. Explain how to use backpropagation to efficiently calculate all partial derivatives $\partial h / \partial a_{i,j}$ for $i = 1, \dots, M$ and $j = 1, \dots, 4$. Estimate the number of additions and multiplications required.

Exercise 7 (Estimating the cost of forward and backpropagation). We consider for this exercise a straightforward example of a fully linear deep learning neural network. It can

be represented by some function $f : \mathbb{R}^N \rightarrow \mathbb{R}^N$, which corresponds to K fully linear layers. Each layer k is represented by a function $f_k : \mathbb{R}^N \rightarrow \mathbb{R}^N$ given by

$$f_k(x) = \phi(A_k x),$$

where A_k is an $N \times N$ matrix, and $\phi : \mathbb{R}^N \rightarrow \mathbb{R}^N$ is, for example, the absolute value function with

$$(\phi(x))_i = |x_i|.$$

In that context, we simply have that

$$f(x) = f_K(f_{K-1}(\dots f_1(x) \dots)).$$

Finally, we choose as a loss function the sum

$$L : \mathbb{R}^N \rightarrow \mathbb{R}, \quad L(x) = \sum_{i=1}^N x_i.$$

- Give the total number of multiplications and additions (counted together) involved in calculating any $f_k(x)$. Do not count taking the absolute values.
- Deduce the total number of multiplications and additions (again counted together) to calculate $L(f(x))$.
- Denote by $f_{k,i}$ all coordinates of $f_k(x) = (f_{k,1}(x), \dots, f_{k,N}(x))$. Estimate the number of operations to calculate all partial derivatives $\partial f_{k,i} / \partial x_j$.
- Denote all coefficients in the matrix A_k by $A_{k,mn}$ for $i, j = 1, \dots, N$. Consider now the function f_k as a function both of x and of $A_{k,mn}$: $f_k = f_k(x, A_k) = f_k(x, (A_{k,mn})_{mn}) = f_k(x, A_{k,11}, A_{k,12}, \dots, A_{k,NN})$. Estimate the number of operations to calculate one partial derivative $\partial f_{k,i} / \partial A_{k,mn}$ for one choice of $m, n \leq N$.
- Estimate the number of operations to calculate all partial derivative $\partial f_{k,i} / \partial A_{k,mn}$ for one choice of $m, n \leq N$.
- Consider similarly the function f as a function of x and all $A_{k,mn}$ for all k and m, n : $f = f(x, (A_k)_{k \leq K}) = f(x, (A_{k,mn})_{k \leq K, m, n \leq N}) = f(x, A_{1,11}, \dots, A_{K,NN})$. Reproduce the analysis of Section 10.5 to estimate the total number of operations to calculate all partial derivatives $\partial L(f) / \partial A_{k,mn}$ for all $k \leq K$ and $m, n \leq N$.

Exercise 8 (Implementing backpropagation). In this exercise, you will implement the backpropagation and observe how much faster it is than finite difference. This is a programming exercise based on Python.

- Complete the following steps to build a simple DNN in Python without the aid of Pytorch. This DNN has many layers with only one neuron in each layer.
 - Copy and paste the following code to import Numpy and set the number of layers:

```
import numpy
num_layers = 30
```

- ii. Complete the following code for the so-called “leaky ReLU function” by replacing ... with commands that return x if $x > 0$ and $0.5x$ otherwise:

```
def ReLU(x):
    ...
```

We will also find it useful to have a function for the derivative of ReLU. Complete the following code for this derivative by replacing ... with commands that return 1 if $x > 0$ and 0.5 otherwise:

```
def dReLU(x):
    ...
```

- iii. Copy and paste the following code:

```
def DNN(x, a, b, return_list):
    h = [x]
    for i in range(num_layers):
        h.append(ReLU(a[i]*h[i]+b[i]))
    if return_list:
        return h
    return h[num_layers]
```

This code defines a function DNN accepting four inputs: x is the scalar input to the DNN, a and b are respectively the weight and bias vectors of the DNN, and return_list is a true/false variable, which determines whether the function should return only the output of the last neuron or a list of the output of all neurons (the later option is useful in backpropagation). Then define a function L that accepts a single variable and returns the square of that variable. This will be the loss function.

- iv. Copy and paste the following code implementing the backpropagation algorithm that calculates the gradient of the loss function with respect to the parameter vectors a and b :

```
def BP(x, a, b):
    h = DNN(x,a,b,True)
    dLdh = [2*h[num_layers]]
    for i in range(num_layers-1,0,-1):
        dLdh.insert(0,dLdh[0]*dReLU(a[i]*h[i]+b[i])*a[i])
        dLda = [dLdh[i]* dReLU(a[i]*h[i]+b[i])*h[i]
                for i in range(num_layers)]
        dLdb = [dLdh[i]* dReLU(a[i]*h[i]+b[i])
                for i in range(num_layers)]
    return dLda + dLdb
```

Write a short comment after each line describing what it does.

- v. Copy and paste the following code, which implements the finite difference algorithm that approximates the gradient of the loss function with respect to the parameter vectors *a* and *b*:

```
def FD(x, a, b): # Definition of the Finite Difference function
    base = L(DNN(x,a,b,False))
    eps = 0.001 # define a small parameter
    dLda=[0 for i in range(num_layers)]
    dLdb=[0 for i in range(num_layers)]
    for i in range(num_layers): # for-loop over all layers
        aeps = a.copy() # create a copy of the list of parameters a
        aeps[i]+= eps
        beps = b.copy()
        beps[i]+= eps
        dLda[i] = (L(DNN(x,aeps,b,False))-base)/eps
        dLdb[i] = (L(DNN(x,a,beps,False))-base)/eps
    return dLda + dLdb # concatenate two lists
```

Write a short comment after each line of the code that does not have a comment describing what it does.

- b. Generate two random lists of parameters *a* and *b* of length *num_layers* and a random number *x*. These values are, respectively, the lists of weights, biases, and the input for the DNN. Copy and paste the following code, which computes the average time taken to compute the gradient of the loss function via finite difference and backpropagation:

```
import time

t1 = time.time()
for i in range(100):
    deriv=BP(my_x,a,b)
t2 = time.time()
for i in range(100):
    deriv = FD(my_x,a,b)
t3 = time.time()

print('avg time for backpropagation: '+str((t2-t1)/100))
print('avg time for finite difference: '+str((t3-t2)/100))
```

What are the times output by this code? Which method is more time consuming, finite difference or backpropagation?

- c. Double the variable *num_layers* and run all the code again. What is the ratio between the computation times obtained in this step and the computation times obtained in part *b*. Explain how these ratios agree with the computational complexities of backpropagation and gradient descent.

11 Convolutional neural networks (CNNs)

We now introduce *convolutional neural networks* (CNNs), which are a type of DNN well suited for image and audio data when not all but some local features of the data are important.

CNNs were inspired by the visual perceptive systems of mammals [62], through the first experiments on the underlying brain mechanism of vision by Nobel prize winners Hubel and Wiesel (see [30, 31], and the appendix of [26]).

However, in the earlier implementations of CNNs as in [45] or [22], the backpropagation algorithm had not yet been implemented. In 1989, LeCun et al. applied a new architecture for neural networks to recognize handwritten digits on the MNIST dataset consisting of 28×28 pixel images [39]. Although we focus on the simpler architecture of LeCun et al., we do want to emphasize that many recent contributions have dramatically improved upon the CNN architecture we describe here; see, for example, some of the recent winners [37, 67] of the crowdsourced image labeling competition ILSVRC.

CNNs involve a type of layer naturally called *convolutional layers*, in addition to the fully connected layers that we have studied thus far.

Convolutional layers try to take advantage of the *locality* of features in the data. For example, in the facial recognition problems, some of the main features of interest are the eyes, hair, mouth, etc. These features are represented by pixels in the same part of the image, which explains the term locality. In contrast, each neuron in a fully connected layer depends on every pixel in the input image leading to dependence on nonlocal information and many more parameters. Because each neuron in a convolutional layer only depends on a small number of pixels in the image, we can hope that the neuron will learn the features in these pixels much more quickly than if it depended on all of the pixels.

CNNs may also use other types of layers that we have not described so far. A common example consists of so-called pooling layers. Each neuron in a max pooling layer compares the values of several neurons in the previous layer and keeps the largest. This helps filtering out less important information in the network. Pooling layers are also helpful because although convolutional layers tend to have fewer parameters than fully connected layers, they generally have many more neurons. Placing a pooling layer following a convolution layer effectively reduces this number, keeping the total number of neurons in the network manageable.

11.1 Convolution

11.1.1 Convolution of functions

Convolutional layers implement a procedure called *convolution of matrices*, which is closely related to the usual continuous operation of *convolution* on functions defined as follows:

Definition 11.1.1. Let, $f, g : \mathbb{R}^n \rightarrow \mathbb{R}$ be continuous functions with compact support. Then the *convolution* of f and g is

$$(f * g)(\mathbf{x}) = \int_{\mathbb{R}^n} f(\mathbf{y})g(\mathbf{x} - \mathbf{y}) d\mathbf{y}. \quad (11.1)$$

Example 11.1.1. Consider the convolution defined (11.1.1) in one dimension. Choose $g = \chi_{[0, \varepsilon)}$, the *characteristic function* on $[0, \varepsilon)$ defined as

$$\chi_{[0, \varepsilon)} = \begin{cases} 1 & \text{if } x \in [0, \varepsilon), \\ 0 & \text{otherwise.} \end{cases} \quad (11.2)$$

Then consider the convolution of $\frac{1}{\varepsilon}\chi_{[0, \varepsilon)}$ with a continuous function $f : \mathbb{R} \rightarrow \mathbb{R}$:

$$\left(f * \frac{1}{\varepsilon}\chi_{[0, \varepsilon)}\right)(x) = \frac{1}{\varepsilon} \int_{\mathbb{R}} \chi_{[0, \varepsilon)}(y)f(x - y) dy. \quad (11.3)$$

Since the function $\frac{1}{\varepsilon}\chi_{[0, \varepsilon)}(y) = 0$ for $y \notin [0, \varepsilon)$, rewrite (11.3) as

$$\frac{1}{\varepsilon} \int_{\mathbb{R}} \chi_{[0, \varepsilon)}(x)f(x - y) dy = \frac{1}{\varepsilon} \int_0^\varepsilon f(x - y) dy. \quad (11.4)$$

Making the change of variables $t = x - y$, we obtain

$$\frac{1}{\varepsilon} \int_x^{x+\varepsilon} f(t) dt. \quad (11.5)$$

This is the average of f on the interval $(x, x + \varepsilon)$. Therefore, for each x , the function $\frac{1}{\varepsilon}\chi_{[0, \varepsilon)}$ selects a small interval $[x, x + \varepsilon)$ such that the value of the convolution $(\frac{1}{\varepsilon}\chi_{[0, \varepsilon)} * f)(x)$ only depends only on $f(t)$ for t in this small interval. For a different function $\tilde{f}(t)$ such that $\tilde{f}(t) = f(t)$ for $t \in [x, x + \varepsilon)$, the convolution of $\chi_{[0, \varepsilon)}$ with f and $\tilde{f}$ is the same at the point x , even if the two functions $f(t) \neq \tilde{f}(t)$ for $t \notin [0, \varepsilon)$.

Remark 11.1.1. The convolution operation has many applications in both mathematics and physics, which we do not describe here.

11.1.2 Convolution of matrices

2D grayscale images can naturally be represented by matrices, whose entries encode the pixel intensity values. Some local features can be described by the convolution of matrices, which is the discrete analog of Definition 11.1.2.

To describe this, it is useful to rewrite Definition 11.1.1 for $\mathbb{R}^2$ to emphasize the role played by each variable.

Definition 11.1.2. Let $f, g : \mathbb{R}^2 \rightarrow \mathbb{R}$ be continuous functions with compact support. Then the *convolution* of f and g is

$$(f * g)(x, y) = \int_{\mathbb{R}^2} f(x', y') g(x - x', y - y') dx' dy'. \quad (11.6)$$

This inspires the following definition.

Definition 11.1.3 (Convolution of matrices). Let K be a $D \times D$ matrix, and let A be an $N \times M$ matrix with $D < N, M$. Then the convolution of K and A is defined as the $(N - D + 1) \times (M - D + 1)$ matrix $B = (b_{ij})$ with

$$b_{ij} = \sum_{n,m=1}^D k_{n,m} a_{i+n-1, j+m-1}, \quad \begin{array}{l} i = 1, \dots, N - D + 1, \\ j = 1, \dots, M - D + 1. \end{array} \quad (11.7)$$

We say that B is K convolved with A (indices m and n correspond to the dummy variables in the integral in (11.6)).

Remark 11.1.2. The convolution B is smaller than A ; A has dimension $N \times M$, but B has dimension $(N - D + 1) \times (M - D + 1)$.

Remark 11.1.3. In (11.7) the matrix A is evaluated at indices $i+n-1$ and $j+m-1$, whereas in (11.6) the function g is evaluated at $(x - x', y - y')$. The -1 in both indices is also for technical convenience, so that the indices i, j of B start at 1.

Remark 11.1.4. We choose addition in the indices in Definition 11.1.3 (rather than subtraction used in Definition 11.1.2), so that it results in the so-called Hadamard product between matrices as described later (Definition 11.1.4). Often, using the Hadamard product is more convenient because it simplifies notation.

Example 11.1.2 (Simple low-dimension matrix convolution). Consider two square matrices

$$A = \begin{pmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 2 & 1 & 2 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 2 & 1 & 2 & 1 \\ 0 & 1 & 0 & 1 & 0 \end{pmatrix} \quad (11.8)$$

and

$$K = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 0 \end{pmatrix}. \quad (11.9)$$

Calculate the matrix $K * A$. Since A is 5×5 and K is 3×3 , we have $N = M = 5$ and $D = 3$. Then the size of $K * A$ is $(N - D + 1) \times (M - D + 1) = 3 \times 3$. We will use (11.7) to calculate each entry of $K * A(\alpha, \beta)$ for $1 \leq \alpha, \beta \leq 3$. Due to the symmetries in A , many of the entries of $K * A$ are the same. In fact, there are only three distinct values of the entries of $K * A$. We next present the calculation of the $(1, 1)$ convolution entry (entry $b_{1,1}$ in definition (11.7)):

$$\begin{aligned}
 (K * A)(1, 1) &= (K * A)(3, 1) = (K * A)(1, 3) = (K * A)(3, 3) \\
 &= \sum_{\alpha', \beta'=0}^3 K(\alpha', \beta') A(1 + \alpha' - 1, 1 + \beta' - 1) \\
 &= K(1, 1)A(1, 1) + K(1, 2)A(1, 2) + K(1, 3)A(1, 3) \\
 &\quad + K(2, 1)A(2, 1) + K(2, 2)A(2, 2) + K(2, 3)A(2, 3) \\
 &\quad + K(3, 1)A(3, 1) + K(3, 2)A(3, 2) + K(3, 3)A(3, 3) \\
 &= 0 \cdot 0 + 1 \cdot 1 + 0 \cdot 0 + 1 \cdot 1 + 2 \cdot 2 + 1 \cdot 1 + 0 \cdot 0 + 1 \cdot 1 + 0 \cdot 0 \\
 &= 8.
 \end{aligned} \tag{11.10}$$

The other entries are calculated similarly:

$$B := K * A = \begin{pmatrix} 8 & 6 & 8 \\ 6 & 4 & 6 \\ 8 & 6 & 8 \end{pmatrix}. \tag{11.11}$$

11.1.3 Hadamard product and feature detection

The convolution can also be understood via an inner product on matrices called the Hadamard product defined as follows.

Definition 11.1.4 (Hadamard product of matrices). Let A, B be $D \times D$ matrices. Then the Hadamard product of A and B is

$$A \odot B = \sum_{i,j=1}^D a_{i,j} b_{i,j} \in \mathbb{R}. \tag{11.12}$$

We will see that the convolution of a $D \times D$ kernel K and a matrix A is equal to a matrix of Hadamard products between K and various $D \times D$ submatrices of A , which are defined as follows:

Definition 11.1.5 (Submatrix of a matrix). Let $A = (a_{i,j})$ be an $N \times M$ matrix, and let $D > 0$. The $D \times D$ submatrix of A at indices (i, j) with $1 \leq i \leq N - D + 1$ and $1 \leq j \leq M - D + 1$ is

$$A[i, j; D] = \begin{pmatrix} a_{i,j} & \cdots & a_{i,j+D-1} \\ \vdots & \ddots & \vdots \\ a_{i+D-1,j} & \cdots & a_{i+D-1,j+D-1} \end{pmatrix}. \tag{11.13}$$

Simply put, $A[i, j; D]$, the $D \times D$ submatrix of A at indices i, j , consists of the entries of A in rows i through $i + D - 1$ and columns j through $j + D - 1$.

The Hadamard product and the notion of submatrices leads to the following equivalent definition of the convolution.

Definition 11.1.6 (Definition of matrix convolution as Hadamard product). Let K be a $D \times D$ kernel matrix, and let A be an $N \times M$ matrix with $D < \min\{N, M\}$. Then the convolution matrix $B = K * A$ is defined by its entries b_{ij} :

$$b_{ij} = K \odot A[i, j; D], \quad \begin{array}{l} i = 1, \dots, N - D + 1, \\ j = 1, \dots, M - D + 1. \end{array} \quad (11.14)$$

Formula (11.14) shows that the i, j entry of the convolution matrix B is the Hadamard product of K and the submatrix of A of the same size at indices i, j (its top left entry is a_{ij}).

Example 11.1.3 (Feature detection via Hadamard product). Consider the matrices A and K from Example 11.1.2 defined by (11.8) and (11.9), respectively. The entry $b_{1,1} = 8$ in (11.11) is computed as the Hadamard product of the upper-left 3×3 submatrix $A[1, 1; 3]$ and K :

$$A[1, 1; 3] = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 0 \end{pmatrix}, \quad K = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 0 \end{pmatrix}. \quad (11.15)$$

Since the two matrices happen to be the same, the Hadamard product leads to $b_{1,1}$ being the largest value among all the entries of B . Indeed, if we consider all entries of the matrix as a vector, then (11.12) becomes a standard inner product. We next observe that each element of convolution (11.11) is a Hadamard product of a 3×3 submatrix of A and the 3×3 matrix K . Recall the formula for the inner product of $x \cdot y = |x||y| \cos(\gamma)$ and therefore is largest when the vectors are parallel, i. e., $\cos(\gamma) = 1$ or when $x = \lambda y$ for any scalar $\lambda > 0$.

This can be interpreted as follows. The matrix $A[1, 1; 3]$ represents a feature in an image that is of primary interest for us. Then we choose the kernel K that mimics this feature as close as we can; in our case, we choose $K = A[1, 1; 3]$. Now a computer can detect the feature by finding the largest values in the convolution matrix B instead of searching for features represented by submatrices in the original image A . This demonstrates how convolution reduces the problem of feature detection to a simple problem of finding the largest entries in a matrix by choosing a convolution kernel that best mimics the feature.

Example 11.1.4 (Vertical edge detection via convolution). Consider a grayscale image described by a matrix whose entries are in $\{0, 1\}$:

$$A = \begin{pmatrix} 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{pmatrix}. \quad (11.16)$$

The corresponding image has white on the left half and black on the right half, with two regions separated by a vertical edge in the center of Figure 11.1. We call a pair of entries a_{ij} , a_{ij-1} horizontally adjacent, that is, they are located side by side in the same row. To detect the edge in the image from the matrix A directly, a computer would have to compare each pair of horizontally adjacent entries of A to find where there is a jump from 0 to 1 corresponding to a jump in color (white to black).

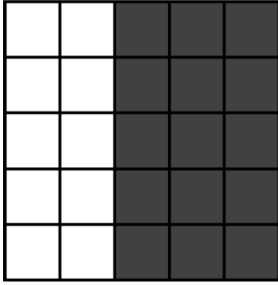


Figure 11.1: Each square in the grid represents a pixel that corresponds to an entry in matrix (11.16). White pixels are represented by 0s, and black pixels are represented by 1s.

Instead, we can consider the convolution with the kernel

$$K = \begin{pmatrix} -1 & 1 \\ -1 & 1 \end{pmatrix}. \quad (11.17)$$

Then the resulting convolution matrix B is

$$B := K * A = \begin{pmatrix} 0 & 2 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 2 & 0 & 0 \end{pmatrix}. \quad (11.18)$$

The kernel K is chosen so that 2 appears as a jump between two horizontally adjacent matrix entries or the pixels corresponding to these entries. Therefore the column of 2s traces the vertical edge.

Note that the values of -1 and 1 in each row of K results in the convolution detecting the difference between horizontally adjacent pixels. Indeed, we use the definition of convolutions $K * A$ via Hadamard product. Then consider the convolution of K with the 2×2 submatrix of the image matrix A :

$$\bar{A} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}, \quad (11.19)$$

which gives the following explicit formula for the i, j entry of the convolution matrix:

$$(K * \bar{A})_{ij} = b - a + d - c. \quad (11.20)$$

For matrix (11.16), there are two cases for the relative values of a, b, c, d in $\bar{A}$:

- If the entries of $\bar{A}$ are all zero or all one, i. e., $a = b = c = d$, then from (11.20) we get $K * \bar{A} = 0$, meaning that there is no edge in $\bar{A}$.
- If the first column in $\bar{A}$ is all zeros and the second is all ones, i. e., $a < b$ and $c < d$, then from (11.20) we get $K * \bar{A} > 0$, i. e., there is a jump in pixel intensity, which identifies an edge in the image.

Let us consider some additional examples highlighting how CNNs work on feature detection.

Example 11.1.5 (CNN layer with a single kernel to capture a single feature with known location). Consider a convolutional layer with a 7×7 input matrix X_{input} and a 3×3 kernel K shown in Figure 11.2. The output $K * X_{\text{input}}$ is a 5×5 matrix for a total of 25 neurons. The 25 entries of the output matrix are obtained by “sliding” the kernel over the entire image. The sliding means that we choose all possible 3×3 submatrices and convolve the kernel with all such submatrices.

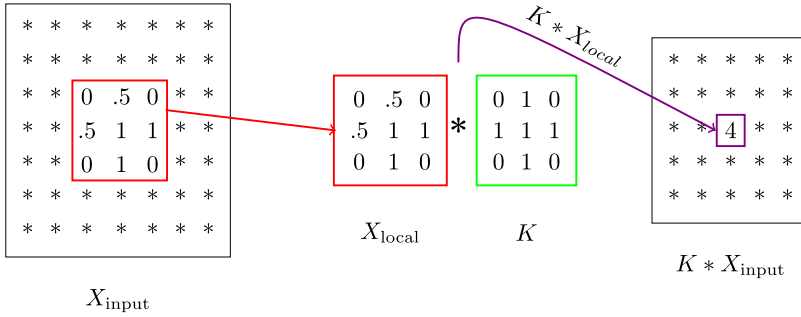


Figure 11.2: X_{input} is the input matrix, e. g. the input image (cat or dog). Properly chosen entries and size of kernel K extract features from X_{input} (e. g. cat’s eye). If this feature is represented by 3×3 “plus sign” submatrix X_{local} , then we choose kernel K that mimics this feature (3×3 “plus sign” submatrix). Because the kernel K and X_{local} are similar, the convolution $K * X_{\text{local}}$ outputs a “big” number, that is comparable to $\max K * X_{\text{local}} = 9$ over all entries in K and X_{local} between 0 and 1.

For the fixed kernel K , depicted in Figure 11.2 in green, the highest possible value of the convolution $K * X_{\text{input}}$ with 3×3 submatrices of any possible input X_{input} is 5 because of the assumption that the entries of X_{input} range from 0 to 1.

We next consider a specific choice of X_{input} with a fixed feature depicted in the red patch in Figure 11.2. Then the purple entry 4 corresponds to the maximum value of the convolution between the given feature (submatrix) in red and any 3×3 kernel whose values range from 0 to 1. More precisely, to maximize the output, we vary the entries of the kernel between 0 and 1. The feature is captured by the value of the output of the convolution, and therefore the higher the value, the better the feature is captured. Thus the maximum value 4 of the convolution in purple in Figure 11.2 corresponds to the best kernel for this patch.

We now compare the CNN with a fully connected DNN layer with the same input data in the form of a vector with $7 \times 7 = 49$ components and the same dimension of the output vector, i. e., $5 \times 5 = 25$ will have a weight matrix with $49 \times 25 = 1225$ parameters. This is to be compared with $3 \times 3 = 9$ entries of the CNN weight matrix, that is, we only have 9 parameters. However, recall that the small size kernel was chosen to match the size of the small feature, assuming that we are only interested in this small feature. If the entire image has other features with different sizes, then the small kernel will not be an appropriate choice.

Example 11.1.6 (CNN layer with a single kernel to capture a single feature with unknown location). In practice, we do not know where the feature is, and we slide the kernel over the entire image, and a position is determined by the largest value of the convolution. This sliding procedure is depicted in Figure 11.3. Note that the kernel K is convolved with the whole X_{input} , so the efficient extraction of the feature requires that the convolution of kernel K with the patches that do not contain this feature results in a small output. On the other hand, if we “slide” the kernel over the entire input X_{input} and if we compute the convolution each time, then the value of the convolution is maximized at the patch that contains the feature.

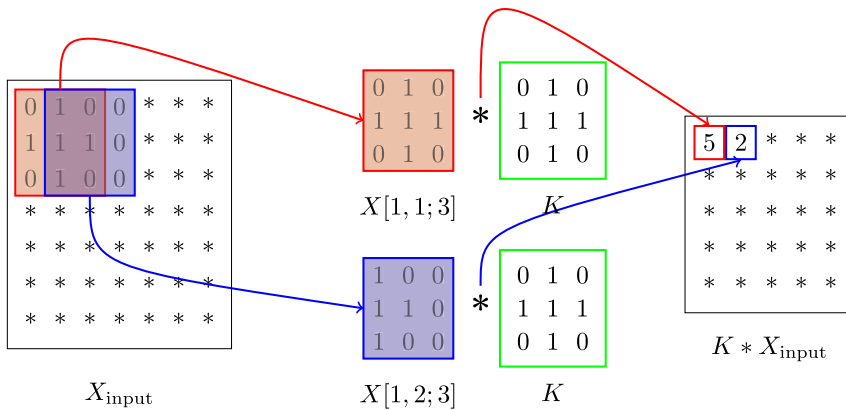


Figure 11.3: X_{input} is the input matrix. We want to find a feature of 3×3 size, which looks like a “plus sign” and therefore we choose the kernel K , which mimics this feature, as depicted in green. We slide this kernel over an image, take convolutions with all 3×3 submatrices X and observe that it is $K * X$ is maximized when $X = X[1,1;3]$, which is the 3×3 “plus sign” feature.

In the examples above, we considered a simple situation where the CNN layer has one single kernel, because we are looking for a single feature. In a more general situation, an image typically has several features of interest. which results in a CNN layer with multiple kernels. For example, such features can be vertical and horizontal lines, which are simple features, whereas more complicated features, such as eyes and noses, are composed from these ones.

Remark 11.1.5. We focus for simplicity on the convolution of matrices corresponding to, e.g., 2D images. However, we could define the convolution of tensors of any rank from 1 (for, e.g., audio data) to 2 (for, e.g., images), to 3 (for, e.g., video), and so on.

Remark 11.1.6. The name kernel for the matrix K that extracts features in the examples above comes from the definition of convolution of functions $f(y)$ with kernel $K(x - y)$:

$$\int K(x - y)f(y) dy.$$

11.2 Convolutional layers

In the examples in the previous section, we considered the use of fixed kernels in feature detection. One of the key ideas in CNNs is to treat the kernel matrices as trainable parameters, that is, we randomly initialize the entries of the kernel, which then becomes optimal during the training. This method is particularly useful when there are several features in the image, and therefore it is hard to choose the kernel by simply matching the entries of the feature. In this section, we present a rigorous mathematical definition of convolutional layers, which is a key building block of the mathematical notion of CNN function.

Applying Definition 4.1.2 of the layer function to the k th layer, we recall that the k th layer function $h_k : \mathbb{R}^{n_{k-1}} \rightarrow \mathbb{R}^{n_k}$ between the $(k - 1)$ th and k th layers is

$$h_k = \bar{\lambda}(A_k \cdot h_{k-1} + b_k), \quad (11.21)$$

where A_k is the weight matrix of the k th layer, b_k is the bias vector of the k th layer, $\bar{\lambda}$ is an activation function, and h_{k-1} is the output of the previous $(k - 1)$ th layer.

We next define the convolutional layer function, which is a basic building block of CNNs. Heuristically, convolutional layer functions can be obtained from fully connected layer functions by fixing certain weights to be zero, so that these weights “cannot be trained,” that is, they are not part of the minimization of the loss function. Moreover, the nonzero weights are shared between neurons as is further explained in Example 11.2.1.

Definition 11.2.1 (Convolutional layer function). Let K be a $D \times D$ kernel matrix of parameters, and let N and M be integers such that $D \leq N, M$. A *convolutional layer function* is a mapping $c : \mathbb{R}^{N \times M} \rightarrow \mathbb{R}^{(N-D+1) \times (M-D+1)}$ of the form

$$c(X) = \bar{\lambda}(K * X + b), \quad (11.22)$$

where X is an $N \times M$ matrix, b is the usual bias represented here by an $(N - D + 1) \times (M - D + 1)$ matrix, and $\bar{\lambda}$ is an activation function applied to each entry of the matrix $K * X + b$ defined in Definition 11.1.3. The matrices X and $c(X)$ are called the input and output of the layer function, respectively.

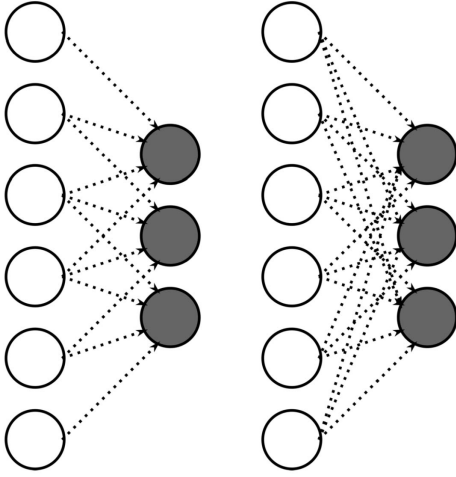


Figure 11.4: Convolutional layer (left) with the output (gray) neurons depending on four nodes of the previous layer denoted by dashed arrows from previous layer to the next layer. Equivalently, this convolutional layer uses a 2×2 kernel K and thus contains four weights. These four weights are assigned to the four dashed lines entering each gray neuron. The number of weights does not depend on the number of neurons in the previous or current layer; it only depends on the size of the kernel. Fully connected layer (right) where every node in the output depends on every node in the input. The number of weights is the product of the number of neurons in input and output layers, i. e., $6 \times 3 = 18$. In contrast with the convolution layer, the number of weights increases as the number of nodes in either layer increases.

Observe that the size of matrix $c(X)$ is smaller than that of a matrix X ; see Remark 11.1.2.

Definition 11.2.2 (Convolutional neuron function). Let K be a $D \times D$ kernel matrix of parameters, and let N and M be integers such that $D \leq N, M$. Then for integers i, j such that $1 \leq i \leq N - D + 1$ and $1 \leq j \leq M - D + 1$, a *convolutional neuron function* is a mapping $q_{i,j} : \mathbb{R}^{N \times M} \rightarrow \mathbb{R}$ of the form

$$q_{i,j}(X) = \lambda(K \odot X[i,j;D] + b_{i,j}), \quad (11.23)$$

where X is an $N \times M$ matrix, $X[i,j;D]$ is the submatrix of X defined by (11.13), b is an $(N - D + 1) \times (M - D + 1)$ matrix, and $\odot$ is the Hadamard product defined in (11.12).

As shown in Figure 11.4, definition 11.1.6 shows that the convolutional layer function $c(X)$ is a matrix whose entries are convolutional neuron functions $q_{i,j}(X)$:

$$c(X) = \begin{pmatrix} q_{1,1}(X) & \cdots & q_{1,M-D+1}(X) \\ \vdots & \ddots & \vdots \\ q_{N-D+1,1}(X) & \cdots & q_{N-D+1,M-D+1}(X) \end{pmatrix}. \quad (11.24)$$

Each entry $q_{i,j}(X)$ is the Hadamard product of K with the submatrix of X of the same size at indices i, j plus the biases.

In fully connected layers, we usually represent the input and output as vectors (see Definition 4.1.2). In contrast, in the convolutional layer defined above, it is easier to represent the input and output as matrices because of Definition 11.1.3 for matrix convolution. However, we may also represent the pixels as a vector if it is more convenient for a given algorithm.

Remark 11.2.1. In a convolutional neural network (CNN), the convolutional layer function c in (11.22) is the k th layer function in the network, i. e., $c(X) = c_k(X)$. The kernel $K = K_k$ has size $D_k \times D_k$, and

$$c_k : \mathbb{R}^{N_{k-1} \times M_{k-1}} \rightarrow \mathbb{R}^{N_k \times M_k}, \quad (11.25)$$

where $N_{k-1} \times M_{k-1}$ is the size of the output of the $(k-1)$ th layer (i. e., the number of neurons in the $(k-1)$ th layer), $N_k = N_{k-1} - D_k + 1$, and $M_k = M_{k-1} - D_k + 1$. The number of neurons in the k th layer is $N_k \cdot M_k$. The matrix X in (11.22) is the output of the $(k-1)$ th layer and input of k th layer, whereas $c_k(X)$ is the output of the k th layer. If $k = 1$, then X is the matrix corresponding to the image, and c_1 is the first layer.

One advantage of convolutional layers is that for the same size of input data, they use *fewer parameters* (weights) than a fully connected layer as the size of the kernel matrix K is much smaller than the size of the input matrix X because we are interested only in the local features of the image. Indeed, the fully connected layer function (11.21) depends on $N_k (N_{k-1} + 1)$ parameters. Since typically $N_k, N_{k-1} \gg 1$, the number of parameters is much larger than both N_k and N_{k-1} . By contrast, the k th convolutional layer function c_k depends only on D_k^2 parameters. Typically, D_k is much less than any of N_k, M_k, N_{k-1} , and M_{k-1} , so the number of parameters in a convolutional layer is much less than either the number of neurons in the layer or the number of neurons in the previous layer.

Example 11.2.1 (Weight sharing and sparsity). In this example, we compare Definition 4.1.2 of the layer function with Definition 11.2.1 of the convolutional layer function. The idea is to rewrite the definition of a convolutional layer in terms of matrix multiplications rather than convolutions. Consider the following two matrices for a convolution kernel and input to a convolutional layer function, respectively:

$$K := \begin{pmatrix} a & b \\ c & d \end{pmatrix}, \quad X := \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}. \quad (11.26)$$

Then applying Definition 11.1.3, we obtain

$$K * X = \begin{pmatrix} 1 \cdot a + 2 \cdot b + 4 \cdot c + 5 \cdot d & 2 \cdot a + 3 \cdot b + 5 \cdot c + 6 \cdot d \\ 4 \cdot a + 5 \cdot b + 7 \cdot c + 8 \cdot d & 5 \cdot a + 6 \cdot b + 8 \cdot c + 9 \cdot d \end{pmatrix}. \quad (11.27)$$

Next, rewrite the matrices $X \in \mathbb{R}^{3 \times 3}$ and $K * X \in \mathbb{R}^{2 \times 2}$ as a nine-dimensional vector $\bar{X}$ and a four-dimensional vector $\overline{K * X}$, respectively:

$$\bar{X} = (1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7 \ 8 \ 9)^T, \quad (11.28)$$

$$\overline{K * X} = \begin{pmatrix} 1 \cdot a + 2 \cdot b + 4 \cdot c + 5 \cdot d \\ 2 \cdot a + 3 \cdot b + 5 \cdot c + 6 \cdot d \\ 4 \cdot a + 5 \cdot b + 7 \cdot c + 8 \cdot d \\ 5 \cdot a + 6 \cdot b + 8 \cdot c + 9 \cdot d \end{pmatrix}. \quad (11.29)$$

These vectors are related via the usual matrix–vector multiplication:

$$\overline{K * X} = \mathcal{K} \tilde{X}, \quad (11.30)$$

where $\mathcal{K}$ is the matrix

$$\mathcal{K} = \begin{pmatrix} a & b & 0 & c & d & 0 & 0 & 0 & 0 \\ 0 & a & b & 0 & c & d & 0 & 0 & 0 \\ 0 & 0 & 0 & a & b & 0 & c & d & 0 \\ 0 & 0 & 0 & 0 & a & b & 0 & c & d \end{pmatrix}. \quad (11.31)$$

Using (11.30), the convolutional layer $c(X) = \tilde{\lambda}(K * X)$ with kernel K can be rewritten as a fully connected layer $h(\tilde{X}) = \tilde{\lambda}(\mathcal{K} \tilde{X})$ with parameter matrix $\mathcal{K}$. In other words, the output vector $h(\tilde{X})$ is equal to a “flattened” output matrix $c(X)$, and the input vector $\tilde{X}$ is equal to the “flattened” matrix X . Here by a “flattened” matrix we mean a vector obtained by rewriting all matrix entries in a single column; see, e. g., (11.26) and (11.28), respectively. In fact, each of the four neurons of a the fully connected layer h (i. e., four rows of $\mathcal{K}$) have the same four nonzero weights a , b , c , and d (though not all in the same columns). In this sense, all neuron functions of a convolutional layer *share* the same set of weights, e. g., a , b , c , and d in the above examples. Furthermore, these weights are entries of the convolution kernel K because we take the Hadamard product of all 2×2 submatrices of X with the same kernel K . This property is called *weight sharing*, and it reflects the reduction of parameters when using convolutional layers versus using fully connected layers. Finally, we note that just as in a fully connected layer, the weights (kernel) of a convolutional layer are trained via, e. g., gradient descent.

11.3 Padding layer

A common type of layers often employed together with CNNs are so-called padding layers. Padding layers typically add several rows and columns of 0 (or another constant) to an image near its boundary.

There can be several reasons to add such a layer. They are commonly used when the initial dataset is composed of objects that do not all have the same dimensions. For example, our dataset may correspond to a collection of gray-scale images with different resolutions: An image may have N horizontal pixels and M vertical pixels, whereas another image has $\tilde{N}$ horizontal pixels and $\tilde{M}$ vertical pixels with $N \neq \tilde{N}$ and/or $M \neq \tilde{M}$.

This may present a challenge when applying DNN techniques: First, numerical implementations can be difficult with inputs of varying sizes. Second, the architecture of a network often presumes a given size for the input. This is particularly obvious if the first layer is fully connected: As a fully connected layer includes a multiplication by a matrix, the sizes would have to match.

In such a setting padding can be added to each image separately. This does not affect the information contained in each one of them, but it is a straightforward solution to add to each image to make them all of identical size.

Padding layers can also be specifically useful when combined with CNNs. Observe that in the definition of the convolution of matrices (see Definition 11.1.3), the entry $a_{1,1}$ is only used once when calculating $b_{1,1}$ and is not used when calculating any $b_{i,j}$ with $i > 1$ or $j > 1$. Similarly, we can check that an entry on the first row, $a_{1,j}$ for $1 < j < M$, or first column, $a_{i,1}$ for $1 < i < M$, is used D times. However, any entry in the middle of the matrix, a_{ij} with $D \leq i, j \leq N - D + 1$, will be used D^2 times in the calculations.

Consequently, whichever type of information or feature was contained in the first or last rows/columns of the matrix a is weighted less on the resulting matrix b than features encoded in the center of a . This means that CNNs may be less sensitive to features near the boundary of images. Padding layers offer a straightforward solution to this issue: by adding rows and columns of 0s around a we can make it so that what were the first/last rows/columns are now enough in the center to be used as many times as other entries. We will also use padding layers later in this chapter in a somewhat similar manner to construct a DNN that is translational invariant.

In mathematical terms, given a matrix M in $\mathbb{R}^{n \times n}$, a padding layer with padding size $2s$ will add s components to the matrix at all extremities and fill them with 0 leading to the following definition.

Definition 11.3.1. A padding layer function with padding size $2s$ is a function $P : \mathbb{R}^{n \times n} \rightarrow \mathbb{R}^{(n+2s) \times (n+2s)}$ defined by

$$(P(M))_{\alpha,\beta} = M_{\alpha-s,\beta-s} \quad \text{if } s+1 \leq \alpha, \beta \leq n+s$$

and

$$(P(M))_{\alpha,\beta} = 0 \quad \text{if } \alpha \leq s, \alpha \geq n+s+1, \beta \leq s, \text{ or } \beta \geq n+s+1.$$

Simply put, a padding layer adds a “border” of 0s of width s around the input matrix.

11.4 Pooling layer

In this section, we discuss another type of layers that may be found in CNNs, *pooling layers*. Pooling layers may, for example, alternate with convolutional layers. The input of a pooling layer is the output matrix of the previous convolutional layer. The idea is that pooling layers remove smaller entries and keep large entries of this matrix, which hopefully correspond to the more important features. Mathematically, this means that the output of the pooling layer has smaller size than the output matrix of the previous convolutional layer.

Although there are many ways to perform pooling, for simplicity, we focus here on *max pooling*. We first introduce the entrywise max function on matrices $\overline{\max} : \mathbb{R}^{N \times N} \rightarrow \mathbb{R}$, which chooses the maximum element of a matrix. This definition will be applied below to submatrices of a larger matrix.

Definition 11.4.1. Let $X \in \mathbb{R}^{N \times N}$ be an $N \times N$ matrix. The entrywise max function $\overline{\max} : \mathbb{R}^{N \times N} \rightarrow \mathbb{R}$ is defined as

$$\overline{\max}(X) := \max_{1 \leq i, j \leq N} X_{ij}. \quad (11.32)$$

We next write X as a block matrix whose elements $X[i, j; D]$ are themselves $D \times D$ submatrices as in Definition 11.1.5. Every element of X is in exactly one of the blocks $X[i, j; D]$, that is, the blocks are disjoint. For example, choose $D = 2$, and if

$$X = \begin{pmatrix} X_{11} & X_{12} & \cdots & X_{1N} \\ X_{21} & X_{22} & \cdots & X_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ X_{N1} & X_{N2} & \cdots & X_{NM} \end{pmatrix}, \quad (11.33)$$

then the block $X[1, 1; 2]$ is the following submatrix of X :

$$X[1, 1; 2] = \begin{pmatrix} X_{11} & X_{12} \\ X_{21} & X_{22} \end{pmatrix}. \quad (11.34)$$

The max pooling layer is defined by applying the elementwise max function to each block.

Definition 11.4.2 (Max pooling layer function). Let $X \in \mathbb{R}^{N \times M}$ be a matrix represented by $D \times D$ block matrices $X[i, j; D]$, $i = 1, \dots, S = N/D$ and $j = 1, \dots, T = M/D$:

$$\begin{pmatrix} X[1, 1; D] & X[1, D+1; D] & \cdots & X[1, (T-1)D+1; D] \\ X[D+1, 1; D] & \ddots & \cdots & X[D+1, (T-1)D+1; D] \\ \vdots & \vdots & \ddots & \vdots \\ X[(S-1)D+1, 1; D] & \cdots & \cdots & X[(S-1)D+1, (T-1)D+1; D] \end{pmatrix}. \quad (11.35)$$

Then the max pooling layer function $p : \mathbb{R}^{N \times M} \rightarrow \mathbb{R}^{S \times T}$ is defined as

$$p(X) := \begin{pmatrix} \overline{\max}(X[1, 1; D]) & \cdots & \overline{\max}(X[1, (T-1)D+1; D]) \\ \vdots & \ddots & \vdots \\ \overline{\max}(X[(S-1)D+1, 1; D]) & \cdots & \overline{\max}(X[(S-1)D+1, (T-1)D+1; D]) \end{pmatrix}. \quad (11.36)$$

Thus the max pooling layer function reduces the dimension of the original matrix X by replacing each block matrix with the maximum element over that block.

Example 11.4.1 (Edge detection). Consider the convolution matrix $B = K * A$ obtained in (11.18). Represent the matrix B via 2×2 block matrices $\{X^{ij}\}_{i,j=1}^2$:

$$B = \begin{pmatrix} X[1, 1; 2] & X[1, 3; 2] \\ X[3, 1; 2] & X[3, 3; 2] \end{pmatrix} = \left(\begin{array}{cc|cc} 0 & 2 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ \hline 0 & 2 & 0 & 0 \\ 0 & 2 & 0 & 0 \end{array} \right). \quad (11.37)$$

Then apply max pooling to obtain the matrix

$$p(B) = \begin{pmatrix} 2 & 0 \\ 2 & 0 \end{pmatrix}, \quad (11.38)$$

in which only the largest element of each 2×2 submatrix is retained after max pooling.

The pooling layer reduces the complexity of a CNN by removing unimportant data at the cost of losing fine details of the features. For example, the upper left 2×2 block $X[1, 1; 2]$ in Example 11.4.1 is replaced by one number representing the feature. In the block matrix, we know that there is at least one edge, and we know the location between two white pixels in the left column and two black pixels in the right column. In (11.38), we only know that there is at least one edge, but we cannot easily reconstruct where it should be.

11.5 Building CNNs

A common architecture of CNNs can be described as follows: They typically consist of convolutional layers alternating with pooling layers with sometimes fully connected layers used before or after. See Figure 11.5 for a simple CNN with one convolutional layer, pooling layer, and fully connected layer, together with Figure 11.6 for a schematic representation of the various elements of a convolutional layer.

For simplicity of presentation, here we only consider CNNs from [40] that consist of a sequence of alternating convolutional layer c_i and pooling p_i followed by a fully connected layer $h(x)$. Thus we define a CNN as follows:

$$\phi(X) := h \circ p_K \circ c_K \circ \cdots \circ p_1 \circ c_1(X). \quad (11.39)$$

In practice, more general CNNs are used; see [37, 67]. In particular, padding layers are used in CNNs.

A basic idea behind this choice of architecture can be summarized as follows. Convolutional layers identify local features in an image via an output matrix whose large entries correspond to these features. Then pooling layers reduce the size of the convolutional output matrices by discarding their small entries. Finally, fully connected layers

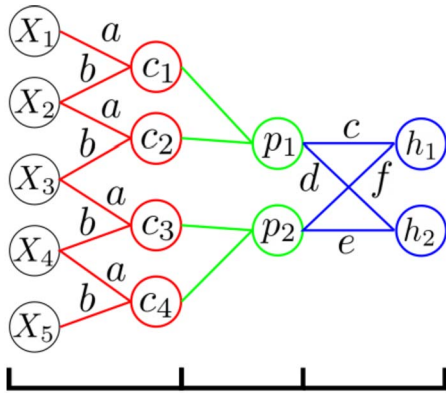


Figure 11.5: A simple CNN with three layers: convolutional (in red on the left), pooling (in green in the middle), and fully connected (in blue on the right).

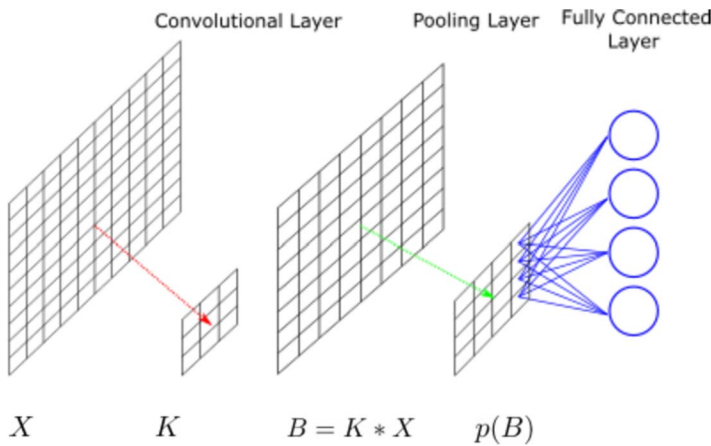


Figure 11.6: Schematic representation of convolutional matrix $c(X)$, kernel matrix K , pooling layer matrix $p(c(X))$, and the fully connected layer vector $h(p(c(X)))$.

make classifications based on the local features found via convolutional layers and their relative position.

For simplicity of presentation, we considered above only one kernel per convolutional layer. In applications multiple kernels are used, leading to so-called multi-channel convolutional layers. Each kernel corresponds to a different feature, which is learned simultaneously. A multi-channel convolutional layer may receive input in the form of several matrices (corresponding to several input channels) and provides as output several matrices (corresponding to several output channels).

Indeed, the input data are often multi-channel as well. In color images, for example, the color of each pixel may be described by the intensity of red, green, and blue in the pixel (this is called RGB encoding). Thus a single $N \times M$ image is represented by three matrices, one for the intensity of each color. Each matrix is called a *channel* of the image.

Audio data can be multi-channel as well; for example, stereo music contains one channel for the left ear and one for the right to provide a directional quality to the sound.

In such a case, the inputs and outputs of convolutional and pooling layers need to be vectorized.

Definition 11.5.1. Let $\{K_{ij}\}_{i=1\dots k, j=1\dots \ell}$ be $D_j \times D_j$ matrices of parameters with $k, \ell \geq 1$ positive integers corresponding to the number of input and output channels. A *multi-channel convolutional layer function* is a mapping $c : \mathbb{R}^{N \times M \times k} \rightarrow \mathbb{R}^{(N-D_1+1) \times (M-D_1+1) \times \mathbb{R}^{(N-D_2+1) \times (M-D_2+1)} \times \dots \times \mathbb{R}^{(N-D_\ell+1) \times (M-D_\ell+1)}}$ of the form

$$c(X) = \left(\bar{\lambda} \left(\sum_{i=1}^k K_{i1} * X_i \right), \bar{\lambda} \left(\sum_{i=1}^k K_{i2} * X_i \right), \dots, \bar{\lambda} \left(\sum_{i=1}^k K_{i\ell} * X_i \right) \right), \quad (11.40)$$

where the input is represented by k matrices $(X_i)_{i=1,\dots,k}$ of size $N \times M$.

Similarly, a multi-channel pooling layer is defined as follows.

Definition 11.5.2. Let $\{X_{ij}\}_{i=1}^\ell$ be ℓ matrices of dimensions $N_i \times M_i$ represented by $C_i \times D_i$ block matrices $\{X^{ij}\}$. Then the multi-channel pooling layer $\bar{p} : \mathbb{R}^{N_1 \times M_1} \times \mathbb{R}^{N_2 \times M_2} \times \dots \times \mathbb{R}^{N_\ell \times M_\ell}$ is defined as

$$\bar{p} = (p(X_1), p(X_2), \dots, p(X_\ell)), \quad (11.41)$$

where $N_i \times M_i = (N - D_i + 1) \times (M - D_i + 1)$ from Definition 11.5.1 when a pooling layer directly follows a convolutional layer.

11.6 Equivariance and invariance

A difficult but interesting question is what kind of invariance a DNN may possess. For example, the training set may contain images related via some transformation, e. g., one image may be obtained by rotating another. Then the invariance property or simply invariance would mean that these images would be automatically classified into the same class. In other words, the image is invariant (that is, it stays the same) under certain transformations.

On the other hand, a DNN classifier function may not be automatically invariant under rotation for all choices of parameters. We may wonder if there exists some choice of DNN parameters through a proper choice of architecture such as the number of layers, their widths, or connectivity of layers for which the network classifier function is invariant.

Understanding which DNN architecture leads to which type of invariance is hence important but also usually complex to answer. Deep convolutional networks in particular have been recognized to possess good invariance properties; see, for example, [2, 13, 38]. We also refer the reader to the recent approach to this question through the notion of scattering transform in [12, 43] and to the review [44]. In this subsection, we first

focus on the question of translational invariance. To this end, we first introduce a more general notion of equivariance, see Figure 11.7. Once translational equivariance is introduced and explained, we conclude with an example of a translationally invariant CNN.

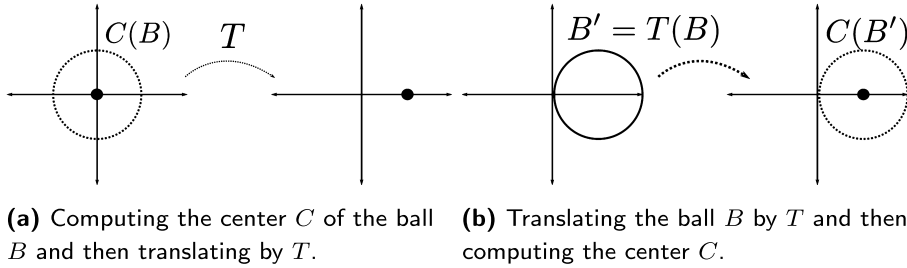


Figure 11.7: The center C of a ball B is equivariant under translations T , that is, $C \circ T(B) = T \circ C(B)$ for all balls $B \subset \mathbb{R}^2$.

A remarkable feature of CNN classifiers is that they are able to correctly classify translations of images. Several images can have the same local feature in different locations. For example, Figure 11.8 shows two images, each with the same pen in different locations. Whereas fully connected DNNs classify images based on all individual pixels in the image, CNNs learn local features in the image by using only local patches of pixels that correspond to these features. This lends CNNs a very useful robustness in image processing.



(a) A pen on the left.

(b) A pen on the right.

Figure 11.8: Two images, both of which contain a pen. In one image the pen is on the left, and in the other the pen is on the right. A convolutional layer will identify a pen in both images, unlike a fully connected layer, which would see the two images as entirely unrelated objects.

More specifically, convolutional layer functions $c(X)$ (see Definition 11.2.1) possess the property that when a feature (e.g., pen in Figure 11.8a) is shifted in the input image X , there is a corresponding shift in the output matrix $c(X)$. This property is called *translational equivariance*, which is described in a simple setting below.

To define translational equivariance for matrices, we first introduce the *horizontal and vertical shift operators* for matrices.

Definition 11.6.1. Let $\mathbb{R}^{N \times M}$ be the vector space of $N \times M$ matrices. The horizontal shift operator $T_h : \mathbb{R}^{N \times M} \rightarrow \mathbb{R}^{N \times M}$ is defined as

$$(T_h(A))_{ij} = \begin{cases} a_{ij-1}, & 2 \leq j \leq M, \\ a_{iM}, & j = 1. \end{cases} \quad (11.42)$$

The vertical shift operator $T_v : \mathbb{R}^{N \times M} \rightarrow \mathbb{R}^{N \times M}$ is defined similarly:

$$(T_v(A))_{ij} = \begin{cases} a_{i-1j}, & 2 \leq i \leq N, \\ a_{Nj}, & i = 1. \end{cases} \quad (11.43)$$

Simply put, $T_h(A)$ shifts each entry of A horizontally to the next column and moves the entries of A in the last column to the first column. Similarly, $T_v(A)$ shifts each entry of A vertically to the next row and moves the entries of the last row to the first row. Then translational equivariance is defined via the translation operators as follows.

Definition 11.6.2. Let $\mathbb{R}^{N_1 \times M_1}$ and $\mathbb{R}^{N_2 \times M_2}$ be the spaces of $N_1 \times M_1$ and $N_2 \times M_2$ matrices, respectively. A function $f : \mathbb{R}^{N_1 \times M_1} \rightarrow \mathbb{R}^{N_2 \times M_2}$ is translationally equivariant if

$$f \circ T_h(A) = T_h \circ f(A) \quad \text{and} \quad f \circ T_v(A) = T_v \circ f(A) \quad (11.44)$$

for each $A \in \mathbb{R}^{N_1 \times M_1}$.

We encourage the reader to compare this with the definition of translational invariance below in Eq. (11.56). We see from Definition 11.6.2 that f is translationally equivariant if and only if it commutes with the vertical and horizontal shift operators. This is because any translation in any direction can be decomposed into a composition of horizontal and vertical shifts.

For a given kernel K , the convolution layer function $c(X)$ as defined in Definition 11.2.1 is not translationally equivariant, but it is *almost* translationally equivariant in the following sense: for indices $i, j \geq 2$,

$$(c \circ T_h(X))_{ij} = (T_h \circ c(X))_{ij} \quad \text{and} \quad (c \circ T_v(X))_{ij} = (T_v \circ c(X))_{ij}, \quad (11.45)$$

that is, the convolutional layer commutes with the vertical and horizontal shift operators in all of its entries except for those in the first row ($i = 1$) and first column ($j = 1$).

On the other hand, pooling layers are not translationally equivariant. Note that pooling layers extract the feature corresponding to large entries from the convolution matrix at the cost of precise spatial information of the feature as in the following example.

Example 11.6.1 ($p(X)$ is not translationally equivariant). Consider the 4×4 matrix

$$X = \begin{pmatrix} 2 & 2 & 0 & 1 \\ 2 & 2 & 0 & 1 \\ 2 & 2 & 0 & 1 \\ 2 & 2 & 0 & 1 \end{pmatrix}. \quad (11.46)$$

Decomposing X into four 2×2 block matrices, after max-pooling, we obtain the matrix

$$p(X) = \begin{pmatrix} 2 & 1 \\ 2 & 1 \end{pmatrix}. \quad (11.47)$$

Now consider the horizontal shift of X ,

$$T_h(X) = \begin{pmatrix} 1 & 2 & 2 & 0 \\ 1 & 2 & 2 & 0 \\ 1 & 2 & 2 & 0 \\ 1 & 2 & 2 & 0 \end{pmatrix}. \quad (11.48)$$

After pooling, we obtain the different 2×2 matrix

$$p(T_h(X)) = \begin{pmatrix} 2 & 2 \\ 2 & 2 \end{pmatrix}. \quad (11.49)$$

Since $p(T_h(X)) \neq T_h(p(X))$, the pooling layer $p(X)$ is not translationally equivariant. Note, that if (11.49) were

$$\begin{bmatrix} 1 & 2 \\ 1 & 2 \end{bmatrix}, \quad (11.50)$$

then this would imply translational equivariance for the particular matrix X .

Example 11.6.2 (Fully connected layer $h_i(x)$ is not translationally equivariant). Consider the following matrix A and input vector v :

$$A = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{pmatrix}, \quad v = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}. \quad (11.51)$$

By straightforward calculation,

$$Av = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}. \quad (11.52)$$

If we consider v as a 3×1 matrix, then we may apply the vertical shift operator T_v and get

$$A(T_v(v)) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 2 \\ 0 \end{pmatrix}. \quad (11.53)$$

Again $T_v(Av) \neq A(T_v(v))$. This shows that the linear function $f_A(v) := Av$ defined on 3×1 matrices is not equivariant in the sense of Definition 11.6.2, that is,

$$f_A \circ T_v(v) \neq T_v \circ f_A(v). \quad (11.54)$$

To provide a connection with DNNs, we recall that the definition of a layer function contains an affine map

$$h_i(x) = \bar{\lambda}(Ax + b) \quad (11.55)$$

(see Definition 4.1.2). Since $h_i(x)$ is defined via multiplication by a matrix A of parameters, fully connected layer functions are not translationally equivariant in general either.

Equivariance of each layer is a desirable property because it can lead to the invariance of the classifier. Following Definition 11.6.2, we introduce the following:

Definition 11.6.3. Let $\mathbb{R}^{N \times M}$ be the space of $N \times M$ matrices. Let $f : \mathbb{R}^{N \times M} \rightarrow \{0, 1\}$ be a classifier function. Then the function f is translationally invariant if

$$f \circ T_h(A) = f(A) \quad \text{and} \quad f \circ T_v(A) = f(A) \quad (11.56)$$

for each $A \in \mathbb{R}^{N \times M}$.

Observe the difference with the definition of translational equivariance:

$$f \circ T_h(A) = f(A) \quad \text{vs.} \quad f \circ T_h(A) = T_h \circ f(A).$$

Translational invariance is a desirable property in image classification. For example, if a CNN ϕ is tasked with classifying images of cats and dogs (corresponding to classes 0 and 1, respectively), and A is an image corresponding to a dog, then the classifier ϕ should hopefully be invariant under both T_v and T_h :

$$\phi(T_h(A)) = \phi(T_v(A)) = \phi(A) = 1,$$

that is, the classifier would correctly classify an image of a dog whether it is on the left side of the image or on the right. Then we would naturally hope that if A is an image corresponding to a dog, then $T_h(A)$ is classified in the same way since it obviously corresponds to the same dog.

In practice, CNNs tend to behave well with respect to translations in many situations, even when they are not strictly speaking translational invariant. Furthermore, translational invariance cannot be ensured by just the use of some convolutional layers that are translationally equivariant or even almost translationally equivariant.

We now present a simple, somewhat artificial, example illustrating how we may use the translational equivariance of CNNs to build a network that is actually translationally invariant.

Example 11.6.3. We consider a dataset of two-dimensional gray-scale images with $N \times N$ pixels. Our input hence consists of some finite subset of $\mathbb{R}^{N \times N}$. Our goal is to classify the images in the dataset into κ given classes.

In this example, we do not have exact translational invariance of CNNs. The idea is to add padding layers so that the CNN convolution networks into an exact translational invariance. In other words, adding padding layers resolves the issue of the last row/column “jumping” to the first row/column in the translational shift operators.

We start our DNN by alternating d padding layer functions P_k with d convolutional functions c_k , $k = 1, \dots, d$, as given by Definition 11.2.1. We hence have d kernels K_k of dimension $D_k \times D_k$, whereas each padding layer P_k has the padding size $2s_k$. We begin from showing how a single convolutional layer $c(X)$, which is almost translationally equivariant, can be made translationally equivariant by using a padding layer $P(X)$ of width 1.

Lemma 11.6.1. *Let $X \in \mathbb{R}^{N \times N}$ be an input matrix with*

$$X_{i,N} = 0 \quad \text{for } 1 \leq i \leq N. \quad (11.57)$$

*Let $c(X) = \lambda(K * X + b)$ be a convolutional layer with $K \in \mathbb{R}^{D \times D}$ and bias vector $b = 0$, and let $P(X)$ be a padding layer of width $s \geq D$. Then the composition $(c \circ P)(X)$ is translationally equivariant.*

Proof. Equation (11.45) shows that the convolutional layer function $c(X)$ is almost translationally equivariant in the sense that

$$(c \circ T_h)_{ij} = (T_h \circ c)_{ij} \quad \text{for } 1 \leq i \leq N - D + 1, 2 \leq j \leq N - D + 1, \quad (11.58)$$

that is, all but the first column of the convolutional layer are the same after a horizontal translation.

We next show the translational equivariance of padding layers $P(X)$ of arbitrary width.

To this end, we observe that all the non-zero elements of X are preserved in $T_h(X)$, because by assumption the last column of X is 0. This implies that padding is equivariant with respect to horizontal translation T_h , that is, for each input X satisfying (11.57),

$$(P \circ T_h)(X) = (T_h \circ P)(X). \quad (11.59)$$

This idea behind relation (11.59) is illustrated by an example on Figure 11.9 for a 3×3 input matrix X and a padding layer of padding size $2s = 2$.

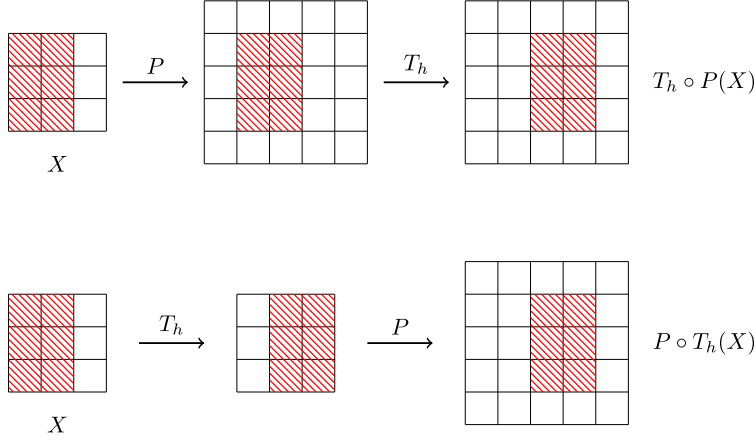


Figure 11.9: Translational equivariance of a padding layer of width $s = 1$. Red shaded squares correspond to nonzero entries, and white squares correspond to zero entries.

We next show that the composition $c \circ P$ is translationally equivariant. First, we use Eqs. (11.59) and (11.58) to show that $c \circ P$ is translationally equivariant with respect to horizontal translation, that is,

$$\begin{aligned} (c \circ P \circ T_h)_{ij} &= (c \circ T_h \circ P)_{ij} \\ &= (T_h \circ c \circ P)_{ij} \quad \text{for } 1 \leq i \leq N + 2s - D + 1, 2 \leq j \leq N + 2s - D + 1. \end{aligned} \quad (11.60)$$

It is sufficient to show that the first column of the matrix $(c \circ P)$ before and after translation is 0, that is, to show

$$(c \circ P \circ T_h)_{ij} = 0 \quad \text{and} \quad (T_h \circ c \circ P)_{ij} = 0 \quad \text{for } 1 \leq i \leq N + 2s - D + 1, j = 1. \quad (11.61)$$

By assuming that $s \geq D$ (that is, the width of the padding layer is larger than the dimension of the convolutional kernel) we have that the first column is 0:

$$(c \circ P \circ T_h(X))_{i,1} = 0 \quad \text{for } 1 \leq i \leq N + 2s - D + 1. \quad (11.62)$$

This is because the $D \times D$ convolutional kernel K is convolved with $s \geq D$ columns of zeros on the left of the matrix $T_h(X)$, leading to the zero first column of $(c \circ P \circ T_h(X))$.

For a similar reason in $(c \circ P(X))$, the kernel K is convolved with $s \geq D$ columns of zeros on the right of the matrix X , leading to the zero last column of $(c \circ P(X))$:

$$(c \circ P(X))_{i,N+2s-D+1} = 0 \quad \text{for } 1 \leq i \leq N + 2s - D + 1. \quad (11.63)$$

By the definition of T_h this implies that the first column of $(T_h \circ c \circ P(X))$ is 0:

$$(T_h \circ c \circ P(X))_{i,1} = 0 \quad \text{for } 1 \leq i \leq N + 2s - D + 1. \quad (11.64)$$

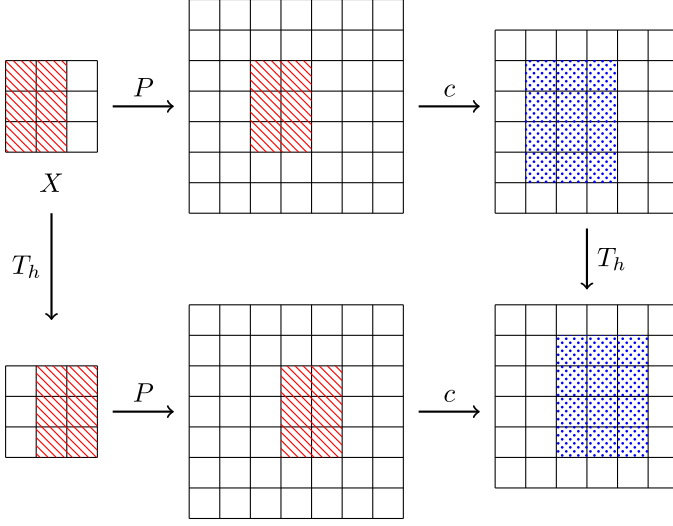


Figure 11.10: An $N \times N$ input X of size $N = 3$, padding size $2s = 4$, kernel with all non-zero entries of size $D = 2$. Since $s \geq D$ and the rightmost column of X is zero, $(c \circ P)(X)$ has the leftmost column zero (for any values of other entries), which shows (11.61). Padding size $2s = 2$ would not be sufficient as the leftmost column of $(c \circ P)(X)$ would not be zero.

Recall that (11.60) shows almost translational equivariance for all but the first column, and therefore (11.62) and (11.64) prove translational equivariance by showing that the first column of both $(T_h \circ c \circ P)$ and $(c \circ P \circ T_h)$ is 0, that is,

$$(T_h \circ c \circ P) = (c \circ P \circ T_h). \quad (11.65)$$

The idea behind relation (11.65) is illustrated by Figure 11.10 for a 3×3 input matrix X . $\square$

We can expand this simple example and obtain the translational equivariance of multiple compositions of convolution and padding layers.

Lemma 11.6.2. *The composition of d alternating convolution and padding layers*

$$c_d \circ P_d \circ c_{d-1} \circ P_{d-1} \circ \cdots \circ c_1 \circ P_1$$

is translationally equivariant.

Proof. The proof of the case of d -fold composition of $(c \circ P)$ is an extension of the argument of the previous proposition by induction. $\square$

Going back to the general case and starting with a matrix of size N , we apply the first padding function to obtain a matrix of size $N + 2s_1$. We follow with the convolutional function c_1 , yielding a matrix of size $N_1 = N + 2s_1 - D_1 + 1$. Each combination of a padding

function and a convolutional function hence yields matrices of size N_k with $N_0 = N$ and $N_k = N_{k-1} + 2s_k - D_k + 1$ for $k \geq 1$.

We could consider convolutional layers with various input or output channels, but to keep things as simple as possible, in this example, we assume that all convolutional layers, *except the last one*, only have one input and output channel. Our last convolutional layer for $k = d$ has one input channel but κ (the number of classes) output channels and is hence represented by κ kernels $K_{d,i}$ for $i = 1, \dots, \kappa$. We assume for simplicity that all kernels $K_{d,i}$ have the same dimension D_d .

Applied on a matrix $M_0 \in \mathbb{R}^{N \times N}$, the combination of the layer functions yields κ matrices $M^i(M)$, $i = 1, \dots, \kappa$, of dimension $N_d \times N_d$ with

$$N_d = N + 2s_1 + \dots + 2s_d - D_1 - \dots - D_d + d.$$

We then apply a max pooling layer. In this case, we take the simplest possible such layer by taking the maximum over the whole $N_d \times N_d$ matrix. In mathematical terms, to our κ matrices M^i , this max pooling layer associates κ numbers q_i for $i = 1, \dots, \kappa$ with

$$q_i = \max_{\alpha, \beta=1, \dots, N_d} M_{\alpha\beta}^i.$$

Here we took the simplest max pooling where the maximum is taken over the whole $N_d \times N_d$, but the results can be generalized to other variations of max pooling layers. This max pooling layer will change the translational equivariance of the previous padding and convolutional layers into translation invariance of this network with additional padding layers.

We finally apply a softmax layer function that transforms the vector $(q_1, \dots, q_\kappa)$ into a normalized probability distribution with formula $(p_1, \dots, p_\kappa) = (p_1(M), \dots, p_\kappa(M))$ for all $M \in \mathbb{R}^{N \times N}$.

Let us now check that the resulting classifier $(p_1(M), \dots, p_\kappa(M))$ is translationally invariant under some appropriate conditions on all parameters. Consider, for instance, the horizontal shift operator T_h . Then our goal is to show that $p_i(T_h(M)) = p_i(M)$ for all $i = 1, \dots, \kappa$.

We first assume that the initial images were themselves padded on the right so that $M_{\alpha, \beta} = 0$ for $\beta = N$ and all $1 \leq \alpha \leq N$. This ensures that $T_h(M)$ contains the same information as M : any feature present in M is still present in $T_h(M)$. It also ensures that padding the initial image is equivariant through T_h , that is, $P_1(T_h(M)) = T_h(P_1(M))$.

We may consequently use the translational equivariance of convolutional functions and deduce that

$$(c_1 \circ P_1 \circ T_h(M))_{\alpha\beta} = (T_h \circ c_1 \circ P_1(M))_{\alpha\beta} \quad \text{for all } \alpha \text{ and } \beta \geq 2. \quad (11.66)$$

Moreover, assume that $s_1 \geq D_1$ (that is, the padding is larger than the kernel size). Then we also immediately have

$$(c_1 \circ P_1 \circ T_h(M))_{\alpha,1} = 0 \quad \text{for all } \alpha.$$

For the same reason, we have

$$(c_1 \circ P_1(M))_{\alpha,N_1} = 0 \quad \text{for all } \alpha. \quad (11.67)$$

By the definition of T_h this implies

$$(T_h \circ c_1 \circ P_1(M))_{\alpha,1} = 0 \quad \text{for all } \alpha,$$

and therefore we obtain the strict translational equivariance

$$(c_1 \circ P_1 \circ T_h(M))_{\alpha\beta} = (T_h \circ c_1 \circ P_1(M))_{\alpha\beta} \quad \text{for all } \alpha, \beta.$$

We emphasize again that a padding layer with a large enough padding size $s_1 \geq D_1$ is critical for this property.

We also note that (11.67) shows that the last column of $c_1 \circ P_1(M)$ is composed only of 0, just as we had initially assumed for M . Therefore we can repeat the argument for the next padding and convolutional layer and obtain that

$$(c_2 \circ P_2 \circ c_1 \circ P_1 \circ T_h(M))_{\alpha\beta} = (T_h \circ c_2 \circ P_2 \circ c_1 \circ P_1(M))_{\alpha\beta} \quad \text{for all } \alpha, \beta,$$

provided that $s_2 \geq D_2$.

We can keep on repeating this argument on each combination of padding and convolutional layer until the last such layer with the κ output channels. The strict translational equivariance applies to each output channel, and we obtain that

$$(M^i \circ T_h(M))_{\alpha\beta} = (T_h \circ M^i(M))_{\alpha\beta} \quad \text{for all } \alpha, \beta, i,$$

provided that

$$s_k \geq D_k \quad \text{for all } k = 1, \dots, \kappa. \quad (11.68)$$

Finally, we note that the max pooling layer we are using is translation invariant: we can readily check from the definition of T_h that

$$\max_{\alpha,\beta=1,\dots,N_d} M_{\alpha\beta} = \max_{\alpha,\beta=1,\dots,N_d} (T_h M)_{\alpha\beta}$$

for every matrix M .

We can immediately deduce that $q_i \circ T_h(M) = q_i(M)$ and conclude that, as desired,

$$p_i \circ T_h(M) = p_i(M).$$

We finally note that, however, although translational invariance may be useful, there are in practice often compelling reasons to deviate from this network construction with

large number of padding layers, special max pooling, and no fully connected layers even if we lose the invariance. Constraints (11.68) on the padding size, which may be large, are demanding and lead to large layers, which may be, for example, too costly computationally. We may also obtain a better accuracy for the classifier by replacing the max pooling layer by a fully connected layer, which can be trained but is not translational invariant.

It is also important to observe that we only proved the invariance under a single shift T_h . To obtain the same result by application of n shifts $T_h, T_h \circ \dots \circ T_h$, we would need to ensure that the initial matrices M have their n last columns filled with 0, which is also much more demanding.

11.7 Summary of CNNs

To summarize this brief presentation of CNNs, we highlight the following:

1. *Convolution*: CNNs are obtained by alternating convolutional and other type of layers such as pooling or fully connected layers.
2. *Weight sharing*: Convolutional layers typically have far fewer weights than fully connected layers, which has several advantages from lowered computation time to less overfitting.
3. *Locality*: CNNs may find more easily local features. The convolutional layers extract the most important features of images by encoding them into large elements of their output matrices.
4. *Translational equivariance*: CNNs enjoy some robustness with respect to feature location through the almost translational equivariance of their convolution layers, see Figure 11.11.

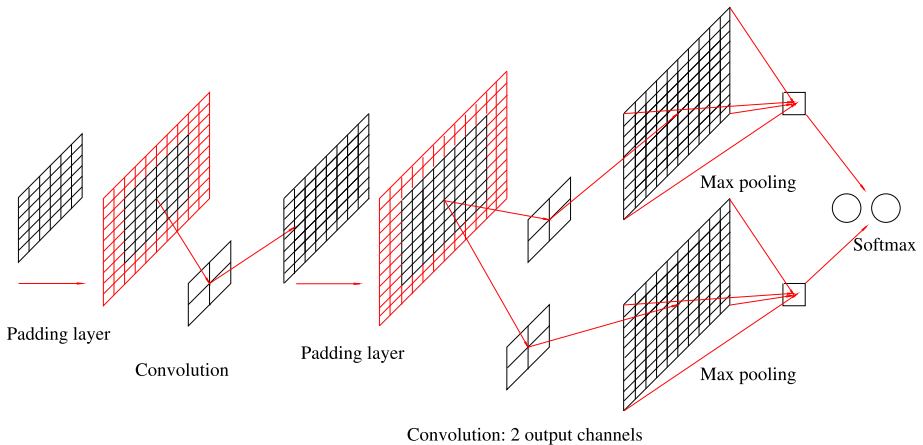


Figure 11.11: A representation of a simple network with translation invariance for two classes.

11.8 Exercises

Exercise 1 (Basic definition of convolutions). Consider a matrix M in $\mathbb{R}^{N \times M}$ and a matrix K in $\mathbb{R}^{D \times D}$. Recall the definition of convolution $K * A$ and the condition on the sizes D, N , and M for the operation to be valid.

Exercise 2 (Some examples of convolutions).

i. Calculate $K * M$ for

$$K = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}, \quad M = \begin{bmatrix} 2 & 2 & -2 \\ 2 & 2 & -2 \\ -2 & -2 & -2 \end{bmatrix}.$$

ii. Calculate $K * M$ for

$$K = \begin{bmatrix} 2 & -1 & -1 \\ -2 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix}, \quad M = \begin{bmatrix} 2 & 2 & 2 & -2 & -2 \\ 2 & 2 & 2 & 2 & -2 \\ 2 & 2 & 2 & 2 & 2 \\ 100 & 100 & 100 & 100 & 100 \end{bmatrix}.$$

Exercise 3 (Linearity of convolutions). Consider matrices A, B in $\mathbb{R}^{N \times M}$ and two kernels K, L in $\mathbb{R}^{D \times D}$ with $D \leq N$ and $D \leq M$.

i. Show that

$$(K + L) * A = K * A + L * A \quad \text{and} \quad K * (A + B) = K * A + K * B.$$

ii. For any $\lambda \in \mathbb{R}$, show that

$$(\lambda K) * A = K * (\lambda A) = \lambda (K * A).$$

Exercise 4 (Commuting convolutions).

i. Consider two continuous functions $f, g : \mathbb{R}^n \rightarrow \mathbb{R}$ with compact support. Show that for all $\mathbf{x} \in \mathbb{R}^n$,

$$(f * g)(\mathbf{x}) = (g * f)(\mathbf{x}).$$

ii. Consider a matrix A in $\mathbb{R}^{N \times N}$ and a matrix B in $\mathbb{R}^{D \times D}$. Show that we can calculate both convolutions $A * B$ and $B * A$ only if $N = D$.

iii. In the case where A, B are matrices in $\mathbb{R}^{N \times N}$, show that $A * B \in \mathbb{R}$ and $A * B = B * A$.

Exercise 5 (Some particular cases of convolutions). We consider a matrix M in $\mathbb{R}^{N \times N}$ with $N \geq 2$.

i. Consider a scalar $\lambda \in \mathbb{R}$ as a 1×1 matrix $K \in \mathbb{R}^{1 \times 1}$ with simply $K_{1,1} = \lambda$. Prove that

$$K * M = \lambda M, \quad \text{or} \quad (K * M)_{ij} = \lambda M_{ij} \quad \text{for all } 1 \leq i, j \leq N.$$

- ii. For any kernel $K \in \mathbb{R}^{D \times D}$ with $D \leq N$, show that $K * M = M$ if and only if $D = 1$ and $K_{1,1} = 1$.
- iii. Consider now the kernel $K \in \mathbb{R}^{2 \times 2}$ given by

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}.$$

Show that $K * M$ is equal to a block of M in the following sense:

$$(K * M)_{ij} = M_{ij} \quad \text{for all } 1 \leq ij \leq N - 1.$$

Exercise 6 (The identity matrix and convolutions). Consider a matrix M in $\mathbb{R}^{N \times M}$ and the identity matrix I in $\mathbb{R}^{D \times D}$ for $D > 1$, $D \leq N$, and $D \leq M$. Explain what is the expression for $I * M$ and show that $I * M$ is always different from M .

Exercise 7 (Padding is linear). Consider matrices $A, B \in \mathbb{R}^{N \times N}$, a scalar $\lambda \in \mathbb{R}$, and a padding layer function P with padding size $2s$. Show that

$$P(A + \lambda B) = P(A) + \lambda P(B).$$

Exercise 8 (Max pooling may lose some information).

- i. Consider a matrix $M \in \mathbb{R}^{N \times N}$ with all non-negative entries: $M_{ij} \geq 0$ for all $1 \leq ij \leq N$. Assume that $N = SD$ and consider further a max pooling layer function $p: \mathbb{R}^{N \times N} \rightarrow \mathbb{R}^{S \times S}$. Prove that if $p(M) = 0$, then $M = 0$.
- ii. Give an example of matrix $M \in \mathbb{R}^{6 \times 6}$, with possibly negative entries, such that $p(M) = 0$ but $M \neq 0$ for the max pooling layer function $p: \mathbb{R}^{6 \times 6} \rightarrow \mathbb{R}^{3 \times 3}$.

Exercise 9 (Max pooling is not linear). Consider matrices $A, B \in \mathbb{R}^{N \times N}$ with $N = SD$ and a max pooling layer function $p: \mathbb{R}^{N \times N} \rightarrow \mathbb{R}^{S \times S}$.

- i. For any $\lambda \geq 0$, show that

$$p(\lambda M) = \lambda p(M).$$

- ii. In the particular case $D = N$, show that

$$p(-M) = p(M)$$

only if $\min_{1 \leq i, j \leq N} M_{ij} = -\max_{1 \leq i, j \leq N} M_{ij}$.

- iii. For $N = 4$ and $D = 2$, find an example of matrices A, B such that

$$p(A + B) \neq p(A) + p(B).$$

Exercise 10 (Revisiting translational equivariance). Consider a matrix kernel K of size $s \times s$, which we will apply to gray-scale images via convolution.

- i. For an image represented by a matrix A of size $N \times N$, explain how to obtain the convolution $B = K * A$ and give its size.
- ii. Let D be a matrix of size $M \times M$ with $M < N$. Suppose that $N - M > 0$ is an even number, so that $N - M = 2t$ for an integer $t > 0$. Recall how we may transform the matrix D into a larger matrix C of size $N \times N$ by adding 0 in a symmetric manner.
- iii. For a matrix A of size $N \times N$, we define the matrix $\tau_{k_0, \ell_0} A$ by indices k_0, ℓ_0 as follows:

$$\tau_{k_0, \ell_0} A_{ij} = A_{i-k_0, j-\ell_0} \quad \text{for } \max 0, k_0 \leq i \leq \min(m, m + k_0) \\ \text{and } \max 0, \ell_0 \leq j \leq \min(n, n + \ell_0),$$

and $\tau_{k_0, \ell_0} A_{ij} = 0$ in all other cases. Explain why if A was obtained from a matrix D as in the previous question with $|k_0|, |\ell_0| \leq t$, then the image corresponding to $\tau_{k_0, \ell_0} A_{ij}$ is the translation of the image corresponding to A .

- iv. Assume again that the matrix A is obtained through a matrix D from question 2 with now $|k_0|, |\ell_0| \leq t$. Show that

$$K * (\tau_{k_0, \ell_0} A) = \tau_{k_0, \ell_0} (K * A).$$

Explain why this property is useful when doing image processing.

A Review of the chain rule

Recall that the derivative of a composition of two functions can be calculated using the following theorem.

Theorem A.0.1. *If $f, g : \mathbb{R}$ are differentiable functions, then $f \circ g : \mathbb{R} \rightarrow \mathbb{R}$ is also differentiable, and*

$$\frac{d}{dx} f \circ g(x) = \left. \frac{df}{dy}(y) \right|_{y=g(x)} \frac{dg}{dx}(x) = f'(g(x))g'(x). \quad (\text{A.1})$$

Theorem A.0.1 is called the *chain rule*, and it is central to backpropagation.

Recall from Definition 4.1.3 that DNNs are typically composed not of functions from $\mathbb{R}$ to $\mathbb{R}$ but of functions from $\mathbb{R}^{n_i}$ to $\mathbb{R}^{n_{i+1}}$, where $n_i, n_{i+1} \geq 1$. Therefore we introduce the multivariable chain rule.

Theorem A.0.2. *If $g : \mathbb{R}^{n_1} \rightarrow \mathbb{R}^{n_2}$ and $f : \mathbb{R}^{n_2} \rightarrow \mathbb{R}^{n_3}$ are differentiable, then $f \circ g : \mathbb{R}^{n_1} \rightarrow \mathbb{R}^{n_3}$ is also differentiable, and its partial derivatives are given by*

$$\frac{\partial}{\partial x_i} f \circ g(\mathbf{x}) = \left. \frac{\partial f}{\partial y_1}(\mathbf{y}) \right|_{\mathbf{y}=g(\mathbf{x})} \frac{\partial g_1}{\partial x_i}(\mathbf{x}) + \cdots + \left. \frac{\partial f}{\partial y_{n_2}}(\mathbf{y}) \right|_{\mathbf{y}=g(\mathbf{x})} \frac{\partial g_{n_2}}{\partial x_i}(\mathbf{x}) \quad (\text{A.2})$$

$$= \sum_{j=1}^{n_2} \left. \frac{\partial f}{\partial y_j}(\mathbf{y}) \right|_{\mathbf{y}=g(\mathbf{x})} \frac{\partial g_j}{\partial x_i}(\mathbf{x}). \quad (\text{A.3})$$

Observe that for each i , $\frac{\partial}{\partial x_i} f \circ g(\mathbf{x})$ is a vector with n_3 components because $f \circ g : \mathbb{R}^{n_1} \rightarrow \mathbb{R}^{n_3}$:

$$\frac{\partial}{\partial x_i} f \circ g(\mathbf{x}) = \begin{pmatrix} \frac{\partial}{\partial x_i} f_1 \circ g(\mathbf{x}) \\ \vdots \\ \frac{\partial}{\partial x_i} f_{n_3} \circ g(\mathbf{x}) \end{pmatrix}. \quad (\text{A.4})$$

We can arrange all the partial derivatives of $f \circ g$ into the columns of an $n_3 \times n_1$ matrix:

$$\frac{d}{dx} f \circ g(\mathbf{x}) = \begin{pmatrix} \frac{\partial}{\partial x_1} f_1 \circ g(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_{n_1}} f_1 \circ g(\mathbf{x}) \\ \vdots & & \vdots \\ \frac{\partial}{\partial x_1} f_{n_3} \circ g(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_{n_1}} f_{n_3} \circ g(\mathbf{x}) \end{pmatrix}. \quad (\text{A.5})$$

From (A.3) we see that the matrix $\frac{d}{dx} f \circ g(\mathbf{x})$ can be calculated via the following matrix multiplication:

$$\frac{d}{dx} f \circ g(\mathbf{x}) = \begin{pmatrix} \frac{\partial}{\partial x_1} f_1 \circ g(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_{n_1}} f_1 \circ g(\mathbf{x}) \\ \vdots & & \vdots \\ \frac{\partial}{\partial x_1} f_{n_3} \circ g(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_{n_1}} f_{n_3} \circ g(\mathbf{x}) \end{pmatrix} \quad (\text{A.6})$$

$$= \begin{pmatrix} \frac{\partial f_1}{\partial y_1}(\mathbf{y}) & \cdots & \frac{\partial f_1}{\partial y_{n_2}}(\mathbf{y}) \\ \vdots & & \vdots \\ \frac{\partial f_{n_3}}{\partial y_1}(\mathbf{y}) & \cdots & \frac{\partial f_{n_3}}{\partial y_{n_2}}(\mathbf{y}) \end{pmatrix} \bigg|_{\mathbf{y}=g(\mathbf{x})} \begin{pmatrix} \frac{\partial g_1}{\partial x_1}(\mathbf{x}) & \cdots & \frac{\partial g_1}{\partial x_{n_1}}(\mathbf{x}) \\ \vdots & & \vdots \\ \frac{\partial g_{n_2}}{\partial x_1}(\mathbf{x}) & \cdots & \frac{\partial g_{n_2}}{\partial x_{n_1}}(\mathbf{x}) \end{pmatrix}. \quad (\text{A.7})$$

Observe that the first matrix is $\frac{df}{dy}(\mathbf{y})|_{\mathbf{y}=g(\mathbf{x})}$, whereas the second is $\frac{dg}{dx}(\mathbf{x})$. Therefore

$$\frac{d}{dx} f \circ g(\mathbf{x}) = \frac{df}{dy}(\mathbf{y}) \bigg|_{\mathbf{y}=g(\mathbf{x})} \frac{dg}{dx}(\mathbf{x}), \quad (\text{A.8})$$

which has the same form as the single variable chain rule (A.1).

Bibliography

- [1] J. Alman and V. Williams. A refined laser method and faster matrix multiplication. In *32nd Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2021)*, 2021.
- [2] F. Anselmi, J. Leibo, L. Rosasco, J. Mutch, A. Tacchetti, and T. Poggio. Unsupervised learning of invariant representations. *Theoretical Computer Science*, 633:112–121, 2016.
- [3] P. Battle, M. Darcy, B. Hosseini, and H. Owhadi. Kernel methods are competitive for operator learning. *Journal of Computational Physics*, 496:112549, 2024.
- [4] L. Berlyand, T. Bourdais, H. Owhadi, and Y. Shmalo. Pruning deep neural networks via a combination of the Marchenko–Pastur distribution and regularization. arXiv:2503.01922, 2025.
- [5] L. Berlyand, R. Creese, and P.-E. Jabin. A novel multi-scale loss function for classification problems in machine learning. *Journal of Computational Physics*, 498:171–193, 2024.
- [6] L. Berlyand, P.-E. Jabin, and C. A. Safsten. Stability for the training of deep neural networks and other classifiers. *Mathematical Models and Methods in Applied Sciences*, 31:2345–2390, 2021.
- [7] L. Berlyand, O. Krupchytskyi, and V. Slavin. Random weights of DNNs and emergence of fixed points. arXiv:2501.04182, 2025.
- [8] C. M. Bishop and H. Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024.
- [9] B. E. Boser, I. M. Guyon, and V. N. Vapnik. A training algorithm for optimal margin classifiers. In *Proceedings of the Fifth Annual Workshop on Computational Learning Theory, COLT '92*, pages 144–152. Association for Computing Machinery, New York, NY, USA, July 1992.
- [10] L. Bottou. Large-scale machine learning with stochastic gradient descent. In *Proceedings of COMP-STAT '2010*, pages 177–186, 2010.
- [11] H. Brezis. *Functional Analysis, Sobolev Spaces and Partial Differential Equations*. Springer, New York, 2011.
- [12] J. Bruna and S. Mallat. Invariant scattering convolution networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35:1872–1886, 2013.
- [13] J. Bruna, A. Szlam, and Y. LeCun. Learning stable group invariant representations with convolutional networks. In *ICLR 2014*, page 13, 2014.
- [14] J. Buhmann and H. Kuhnel. Unsupervised and supervised data clustering with competitive neural networks. *IJCNN International Joint Conference on Neural Networks*, 4:796–801, 1992.
- [15] I. Campbell. An introduction to kernel methods. In R. J. Howlett and L. C. Jain, editors, *Radial Basis Function Networks: Design and Applications*, pages 155–192. Springer, Berlin, 2000.
- [16] K. Chaudhury, A. H. Ashok, S. Narumanchi, and D. Shankar. *Math and Architectures of Deep Learning*. Manning, Shelter Island, NY, 2024.
- [17] J. S. Cramer. The origins of logistic regression, 2002.
- [18] M. Deisenroth, A. Faisal, and C. Ong. *Mathematics for Machine Learning*. Cambridge University Press, 2020.
- [19] B. Després. *Neural Networks and Numerical Analysis*. De Gruyter, 2022.
- [20] M. Fauvel. Introduction to kernel methods: classification of multivariate data. *EAS Publications Series*, 77:171–193, 2016.
- [21] A. Fawzi, M. Balog, A. Huang, T. Hubert, B. Romera-Paredes, M. Barekatin, A. Novikov, F. J. R. Ruiz, J. Schrittwieser, G. Swirszcz, et al. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610(7930):47–53, 2022.
- [22] K. Fukushima. Neocognitron: a self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biological Cybernetics*, 36(4):193–202, Apr. 1980.
- [23] F. Galton. Kinship and correlation. *The North American Review*, 150(401):419–431, 1890.
- [24] M. Gaviano. *Some General Results on Convergence of Random Search Algorithms in Minimization Problems*. North Holland, Amsterdam, 1975.
- [25] X. Glorot, A. Bordes, and Y. Bengio. Deep sparse rectifier neural networks. In G. Gordon, D. Dunson and M. Dudík, editors, *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*. PMLR, Fort Lauderdale, FL, USA, 11–13 Apr. 2011, volume 15, pages 315–323, 2011.

- [26] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, Nov. 2016. Google-Books-ID: Np9SDQAAQBAJ.
- [27] G. Hinton and T. Sejnowski. *Unsupervised Learning: Foundations of Neural Computation*. MIT Press, 1999.
- [28] T. Hofmann, B. Schölkopf, and A. Smola. Kernel methods in machine learning. *The Annals of Statistics*, 36(3):1171–1220, 2008.
- [29] K. Hornik. Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2):251–257, Jan. 1991.
- [30] D. H. Hubel and T. N. Wiesel. Receptive fields of single neurones in the cat’s striate cortex. *The Journal of Physiology*, 148:574–591, Oct. 1959.
- [31] D. H. Hubel and T. N. Wiesel. Receptive fields, binocular interaction and functional architecture in the cat’s visual cortex. *The Journal of Physiology*, 160:106–154, Jan. 1962.
- [32] J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Židek, A. Potapenko, A. Bridgland, C. Meyer, S. A. A. Kohl, A. J. Ballard, A. Cowie, B. Romera-Paredes, S. Nikolov, R. Jain, J. Adler, T. Back, S. Petersen, D. Reiman, E. Clancy, M. Zielinski, M. Steinegger, M. Pacholska, T. Berghammer, S. Bodenstein, D. Silver, O. Vinyals, A. W. Senior, K. Kavukcuoglu, P. Kohli, and D. Hassabis. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873):583–589, Aug. 2021.
- [33] J. Kiefer and J. Wolfowitz. Stochastic estimation of the maximum of a regression function. *The Annals of Mathematical Statistics*, 23(3):462, 1952.
- [34] D. G. Kleinbaum, M. Klein. *Logistic Regression*. Statistics for Biology and Health, 3rd edition. Springer, New York, NY, 2010.
- [35] T. Kohonen. *Self-Organizing Maps*. Springer, Berlin, Heidelberg, 2001.
- [36] A. N. Kolmogorov and S. V. Fomin. *Elements of the Theory of Functions and Functional Analysis*. Dover, 1999.
- [37] A. Krizhevsky, I. Sutskever, and G. E. Hinton. ImageNet classification with deep convolutional neural networks. *Communications of the ACM*, 60(6):84–90, May 2017.
- [38] Y. LeCun, Y. Bengio, and G. Hinton. Deep learning. *Nature*, 521:436–444, 2015.
- [39] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel. Backpropagation applied to handwritten zip code recognition. *Neural Computation*, 1(4):541–551, Dec. 1989.
- [40] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel. Backpropagation applied to handwritten zip code recognition. *Neural Computation*, 1(4):541–551, 1989.
- [41] Y. LeCun, L. Bottou, G. Orr, and K. Müller. *Efficient BackProp*. Springer, Berlin Heidelberg, 2012.
- [42] B. Leonid, E. Sandier, S. Yitzchak, and L. Zhang. Enhancing accuracy in deep learning using random matrix theory. *Journal of Machine Learning*, 3(4):347–412, 2024.
- [43] S. Mallat. Group invariant scattering. *Communications on Pure and Applied Mathematics*, 65:1331–1398, 2012.
- [44] S. Mallat. Understanding deep convolution networks. *Philosophical Transactions of the Royal Society A*, 374:20150203, 2016.
- [45] D. Marr and T. Poggio. A computational theory of human stereo vision. *Proceedings of the Royal Society of London. Series B, Biological Sciences*, 204(1156):301–328, 1979.
- [46] W. McCulloch and W. Pitts. A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, 5:115–133, 1943.
- [47] M. Minsky and S. Papert. *Perceptrons: An Introduction to Computational Geometry*. The MIT Press, Cambridge, 1988.
- [48] K. P. Murphy. *Probabilistic Machine Learning: An Introduction*. The MIT Press, 2022.
- [49] Y. Nesterov. *Introductory Lectures on Convex Optimization: A Basic Course*. Springer, New York, NY, 2004.
- [50] P. B. Palmer and D. G. O’Connell. Regression analysis for prediction: understanding the process. *Cardiopulmonary Physical Therapy Journal*, 20(3):23, 2009.

- [51] A. Pinkus. Approximation theory of the MLP model in neural networks. *Acta Numerica*, 8:143–195, 1999. arXiv:1910.13029.
- [52] A. Pinkus. Weierstrass and approximation theory. *Journal of Approximation Theory*, 107(1):1–66, Nov. 2000.
- [53] B. Polyak. *Introduction to Optimization*. Optimization Software, Inc., New York, 1987.
- [54] N. Qian. On the momentum term. In *Gradient Descent Learning Algorithms*, 1999.
- [55] H. Robbins and S. Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400, 1951.
- [56] F. Rosenblatt. The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological Review*, 10(9):386–408, 1958.
- [57] W. Rudin. *Functional Analysis*. McGraw-Hill, 1973.
- [58] D. J. Santry. *Demystifying Deep Learning: An Introduction to the Mathematics of Neural Networks*. IEEE Press, 2024.
- [59] I. J. Schoenberg. Metric spaces and completely monotone functions. *Annals of Mathematics*, 39(4):821, 1938.
- [60] B. Schölkopf, R. Herbrich, and A. Smola. A generalized representer theorem. In *Computational Learning Theory*. Lecture Notes in Computer Science, volume 2111, pages 416–426, 2001.
- [61] B. Schölkopf, A. J. Smola, F. Bach, et al. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press, 2002.
- [62] T. Serre, L. Wolf, and T. Poggio. Object recognition with features inspired by visual cortex. In *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)*, volume 2, pages 994–1000, June 2005. ISSN 1063-6919.
- [63] J. Spall. *Introduction to Stochastic Search and Optimization: Estimation, Simulation, and Control*, Wiley, NJ, 2003.
- [64] J. F. Steffensen. *Interpolation*, 2nd edition. Dover Books on Mathematics. Dover Publications, Mineola, NY, Mar. 2006.
- [65] G. Strang. The functions of deep learning. *SIAM News*, 51(10), 2018.
- [66] V. Strassen. Gaussian elimination is not optimal. *Numerische Mathematik*, 13(4):354–356, 1969.
- [67] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. Going deeper with convolutions. arXiv:1409.4842 [cs], Sept. 2014.
- [68] V. Vapnik. Pattern recognition using generalized portrait method. *Automation and Remote Control*, 24:774–780, 1963.
- [69] V. Vapnik and A. Chervonenkis. *Theory of Pattern Recognition*, 1974.
- [70] G. U. Yule. On the theory of correlation. *Journal of the Royal Statistical Society*, 60(4):812–854, 1897.
- [71] A. Zhang, Z. C. Zachary, C. Lipton, M. Li, and A. J. Smola. *Dive into Deep Learning*. Cambridge University Press, 2023.

Index

- activation function 7, 15
- additive loss function 75
- affine hyperplane 43
- affine map 14
- ANN classifier 11
- architecture of the ANN 11
- artificial intelligence 3
- artificial neuron network 20

- backpropagation 86, 98, 102
- backward calculation 95
- batch 76
- bias 7
- block matrix 125

- chain rule 86, 91
- class 5
- classification problem 5
- clustering problem 22
- computational cost, complexity 76, 86
- convex hull 45
- convolution 112
- convolution of functions 112
- convolution of matrices 112
- convolutional layer 112, 120
- convolutional layer function 120
- convolutional neural network (CNN) 112, 126
- convolutional neuron function 121
- critical point 71
- cross-entropy loss 76
- cross-entropy loss function 32

- decision boundary 33
- deep learning 7
- deep neural network 11
- depth of the ANN 11
- derivative 72
- deterministic gradient descent (GD) 71, 72
- diagrammatic interpretation, representation 92
- dot or inner product 7

- elementary functions 93
- epoch 77
- equivariance 128
- exact classifier 5

- feature detection 115
- feature map 61
- feature space 61
- feedforward networks 7
- forward propagation 94, 103
- fully connected layer 108

- gradient 72
- gradient descent 71

- Hadamard product 114
- Hahn–Banach theorem 43
- handwriting recognition 22
- hard classifier 5
- hard margin SVM 46
- hidden layer 10
- hinge loss function 49
- hyperparameter 50

- input layer 9
- invariance 128

- kernel, Gaussian 68
- kernel, linear 56
- kernel function, definition 66
- kernel methods 54
- kernel trick 66

- layer function 8
- layer width 10
- learning rate 73
- least squares minimization 27
- level set 73
- limit 88
- linear decision boundary 45
- linear regression 27
- Lipschitz function 75
- logistic function 31
- logistic model 30
- logistic regression 30

- machine learning 3
- margin 44
- matrix multiplication 90
- matrix of parameters 9
- max pooling 112, 125
- max pooling layer function 125
- maximum 49, 71
- minimum 71

- multi-channel 127
- multi-channel convolutional layer function 128
- multi-channel pooling layer 128
- neuron function 7
- $O()$, Landau symbol 88
- object 5
- output layer 11
- padding layer 123
- partial derivative 94
- pooling layer 112, 124
- Rectified Linear Unit (ReLU) 8
- regression 27
- regression curve 27
- regression model 30
- regression problem 27
- regularized average hinge loss function 49
- ridge regression 58
- ridge regression, dual formulation 60
- saddle point 71
- semisupervised learning 22, 24
- separating hyperplane 21
- shift operator 130
- sigmoid function 31
- soft approximate classifier 6
- soft classifier 5
- soft margin SVM 46
- softmax function 11
- stationary point 71
- step size 73
- stochastic gradient descent (SGD) 71, 76
- submatrix 115
- supervised learning 22, 24
- support vector machines (SVMs) 45
- support vectors 47
- Taylor formula 72
- training set 5
- translation 130
- translational equivariance 129
- translational invariance 129, 132
- universal approximation theorem 13
- unsupervised learning 22, 24
- vanishing gradient 36
- vector of biases 9
- vectorial activation function 10
- weights 7, 78